

E-CLASSROOM V 3.0

HIRA ATIF (BCS211887)

ABDUL MOIZ (BCS203242)

ALLA UD DIN (BCS203170)



Spring-2024

Supervised By

DR. MUHAMMAD SIRAJ RATHORE

Department of Computer Science

Capital University of Science & Technology, Islamabad

—

Submission Form for Final-Year PROJECT REPORT



Version	V 3.0	NUMBER OF MEMBERS	3
TITLE	E-CLASSROOM		
SUPERVISOR NAME	DR. MUHAMMAD SIRAJ RATHORE		

MEMBER NAME	REG. NO.	EMAIL ADDRESS
HIRA ATIF	BCS211887	hiraatif45@gmail.com
ALLA UD DIN	BCS203170	khanallauddin113@gmail.com
ABDUL MOIZ	BCS203242	Bmoiz977@gmail.com

MEMBERS' SIGNATURES	Supervisor's Signature

Signatures:

Hira Atif

Alla Ud Din

Abdul Moiz

Supervisor

APPROVAL CERTIFICATE

This project, entitled as “E-Classroom” has been approved for the award of

Bachelors of Science in Computer Science

Committee Signatures:

Supervisor: _____

(Dr. Muhammad Siraj Rathore)

Project Coordinator: _____

(Mr. Bilal Ahmed)

Head of Department: _____

(Dr. Abdul Basit)

Approvals:

Supervisor

Project Coordinator

Head of Department

DECLARATION

We, hereby, declare that “No portion of the work referred to, in this project has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning”. It is further declared that this undergraduate project, neither as a whole nor as a part thereof has been copied out from any sources, wherever references have been provided.

MEMBERS' SIGNATURES

Declaration Signatures:

Hira Atif

Alla Ud Din

Abdul Moiz

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude and appreciation to all those who have contributed to the successful completion of this project. Their assistance, guidance, and support have been invaluable throughout the journey. First and foremost, we would like to extend our heartfelt thanks to our supervisor, Dr. Siraj, for his unwavering support and guidance. His expertise, feedback, and dedication played a crucial role in shaping this project and ensuring its successful execution. We are truly grateful for the time and effort he invested in overseeing our progress and providing valuable insights. Additionally, we extend our appreciation to Mr. Salman Ahmed, Ms. Memona Bilal, Mr. Abdul Haseeb for their support and assistance, whether it was through providing resources, offering guidance on specific aspects, or simply being a source of encouragement throughout the project. Their input has been instrumental in overcoming challenges and achieving the desired outcomes.

EXECUTIVE SUMMARY

E-Classroom is an innovative online platform designed to revolutionize education collaboration. Serving as a dynamic hub where educators, students, and enthusiasts can connect, learn, and share knowledge effortlessly, E-Classroom transcends traditional classroom limitations by offering a virtual environment accessible from anywhere in the world. This web-based application facilitates real-time interaction and collaboration among teachers and students, supported by an intuitive user interface, comprehensive educational tools, and robust security measures. Built on modern web technologies like React.js for the frontend and Firebase for the backend, E-Classroom ensures a responsive, seamless experience for users navigating the platform.

React.js powers the frontend, enabling a user-friendly interface that facilitates interactive lessons, real-time discussions, and efficient content management. Its component-based architecture supports rapid development and maintenance of features, ensuring smooth interactions and updates. On the backend, Firebase handles server-side operations, providing scalability, high performance, and real-time data synchronization. Firebase's secure authentication and database management capabilities enhance data integrity and user privacy, crucial for maintaining a safe learning environment.

E-Classroom offers a comprehensive feature set, including advanced teaching tools, collaborative project spaces, and extensive educational resources. Teachers can create virtual classrooms, manage student enrollment, distribute assignments, and track student progress seamlessly. Students benefit from access to learning materials, assignment submission portals, and interactive learning activities, fostering engagement and knowledge sharing. Designed for scalability and reliability, E-Classroom supports a growing user base while ensuring secure access and data management. It empowers educators and students alike to collaborate effectively, explore new learning opportunities, and thrive in a digital educational landscape.

TABLE OF CONTENTS

Contents

CHAPTER 1	16
INTRODUCTION	16
1.1. PROJECT INTRODUCTION	16
1.1.1. THE VISION	16
1.1.2. PROBLEM STATEMENT	17
1.1.3. WHY WE NEED THIS TYPE OF APP?	17
1.1.4. MODULE	18
1.2. EXISTING EXAMPLES/ SOLUTIONS	19
1.2.1. ZOOM	19
1.2.2. GOOGLE MEET	20
1.2.3. MICROSOFT TEAMS	20
1.2.4. CISCO WEBEX	21
1.3. BUSINESS SCOPE	23
1.4. REMOTE WORK TRENDS	24
1.5. PROJECT WORK BREAK DOWN	26
1.6. PROJECT TIME LINE	26
CHAPTER 2	27
REQUIREMENT SPECIFICATION AND ANALYSIS	27
2.1. FUNCTIONAL REQUIREMENTS	27
2.2. NON-FUNCTIONAL REQUIREMENTS	28
2.3. SYSTEM USE CASE MODELING	29
2.4. USE CASE DESCRIPTION	30
2.5. SYSTEM SEQUENCE DIAGRAMS	47
2.6. DOMAIN MODEL	55
CHAPTER 3	56
SYSTEM DESIGN	56
3.1. SOFTWARE ARCHITECTURE	56
3.2. CLASS DIAGRAM	57
3.3. SEQUENCE DIAGRAM	58
3.2. ENTITY RELATIONSHIP DIAGRAM	59
3.5. DATABASE SCHEMA	60
3.6. USER INTERFACE DESIGN	61

3.7. SOFTWARE COTS	70
CHAPTER 4.....	72
SOFTWARE DEVELOPMENT	72
4.1. CODING STANDARDS.....	72
4.2. DEVELOPMENT ENVIRONMENT	72
4.2.1 VISUAL STUDIO CODE	72
4.2.2. VISUAL STUDIO CODE	73
4.3. SOFTWARE DESCRIPTION.....	73
4.3.1 LOGIN PROCESS	73
4.3.2 MAKE PROFILE.....	74
4.3.3. UPDATE PROFILE.....	75
4.3.4. USER (STUDENT, TEACHER) PROFILE.....	76
4.3.5 CLASS	77
4.3.6 CREATE BUTTON	77
4.3.7 CREATED CLASS SECTION	78
4.3.8 SEND NOTIFICATION	79
4.3.9. SEARCH STUDENT BY REG NO:.....	81
4.3.10 TEACHER CAN UPLOAD ASSIGNMENT	82
4.3.11 STUDENT CAN DOWNLOAD ASSIGNMENT	83
4.3.12 USER CAN START CLASS	84
4.3.13. USER CAN JOIN CLASSROOM.....	85
4.3.14 USER CAN MUTE MICROPHONE/ TOGGLE CAMERA.....	86
4.3.15 TEACHER CAN REMOVE STUDENT	87
4.3.16 USER CAN SHARE SCREEN	88
4.3.17 STUDENT CAN CREATE AND SAVE NOTES	89
4.3.18 STUDENT CAN VIEW NOTES	90
4.3.19 STUDENT CAN DELETE NOTES.....	91
4.3.20 TEACHER CREATE GROUPS	92
4.3.21 USER CAN VIEW GROUPS	93
4.3.22 USER CAN START CHATTING.....	94
4.3.23 TEACHER SENDS RANDOM QUESTION TO STUDENT	95
4.3.24 TEACHER SENDS RANDOM ATTENDANCE TO STUDENT.....	96
CHAPTER 5.....	98
SOFTWARE TESTING.....	98
5.1 TESTING METHODOLOGY	98
5.2.1 TEACHER TEST CASES:.....	98

5.2.1.1	LOGIN	98
5.2.1.2	REGISTER.....	99
5.2.1.3	LOGOUT	100
5.2.1.4	UPDATE PROFILE.....	100
5.2.1.6	CREATE CLASSROOM	101
5.2.1.7	CREATED CLASSROOM	102
5.2.1.8	TEACHER START CLASS.....	103
5.2.1.9	UPLOAD ASSIGNMENT QUESTION	103
5.2.1.10	TEACHER CAN MUTE MICROPHONE.....	104
5.2.1.11	TEACHER CAN TOGGLE HIS/HER CAMERA	104
5.2.1.12	TEACHER CAN TOGGLE SCREEN	105
5.2.1.13	TEACHER CAN EXIT CLASS	105
5.2.1.14	TEACHER CAN DOWNLOAD ASSIGNMENT SOLUTION.....	106
5.2.1.15	TEACHER CREATE GROUPS	106
5.2.1.16	TEACHER TYPE AND VIEW GROUP CHATS	107
5.2.1.17	TEACHER CAN REMOVE STUDENTS	107
5.2.1.18	TEACHER SEND RANDOM ATTENDANCE	108
5.2.1.19	TEACHER VIEW MARKED ATTENDANCE	108
5.2.1.20	TEACHER START RANDOM QUESTION	109
5.2.1.21	TEACHER CAN VIEW SUBMITTED ANSWERS	109
5.2.2	STUDENT TEST CASES	110
5.2.2.1	LOGIN	110
5.2.2.2	REGISTER.....	111
5.2.2.3	LOGOUT	112
5.2.2.4	UPDATE PROFILE.....	112
5.2.2.5	NOTIFICATION.....	113
5.2.2.6	STUDENT START CLASS.....	114
5.2.2.7	STUDENT JOIN CLASS.....	114
5.2.2.8	STUDENT MUTE MICROPHONE	115
5.2.2.9	STUDENT TOGGLE CAMERA.....	115
5.2.2.10	STUDENT CAN TOGGLE SCREEN	116
5.2.2.11	STUDENT CAN EXIT CLASSROOM	116
5.2.2.12	STUDENT DOWNLOAD ASSIGNMENT QUESTION	117
5.2.2.13	STUDENT UPLOAD ASSIGNMENT SOLUTION	117
5.2.2.14	STUDENT TYPE AND VIEW GROUP CHAT	118
5.2.2.15	STUDENT CAN CREATE NOTES	118
5.2.2.16	STUDENT VIEW NOTES.....	119

5.2.2.17 STUDENT CAN DELETE NOTES.....	119
5.2.2.18 STUDENT RECEIVE RANDOM ATTENDANCE POPUP.....	120
5.2.2.19 STUDENT RECEIVE RANDOM QUESTION POPUP	120
CHAPTER 6	121
SOFTWARE DEPLOYMENT	121
6.1. INSTALLATION / DEPLOYMENT PROCESS DESCRIPTION	121
6.2. CONCLUSION	126
CHAPTER 7	127
PROJECT EVALUATION	127
7.1. PROJECT EVALUATION REPORT	127
REFERENCES	129

LIST OF FIGURES

Figure 1: project work breaks down	26
Figure 2: project time line.....	26
Figure 3: system use case modeling.....	29
Figure 4: SSD of teacher registration.....	47
Figure 5: SSD of teacher registration	47
Figure 6: SSD of student login.....	48
Figure 7: SSD of teacher login	48
Figure 8: SSD of create classroom	49
Figure 9: SSD of add students in classroom	49
Figure 10: SSD of teacher making groups of students during class.....	50
Figure 11: SSD of student join class.....	50
Figure 12: SSD of teacher create assignment.....	51
Figure 13: SSD of student submit assignment solution.....	51
Figure 14: SSD of student receiving notification before class.....	52
Figure 15: SSD of random attendance.....	52
Figure 16: SSD of student create notes.....	53
Figure 17: SSD of student view notes	53
Figure 18: SSD of remove student from class.....	54
Figure 19: SSD of random questions	54
Figure 20: Domain Model of the system.....	55
Figure 21: Software Architecture.....	56
Figure 22: Class Diagram	57
Figure 23: Sequence Diagram	58
Figure 24: Entity Relationship Diagram	59
Figure 25: Database Schema.....	60
Figure 26: UI of student login.....	61
Figure 27: UI of student signup.....	61
Figure 28: UI of student dashboard.....	62
Figure 29: UI of student add profile	62
Figure 30: UI of student view profile.....	63
Figure 31: UI of student update profile	63
Figure 32: UI of student view updated profile.....	64
Figure 33: UI of student create and view notes.....	64
Figure 34: UI of student upload assignment solution.....	65
Figure 35: UI of student type and view group chat	65
Figure 36: UI of student receiving random attendance and random question popups.....	66
Figure 37: UI of teacher signup.....	66
Figure 38: UI of teacher login.....	67
Figure 39: UI of teacher dashboard	67
Figure 40: UI of teacher creating groups of students.....	68
Figure 41: UI of teacher scheduling class	68
Figure 42: UI of teacher checking students' engagement status and can remove students.....	69
Figure 43: UI of teacher uploading assignment	69

<i>Figure 44: UI of teacher start class.....</i>	<i>69</i>
<i>Figure 45: UI of teacher sending random attendance popup to student</i>	<i>70</i>
<i>Figure 46: GitHub repository and project upload.....</i>	<i>123</i>
<i>Figure 47: vercel login.....</i>	<i>123</i>
<i>Figure 48: vercel project upload</i>	<i>124</i>
<i>Figure 49: import repository.....</i>	<i>124</i>
<i>Figure 50: configuration and deployment</i>	<i>125</i>
<i>Figure 51: website live for end users.....</i>	<i>125</i>

LIST OF TABLES

Table 1: existing solutions features comparison	22
Table 2: remote work trend nowadays	24
Table 3: functional requirements of the system	27
Table 4: non-functional requirements of the system	28
Table 5: UC-01 register	30
Table 6: UC-02 user login.....	31
Table 7: UC-03 user logout	32
Table 8: UC-04 create classroom	33
Table 9: UC-05 add students	34
Table 10: UC-06 create assignment.....	35
Table 11: UC-07 create class material	36
Table 12: UC-08 create announcement	37
Table 13: UC 09 view attendance report.....	38
Table 14: UC-10 mute microphone.....	39
Table 15: UC-11 create groups during class.....	40
Table 16: UC-12 join classroom.....	41
Table 17: UC-13 student can upload assignment	42
Table 18: UC-14 student view notes	43
Table 19: UC-15 view class notification.....	44
Table 20: UC-16 start random attendance	45
Table 21: UC-17 create notes	46
Table 22: test-case 01 teacher login	98
Table 23: test-case 02 teacher login	99
Table 24: test-case 03 teacher register	99
Table 25: test-case 04 teacher logout	100
Table 26: test-case 05 teacher update profile	100
Table 27: test-case 06 teacher update profile	101
Table 28: test-case 07 teacher create classroom	101
Table 29: test-case 08 teacher create classroom	102
Table 30: test-case 09 teacher can see created classroom	102
Table 31: test-case 10 teacher start class	103
Table 32: test-case 11 teacher upload assignment question.....	103
Table 33: test-case 12 teacher can mute microphone	104
Table 34: test-case 13 teacher can toggle camera.....	104
Table 35: test-case 14 teacher can toggle screen	105
Table 36: test-case 15 teacher can exit class.....	105
Table 37: test-case 16 teacher can download assignment solution	106
Table 38: test-case 17 teacher can create groups.....	106
Table 39: test-case 18 teacher can type can view group chats.....	107
Table 40: test-case 19 teacher can remove students.....	107
Table 41: test-case 20 teacher send random attendance to students	108
Table 42: test-case 21 teacher can view marked attendance of the students	108
Table 43: test-case 22 teacher start random questions.....	109
Table 44: test-case 23 teacher can view submitted answers of the students.....	109
Table 45: test-case 24 student login into system.....	110

Table 46: test-case 25 student login into system.....	110
Table 47: test-case 26 student register.....	111
Table 48: test-case 27 student register	111
Table 49: test-case 28 student logout.....	112
Table 50: test-case 29 student update profile	112
Table 51: test-case 30 student receive notification	113
Table 52: test-case 31 student receive notification	113
Table 53: test-case 32 student start class.....	114
Table 54: test-case 33 student join class.....	114
Table 55: test-case 34 student can mute microphone	115
Table 56: test-case 35 student can toggle their camera during class	115
Table 57: test-case 36 student can toggle screen.....	116
Table 58: test-case 37 students can exit classroom	116
Table 59: test-case 38 student can download assignment question.....	117
Table 60: test-case 39 student can upload assignment solution	117
Table 61: test-case 40 student can type and view group chat.....	118
Table 62: test-case 41 student can create notes.....	118
Table 63: test-case 42 student can view their created notes.....	119
Table 64: test-case 43 student can delete notes	119
Table 65: test-case 44 student receive random attendance popup	120
Table 66: test-case 45 student receive random question popup	120
Table 68: artifacts guidelines.....	127

Chapter 1

Introduction

The world has experienced a significant shift in the previous days to online tasks and communication due to technological advances and the severe pandemic, including Industries and pervasive computing. However, this shift has led to a narrowing global attention span, impacting education. In response, E-Classroom proposes a comprehensive online educational system to address these challenges. It includes features such as tracking student attention, assessing knowledge comprehension, encouraging engagement through randomized attendance checks and providing a good environment for the teacher as well as the students in order to enhance teaching experience. The system aims to provide an effective online learning environment and ensure students' attentiveness while allowing students to rate their understanding and instructors' responses at the end of each lecture.

1.1. Project Introduction:

Over the recent years humans have had the rare opportunity to test and scrutinize the efficiency and adaptability of many of the systems that had been working. Fortunately, or not, this pandemic has pointed out too many flaws in our technologies when the capabilities of our systems are strained in dynamic conditions where all the normal life proceedings cease to function.

E-learning is a learning approach that is based on formalized instruction but uses electronic resources. E-learning is a rapidly expanding technology. This marvelous remote education technology, which has the potential to democratize and revolutionize teaching- learning, has undergone a tremendous transformation in the two decades since it was first introduced to the public. Computer based training (CBT), which allowed learners to access study materials saved on CD-ROMs, was the first teaching-learning innovation of the IT (information technology) industry, which bloomed in the mid-1980s. It has been rapidly changing and spreading since then. However, the COVID-19 has heightened the need and necessity of online learning and teaching. The COVID-19 has caused schools all across the world to close.

As a result, education has undergone significant transformations, with the rise of e-learning, in which instruction is done remotely and via digital platforms

1.1.1. The Vision:

At the heart of this project lies a vision of democratizing education, breaking down geographical barriers, and providing individuals of all ages and backgrounds with unparalleled access to knowledge. We believe that learning should be engaging, interactive, and tailored to the unique needs of each learner. Our E- classroom platform seeks to redefine education by delivering a comprehensive suite of features and services that cater to diverse educational needs.

1.1.2. Problem Statement:

In an era defined by digital transformation and remote connectivity, traditional methods of education have been challenged like never before. The COVID-19 pandemic accelerated the shift toward online learning, underscoring both the opportunities and limitations of existing platforms. While platforms like Microsoft Teams, Google Classroom, and many others have played a vital role in enabling remote education, there remain critical challenges and unmet needs that call for the creation of a new, innovative E-classroom app. The problem that is being addressed through this system is the lack of an adaptive online education system where the teacher nor the student are able to physically communicate with each other and there is no such authority to oversee and monitor factors like teacher-student interaction, student focus and knowledge delivery. The current online educational systems no doubt provide capabilities to both parties to communicate with each other. But, student-teacher communication is not something to be taken lightly as it is the one that will help shape up the future of this world. But, when there is no system to monitor this interaction for effective knowledge delivery, and to ensure student focus and knowledge retention, this is where the problem arises. This project is conceptualized with the above-mentioned problems of attention and focus ensuring, effective knowledge delivery and lack of a complete physical classroom fabrication to make the students conscious of the fact that they are being constantly monitored and their activities do not go unwarranted during an online lecture.

1.1.3. Why We Need This Type of App?

We need an E-classroom app that surpasses the limitations of existing platforms by fostering enhanced engagement, personalization, inclusivity, and teacher empowerment. While current platforms offer fundamental tools for remote learning, they often struggle to truly engage students. Our app seeks to revolutionize education with interactive features that captivate learners and encourage participation. Additionally, we recognize the diverse needs of students and aim to personalize their learning experiences. Accessibility and inclusivity are paramount, addressing language barriers, disabilities, and varying technological resources. Empowering educators with streamlined administrative tasks and creative content creation tools enhances the quality of education. Finally, the app's emphasis on real-time interaction ensures that students remain engaged and connected, mirroring the dynamics of in-person classes.

Key Features to Be Implemented in Our E-classroom:

Our proposed system provides following features in order to address above discussed alarming issues:

- Random attendance check
- Random question/answer session

- Students will be notified 5 minutes earlier the class.
- Teachers can create groups during class

1.1.4. MODULE:

Module: Classroom

Teachers are provided with complete control over classroom related activities along with online session pre-configuration features.

- Multiple Classrooms will be able to be created by a single teacher
- Classrooms can have multiple students with isolated data.
- Classrooms will have a notification panel for important messages.
- They will have assignment viewing and uploading sections.
- Each classroom will have a separate online session feature, modifiable by the respective teachers.
- All students will be notified about the start of a Live Session when it is created by the teacher.

Module: Teachers

Teachers will be provided with managerial and configurational features for both classrooms and online sessions. These will ensure student regulation and monitoring

- Teachers will be able to create and manage classrooms.
- Teachers will be able to add students and delete students.
- Class Notification Panel will be under Teacher's control to relay important messages.
- Assignment creation and acceptance will be integrated using file upload/download.
- One-click Live Classroom initialization will take the teacher straight to classroom with video chat capabilities.
- Teachers can pre-configure optional features before the starting classroom
- Teachers can create groups for assignments
- . white board along with tool bar will be present in order to make virtual classes more attractive and for the ease of teacher as well.

Module: Students

Students will be provided with features like joining a classroom, viewing and uploading assignments, view notification, join online sessions and poll.

- Students will be provided with features like joining a classroom, viewing and uploading assignments, view notification, join online sessions and poll Students will be given access to the classrooms allotted by the teachers. They can also join classroom with the provided classroom code.
- They can view and upload assignments from their classrooms.
- They will see important notifications like Live Sessions, Assignment Deadline in

- notification panel.
- Students can interact with various dialogue boxes that will appear in the online sessions.
- Students will be able to interact with surprise time limited attendance check to keep them focused and engaged.
- They will be able to anonymously poll after each Live session about the lecture understanding.

1.2. EXISTING EXAMPLES/ SOLUTIONS:

Several solutions have emerged to address challenges in online learning. Tencent Classroom saw widespread use during the COVID-19 pandemic in China, accommodating a significant portion of K-12 students. Additionally, platforms like Lark, initially created for internal use by Byte Dance, have expanded their features to offer unlimited video conferencing, auto-translation, and real-time collaboration for teachers and students. However, online learning still faces challenges, including distractions during virtual sessions, time management difficulties, and the need for students to maintain focus and adapt to a virtual classroom setting. A detailed account of these solutions is given below.

1.2.1 ZOOM

Zoom, initially a general collaboration tool, experienced explosive growth since the outbreak of COVID-19, becoming a staple for businesses and schools. It offers high usability, quick adoption, and a user-friendly interface. Zoom accommodates various meeting types, from one-on-one chats to large webinars and international conferences. Notable features include whiteboard annotation, screen sharing, document upload, and robust security settings. However, zoom lacks mechanisms for ensuring student attention, tracking learning, and regulating student behavior, and it's vulnerable to "zoom bombing." A big problem with the system is that unwanted people crashing a conference call is a new phenomenon on the platform called zoom bombing. Despite its limitations, Zoom's simplicity and capacity for large gatherings make it a popular choice for virtual meetings and classes

Main features of Zoom

- Whiteboard Annotation and Content Highlighting.
- Chat Box.
- Screen Sharing.
- Video Streaming.
- Documents Upload.
- Screen Recording.
- Annotations and Polling.

Limitations of Zoom

- No way to ensure students' attention.
- No attendance system.

- No way of tracking students' learning.
- It does not have a comprehensive logging system.
- It does not have a student regulatory system.
- No Intruder Prevention.
- Too Many Subscriptions and Add-Ons.
- Zoom bombing.

1.2.2. GOOGLE MEET

Google Meet is a video conferencing tool designed for businesses, organizations, and educators, offering features like attendance reports, controlled access, and real-time captions. It's ideal for remote team communication and online education, with integrations into Google's ecosystem, making it convenient for users. Despite its simplicity and widespread use, it lacks certain features such as complete classroom management and student regulation, often requiring complementing services like Google Classroom. Comparatively, it has fewer features than some other video conferencing tools like Zoom. However, Google Meet can be enhanced with advanced features through the Teaching and Learning Upgrade for educators, including breakout rooms and live streaming.

Main features of Google meet.

- Start and join meetings using a dedicated Link.
- Get attendance reports.
- Start a video meeting from Gmail.
- Control access to video meetings.
- Control participation using moderation features.
- Feature Access Control
- Attendance tracking with automated reports
- File Sharing.
- Real Time Captions

Limitation of Google Meet

To get a complete classroom experience one has to use both Google Classroom and Meet.

- Students are able to share the screen whenever they want.
- Teachers cannot completely end the meeting.
- It does not have a student regulatory system.
- No way to ensure students' attention.

1.2.3. MICROSOFT TEAMS:

0.5" Microsoft Teams is a versatile platform designed for efficient work and collaboration, closely integrated with Microsoft Office applications. It incorporates video conferencing and is particularly valuable for its all-in-one approach, combining chat, file sharing, and team

Capital University of Science and Technology, Islamabad Department of Computer Science

features. Its standout due to its simplicity, accommodating up to 300 users, and seamless integration with other Microsoft 365 tools. However, despite its strengths, it faces competition from numerous similar tools, potential storage consumption issues, and lacks features such as student attention tracking, comprehensive logging, and robust notifications. Educating users about its various functionalities remains a challenge for Microsoft.

In summary, Microsoft Teams offers a comprehensive digital workspace with a strong focus on collaboration and communication, making it a compelling choice for businesses and educators, although it may require users to navigate its rich feature set effectively.

Main features of MS Teams:

- MS Teams have web-based versions of Word, Excel and PowerPoint.
- This system is easy to use externally, external users need not to register an account.
- File storage and sharing
- It can accommodate up to 300 users at a time.
- Collaboration with other Microsoft 365 tools and apps.
- User Friendly.
- Chat Feature is available.

Limitations of MS Teams:

The limitations of the application are listed below.

- There are far too many similar tools.
- Consumption of storage space that isn't required
- No way to ensure students' attention.
- It does not have a comprehensive logging system.
- High security risk
- Notifications are missing.

1.2.4. Cisco Webex

Cisco Webex is a secure and feature-rich conference call service offered by a leading technology and networking company. It provides video conferencing, file sharing, and chat features, catering to various use cases such as webinars, classes, and conferences. With a user-friendly interface and robust security measures, it competes effectively with other platforms. It encrypts user data and offers HD video display for up to 25 participants, along with collaboration tools like whiteboarding and breakout rooms.

Webex excels in scenarios like webinars and online classes, supporting up to 3,000 participants and 40,000 broadcast users. It integrates seamlessly with external Learning Management Systems (LMS) and offers customization options. However, to achieve a complete classroom experience, users may need to combine it with an external LMS. The extensive integration capability can sometimes lead to challenges in platform switching, centralization, and security.

While Webex provides engagement features like whiteboarding and interactive polling, they may not guarantee student engagement but merely track it. Additionally, it lacks student regulation and a mechanism to ensure students' attention during online sessions.

In summary, Cisco Webex offers a comprehensive and secure conferencing solution with extensive integration options but may require users to carefully manage the complexity of multiple integrated platforms for a seamless experience.

Main features of Cisco Webex:

- Online meetings through video conferencing
- Screen Sharing and File Sharing
- Integration with external LMS
- Customizable video layout and meeting access controls
- Active Engagement features such as Digital Whiteboarding, interactive polling
- Multiple use cases
- Robust Security

Limitation of Cisco Webex:

To get a complete classroom experience one has to use both Webex and some external

- Limited Personalization Options
- Multiple Ways to Connect
- It does not have a student regulatory system.
- No way to ensure students' attention.

Table 1: existing solutions features comparison

FEATURES	APPLICATIONS				
	Zoom	GoogleMeet	Cisco Webex	MS Teams	PROPOSED SYSTEM
Comprehensive activities log	No	Yes	Yes	No	Yes
Ensuring Student attention	No	No	No	No	Yes
Knowledge delivery tracking	No	No	Yes	Yes	Yes
Student engagement measurement	No	No	Yes	Yes	Yes
Ease of use	Yes	Yes	Yes	Yes	Yes
Live Session	Yes	Yes	Yes	Yes	Yes
Classroom Management	No	No	Yes	Yes	Yes
Intruder prevention	No	No	Yes	Yes	Yes
Live Notes Section	No	No	Yes	Yes	Yes
File Sharing	No	No	Yes	Yes	Yes
Google Classroom collaboration	No	No	No	No	Yes

1.3. BUSINESS SCOPE:

The business scope of the discussed E-Class system is to provide a comprehensive online educational platform with a focus on enhancing student engagement, attention, and security during virtual classes. Its target market includes educational institutions, teachers, and students looking for an effective online learning solution. E-Class aims to meet the growing demand for improved online teaching experiences, filling the gap left by traditional video conferencing tools by offering features such as automated random attendance, identity verification, real-time student monitoring etc. This system aspires to carve out a unique niche in the market, delivering a high-quality, interactive, and secure online learning experience that sets it apart from traditional online teaching tools.

The proposed system possesses several market-competitive features which helps the students and teachers to get the same experience they get while attending and delivering physical lectures. Following are the modules and features of E-Class.

The business scope of an E-classroom project can be quite broad, as it involves online education and remote learning, which have experienced significant growth and demand. Here are various aspects of the business scope for such a project:

- **Online Education Platform:** The primary focus of the project is to create an online education platform that facilitates remote learning. This platform can serve students of all ages, from K- 12 to higher education and even professional development.
- **K-12 Education:** Targeting K-12 students and offering a range of subjects and courses. This can include curriculum materials, assessments, and interactive lessons.
- **Higher Education:** Expanding the platform to cater to higher education institutions and universities. Offering courses, lectures, and resources for undergraduate and graduate-level students.
- **Corporate Training:** Providing corporate training modules and courses for organizations looking to upskill their employees. This can include certifications and professional development opportunities.
- **Tutoring Services:** Providing one-on-one or small group tutoring services for students who need additional help or personalized instruction.
- **Content Creation:** Allowing educators and content creators to upload and monetize their courses and educational materials.
- **Live Classes and Webinars:** Supporting live classes, webinars, and interactive sessions with teachers and instructors.
- **Assessment and Grading:** Implementing features for conducting assessments, quizzes, exams, and automated grading.
- **Student Management:** Developing tools for tracking student progress, attendance, and performance.
- **Integration:** Integrating with learning management systems (LMS), video conferencing tools, and other educational technologies.
- **Mobile Accessibility:** Ensuring that the platform is accessible on mobile devices, considering the growing use of smartphones and tablets for learning

Remote Work Trends

In 2023, remote work has indisputably become a transformative trend, reshaping the landscape of traditional work environments. Pertinent statistics from this year reveal significant insights:

Table 2: remote work trend nowadays

Year	Remote Work Statistics
2023	- 12.7% of full-time employees work remotely
	- 28.2% adopt a hybrid work model
	- 59.1% work in traditional office settings
2025	- Projected 32.6 million Americans (22% of workforce) working remotely
	- 98% of workers desire remote work
	- 93% of employers continue remote job interviews
	- 16% of companies fully embrace remote work
	- Transition in industries (computer/IT, marketing, accounting, finance, health) towards remote work

1.4. Useful Tools and Technologies:

This section should mention possible technologies that could be used during the designing, development and testing of your project.

You should mention the technologies that you intend to use with a brief but technical justification for your decision. Your discussion should include

What programming language is you using and why?

Ans:

Frontend Tools:

Html, CSS, JavaScript, React.js

the choice of these programming languages and tools is driven by the need to create a dynamic, interactive, and responsive web application. HTML, CSS, and JavaScript are foundational for frontend development, while React.js enhances the development process.

Backend Tools:

Firestore

Firestore simplifies backend development by offering a range of services that align with modern web application requirements.

Which development environment do you intend to use and why?

Ans:

Visual Studio Code (VS Code) is a popular, lightweight, and cross-platform code editor developed by Microsoft. It is highly customizable through extensions and offers features like integrated terminals, Git support, intelligent code assistance, and debugging. VS Code has a vast extension marketplace and is actively updated, making it a preferred choice for many developers. It's free, open-source, and suitable for various programming languages and projects. The choice of VS Code or any other development environment depends on individual preferences and project requirements.

What database (if any) is you using and why?

Ans:

Firestore is a Google-backed platform for building and managing mobile and web applications. It provides a suite of services including real-time databases, authentication, cloud hosting, cloud functions, machine learning, analytics, and more. Firestore is known for its ease of use, scalability, and real-time capabilities, making it popular for startups and developers. It offers a free tier and can be used to quickly develop and deploy applications with various features and functionalities.

Which operating systems will support your software?

Ans:

Windows operating system.

Mac operating system

What network protocol (if any) is implemented and why?

Ans: Https

1.5. Project Work Break Down

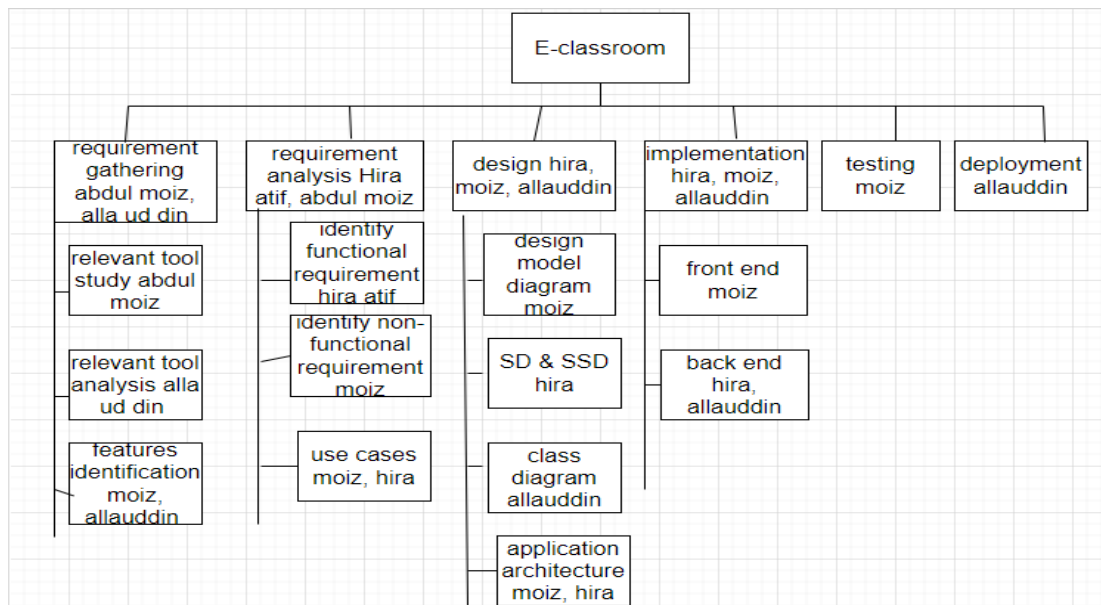


Figure 1: project work breaks down

1.6 Project Time Line

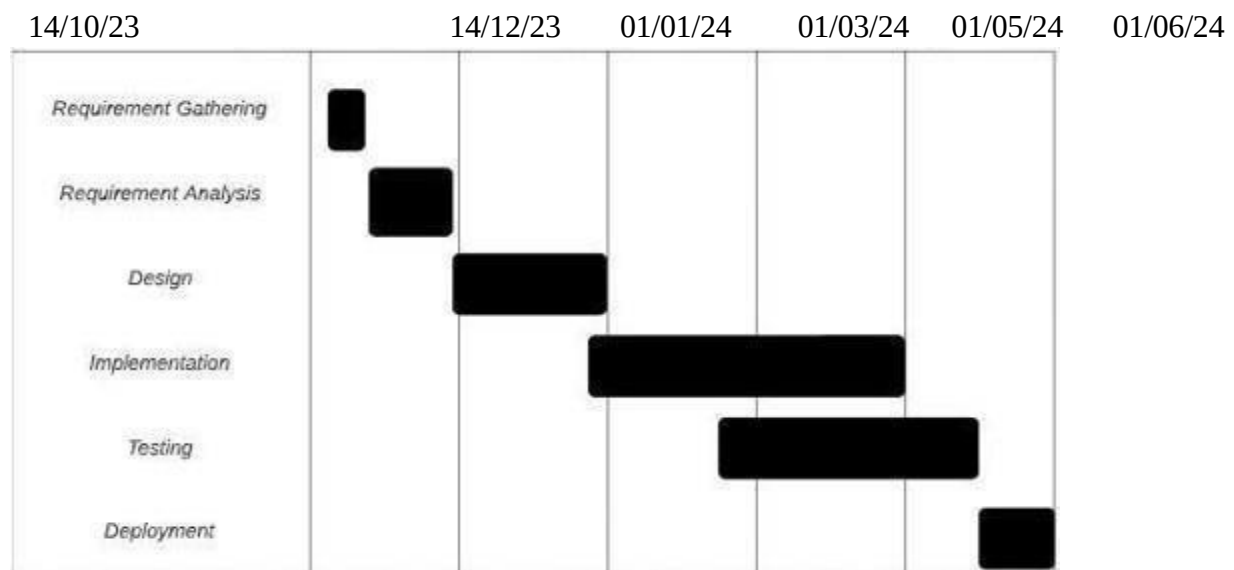


Figure 2: project time line

Chapter 2

Requirement Specification and Analysis

2.1. Functional Requirements

Table 3: functional requirements of the system

S. No.	Functional Requirement	Type	Status
FR1	Teacher can register	Core	completed
FR2	Teacher can login	Core	completed
FR3	Teachers can create classroom	Core	completed
FR4	Teachers can add students	Core	completed
FR5	Teachers can create assignment	Core	completed
FR6	Teachers can view attendance report	Core	completed
FR7	user can mute microphone	Intermediate	completed
FR8	Teacher can download students submitted assignment	Core	Completed
FR9	Teachers can start random attendance	Core	completed
FR10	Teachers can start random Questions	Core	completed
FR11	Teachers can create groups	Core	completed
FR12	Students can join classroom	Core	Completed
FR13	Students can create notes	Core	completed
FR14	Students can upload assignment	Core	Completed
FR15	Students can view class notifications	Intermediate	Completed
FR16	Student can login	Core	Completed
FR17	Student can register	Core	Completed
FR18	Students can view notes	Core	completed

2.2. Non-Functional Requirements

Table 4: non-functional requirements of the system

S. No.	Non-Functional Requirements	Category
NFR1	Data is updated on every state change	Reliability
NFR2	The system should keep and retrieve record correctly	Reliability
NFR3	UI will be as per web guidelines	Usability
NFR4	The system will be available 99.9%of the time	Availability
NFR5	Student will login from their email.	Security

2.3. System Use Case Modeling

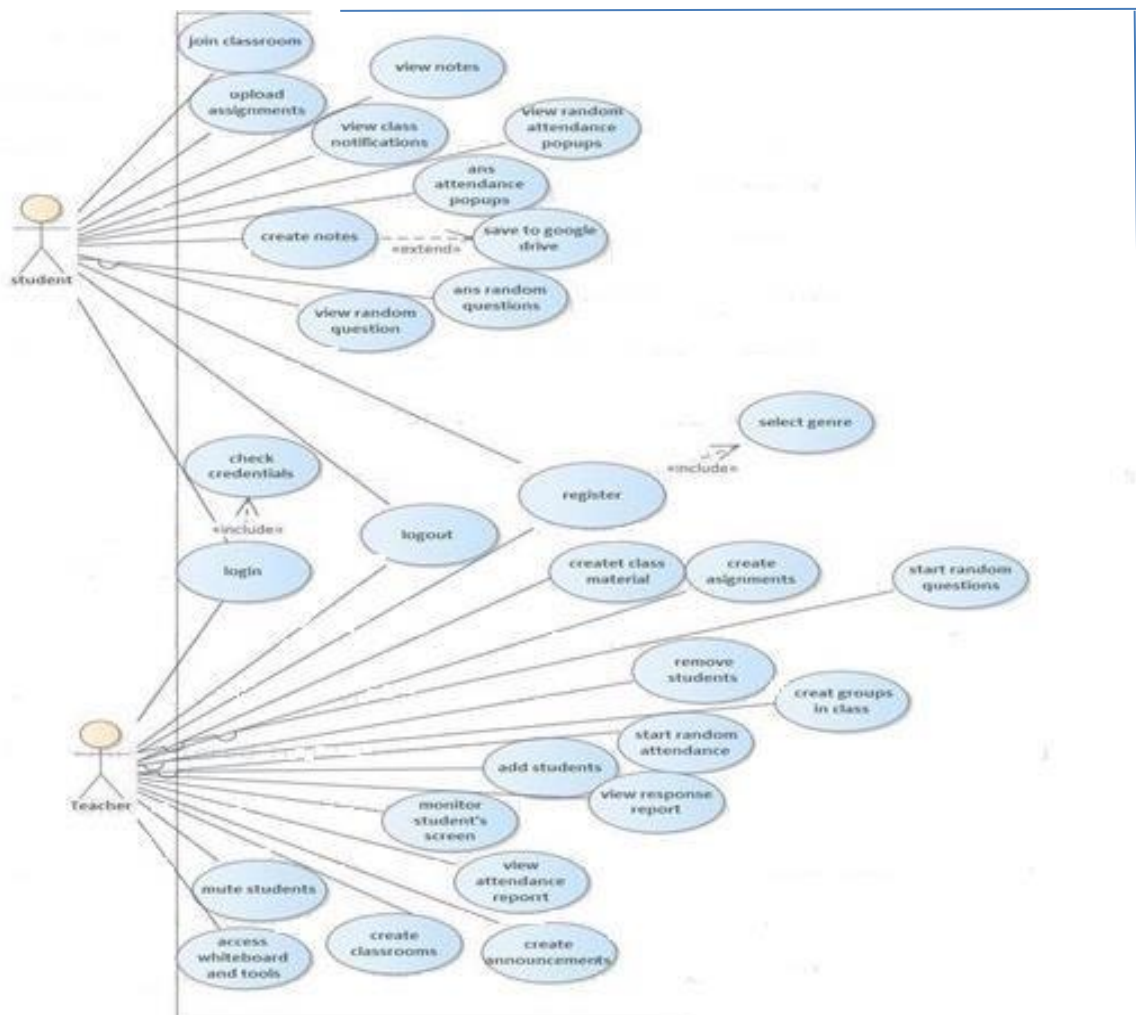


Figure 3: system use case modeling

A use case diagram for an e-classroom illustrates the interactions between users (such as students, teachers, and administrators) and the system's key functionalities. It typically includes use cases like accessing course materials, participating in online discussions, submitting assignments, and managing class schedules. This diagram helps in visualizing the system's workflow and identifying user roles and their specific interactions with the e-classroom platform.

2.4. USE CASE DESCRIPTION

Table 5: UC-01 register

Use Case ID:	UC-01		
Use Case Name:	Register		
Created By:	Hira Atif	Created By:	Hira Atif
Date Created:	10/25/2023	Date Created:	10/25/2023
Actors:	Students, Teacher		
Description:	The purpose of this use case is to describe how a user registers into the system.		
Trigger:	Click on register button.		
Preconditions:	In case of Google signup, user must use a valid google account.		
Post conditions:	• User’s authentication token is generated and marked as signed in. • User is registered in the database in ‘users’ collection with ‘Account type’ set to” teacher” or” student”		
Normal Flow:	Actor	System	
	1. User visits the home page. 3. User clicks the” Signup” button. 5. User is routed to signup page. User enters email, password and account type and clicks on “Register” button. 6. Upon successful authentication user is routed to Dashboard.	2. Website opens new user registration form in which user’s details are required. 4. Web app checks the form and stores the data of new customer in database.	
Alternative Flows:	N/A		

Table 6: UC-02 user login

Use Case ID	UC-3		
Use Case Name	user Login		
Created By	Hira Atif	Created By:	Hira Atif
Date Created	10/25/2023	Date Created:	10/25/2023
Actors	Student, teachers		
Description:	The purpose of this use case is to describe how user can login from their account.		
Trigger:	Click on login button.		
Preconditions:	In case of signing in user must have a valid google account. User data must be present in database.		
Post conditions:	Customer will be logged into his/her account and dashboard will be displayed.		
Normal Flow:	Actor	System	
	1. User enter his/her credentials on login screen and click on login button. 3. Upon successful authentication user is routed to the dashboard.	2. Database validates the entered credentials and then home screen is displayed.	

Table 7: UC-03 user logout

Use Case ID	UC-3		
Use Case Name	user Logout		
Created By	Hira Atif	Created By:	Hira Atif
Date Created	10/25/2023	Date Created:	10/25/2023
Actors	Student, teachers		
Description:	The purpose of this use case is to describe how user can logout from their account.		
Trigger:	Click on logout button.		
Preconditions:	user should be logged in.		
Post conditions:	Customer will be logged out from his/her account and login page will be displayed.		
Normal Flow:	Actor	System	
	1. User clicks on profile button. 3. User clicks on logout button.	2. A new page will be displayed and contain profile information and logout button. 4. Application logs out user and shows login page.	

Table 8: UC-04 create classroom

Use Case ID	UC-04		
Use Case Name	Create classroom		
Created By	Hira At	Created By:	Hira Atif
Date Created	25-10-2023	Date Created:	25-10-2023
Actors	Teachers		
Description:	The purpose of this use case is to describe that how a teacher creates a classroom.		
Trigger:	Click on create classroom button		
Preconditions:	Teacher must be signed in.		
Post conditions:	Classroom must be added to the 'classrooms' collection in database.		
Normal Flow:	Actor	System	
	1. Teacher clicks the "Create Classroom" button. 3. Teacher is routed to classroom creation page. 4. Teachers enters Subject, Title, Section, password and clicks on "Create" button.	2. Application will open a new page that will display the student names to be entered in the classroom	

Table 9: UC-05 add student

Use Case ID	UC-04		
Use Case Name	Add student		
Created By	Hira Atif	Created By:	Hira Atif
Date Created	25-10-2023	Date Created:	25-10-2023
Actors	Teachers		
Description:	The purpose of this use case is to describe how a teacher add or delete student from classroom.		
Trigger:	Click on add student button		
Preconditions:	Student must be signed in and the classroom must already be created.		
Post conditions:	Classroom must be added to the 'Join classrooms' collection in database.		
Normal Flow:	Actor	System	
	1. Teacher click the "add student" button. 3. Teacher is display with a drop-down having text field. 5. Teacher is routed to classroom creation page. 6. Teachers enters Subject, Title, Section, password and clicks on "Create" button.	2. A new page will be displayed and all students to be added or removed from the classroom. 4. Application show editable user action (add or delete).	

Table 10: UC-06 create assignment

Use Case ID	UC-6	
Use Case Name	Create Assignment.	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Teacher	
Description:	The purpose of this use case is that teacher can create assignments in the classroom.	
Trigger:	C lick on create assignment button	
Preconditions:	Classroom must already be present for the assignments to be posted.	
Post conditions:	Assignments must be added to the users' 'class Assignments' field of 'classrooms collection in database.	
Normal Flow:	Actor	System
	1. Teacher switches to "Create Assignment" tab. 3. Teacher is sent to the "Create Assignment" page for creating assignments. 5. Teacher is asked to attach the file to post. 6. Teacher enter the title and description of the assignment, and clicks on "Post" button. 7. Assignment will be posted and the user will be directed to the updated classroom.	2. Application allows the teacher to upload class material in the class material section e.g. book, ppts, notes etc. 4. After upload class material, these files will be accessible to all the students and will also be saved in the database.

Table 11: UC-07 create class material

Use Case ID	UC-7	
Use Case Name	Create class material	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Teacher	
Description:	The purpose of this use case is to describe how teacher create class material in the class-room.	
Trigger:	Click on create class material icon.	
Preconditions:	Classroom must already be present for the class material to be posted.	
Post conditions:	Class Material must be added to the users' 'class Material' field of classrooms' collection in database.	
Normal Flow:	Actor	System
	1. Teacher switch to the "Create Class Material" tab. 3. Teacher is routed to the "Create Class Material" page for creating class material. 5. To post, teacher must attach the file. 6. teacher enters the title and description of the material, and clicks on "Riff" button. 7. Class material will be posted and the user will be directed to the updated classroom.	2. Application allows the teacher to upload class material in the class material section e.g book, ppts, notes etc. 4. After upload class material, these files will be accessible to all the students and will also be saved in the database.

Table 12: UC-08 create announcement

Use Case ID	UC-8	
Use Case Name	Create Announcement	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Teacher	
Description:	The purpose of this use case is to describe how teacher create announcement in class.	
Trigger:	Click on create announcement button	
Preconditions:	Classroom must already be present for announcement to be posted.	
Post conditions:	Announcement always be added to user 'Announcement' field.	
Normal Flow:	Actor	System
	1. Teacher switches to create announcement tab. 3. Teacher is asked to write the text to update in all student boards. 5. Teacher entered the text and click on post button. 6. Announcement will be posted and student will be redirected to update class	2. Application will popup text bar to create announcement that have to share with students. 4. Once the announcement is made it is visible to the student screen

Table 13: UC 09 view attendance report

Use Case ID	UC-9	
Use Case Name	View attendance report	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Teacher	
Description:	The purpose of this use case is to describe how teacher can view assignment report	
Trigger:	Click on view report button	
Preconditions:	Attendance would be taken before viewing the attendance report	
Post conditions:	Download the attendance report	
Normal Flow:	Actor	System
	1. Teacher take the attendance and click on attendance report button 3. Download the report.	2. Attendance report would be displayed 4. Downloaded successfully

Table 14: UC-10 mute microphone

Use Case ID	UC-10	
Use Case Name	Mute student microphone	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Teacher	
Description:	The purpose of this use case is to describe how teacher can mute student microphone	
Trigger:	Click on mute button	
Preconditions:	Student should be login	
Post conditions:	Student cannot unmute his/her mike by himself/herself	
Normal Flow:	Actor	System
	1. Teacher can go to attendees. 3. Teacher can mute specific student.	2. List of attendees should be shown. 4 Mike of specific student should be muted.

Table 15: UC-11 create groups during class

Use Case ID	UC 11		
Use case name:	Create groups during class		
Created By:	Hira Atif	Last Updated By:	Hira Atif
Date Created:	252/11/2024	Last Revision Date:	252/11/2024
Trigger	Teacher will select students for group		
Descriptive Use Case:	Teacher will be able to create groups during class.		
Actor	Teacher		
Precondition	Teacher and student both join the class at the time when teacher is creating groups		
Post condition	Groups are created with specific number of students to perform certain task or group activity		
Main Flow	Teacher	System	
	1. Teacher joins the class. 2. Teacher selects students to make their group.	3. Those students are added in the group.	
Alternative Flow	N/A		
Exception	- System is not responding. - Database is not responding.		

Table 16: UC-12 join classroom

Use Case ID	UC-12	
Use Case Name	Join classroom	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Student	
Description:	The purpose of this use case is to describe how the student can join classroom	
Trigger:	Click on join button	
Preconditions:	Student should be login	
Post conditions:	Student should be the part of all activities in the classroom	
Normal Flow:	Actor	System
	1. Student should login with their id 3. Student should click the "join classroom" button.	2. System checks the credential and allow the student to enter the classroom

Table 17: UC-13 student can upload assignment

Use Case ID	UC-13	
Use Case Name	Student can upload assignment	
Created By	Abdul Moiz	
Date Created	11/14/2022	
Actors	Student	
Description:	The purpose of this use case is to describe how the student can upload assignment	
Trigger:	Click on upload button	
Preconditions:	Student should be login	
Post conditions:	Assignment should be upload	
Normal Flow:	Actor	System
	1. Student select the document and clicks the upload button.	2. Assignment should be uploaded successfully

Table 18: UC-14 student view notes

Use Case ID	UC-14		
Use Case Name	View Notes		
Created By	Allaaddin	Created By:	Allaaddin
Date Created	25/10/2023		
Actors:	Students		
Description:	The use case describes how a user views created lecture notes in the classroom.		
Trigger:	Click on View Notes button.		
Preconditions:	User must be logged in as a student. The notes must be already present in the database.		
Post conditions:	Notes will be uploaded to the users' drive.		
Normal Flow:	Actor	System	
	1. Student clicks the "View Notes" tab. 3. The student can click on a specific note to view it leading to a separate page.	2 The user is directed to notes page for viewing class notes.	

Table 19: UC-15 view class notification

Use Case ID	UC-15		
Use Case Name	View Class Notification		
Created By	Allauddin	Created By:	Allauddin
Date Created	25/10/2023		
Actors	Students		
Description:	The use case describes how a user views class notification in the classroom.		
Trigger:	Click on View Notifications button.		
Preconditions:	User must be logged in as a student. The notification must be already present in the database.		
Post conditions:	Student perform actions according to notifications received.		
Normal Flow:	Actor	System	
	1. Student clicks the "Notifications" tab.	2 The Student is directed to notifications page for viewing class notifications	

Table 20: UC-16 start random attendance

Use Case ID	UC-16		
Use Case Name	Start Random Attendance		
Created By	Allauddin	Created By:	Allauddin
Date Created	25/10/2023		
Actors	Teacher		
Description:	The use case describes how the Random Attendance module will work in E-Classroom		
Trigger:	Click on Start attendance button.		
Preconditions :	There must be a live session in progress		
Post conditions:	Automatic Attendance report will be generated and sent to the teacher after live session.		
Normal Flow:	Actor	System	
	1. Teacher and Students join Live Session 2. Teacher will click on start attendance 4. The students will interact with the popups by clicking and confirming their presence	3. Random attendances through pop ups will appear for a minimum three times on student UI at random intervals for each student 5. The teacher will be notified about two consecutive missed popups during the live session.	

Table 21: UC-17 create notes

Use Case ID	UC-17		
Use Case Name	Create Notes		
Date Created:	25/10/2023		
Actors	Students		
Description:	The use case describes how a student creates lecture notes in the class-room.		
Trigger:	Click on Create Notes button.		
Preconditions:	User must be logged in as a student. The live session must be in progress		
Post conditions:	Notes will be uploaded		
Normal Flow:	Actor	System	
	1 Student clicks the" Create Notes" tab 3 when notes created student clicks on send to my drive	2 The student is directed to the notepad for creating class notes. 4 File will be uploaded to student account 5 Student will be directed to the updated classroom	

3.8. System Sequence diagrams

The sequence diagram for the instructor registration module. The user must select the “Teacher” account type and provide valid credentials, such as an email address and a password to create an account in the system database.

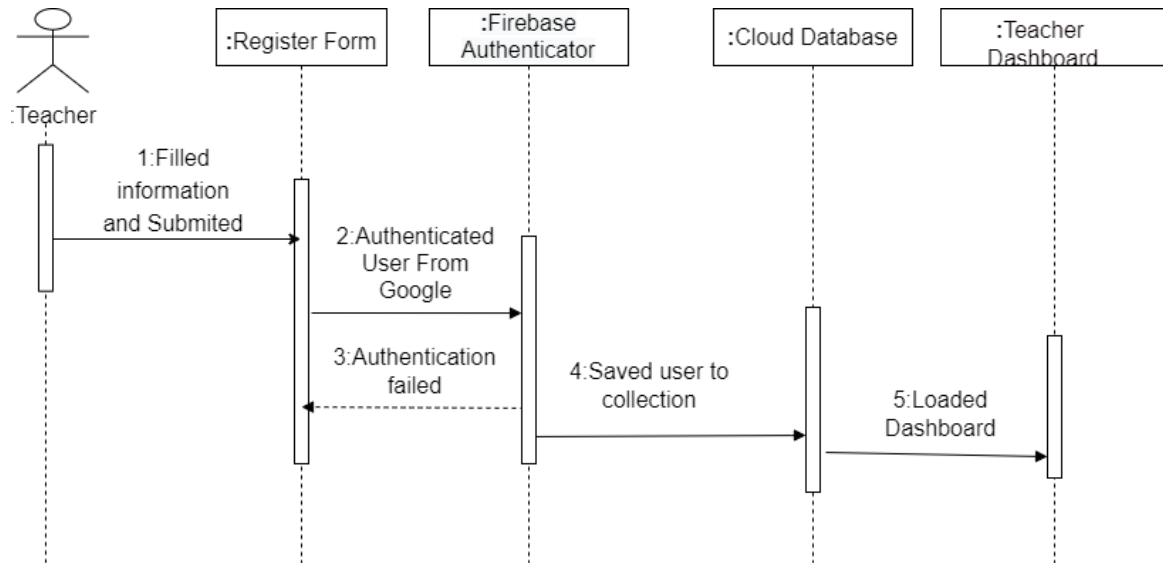


Figure 4: SSD of teacher registration

Sequence diagram for the student registration module to create an account in the system database, the user must first select the “Student” account type and provide valid information, such as an email address and a password.

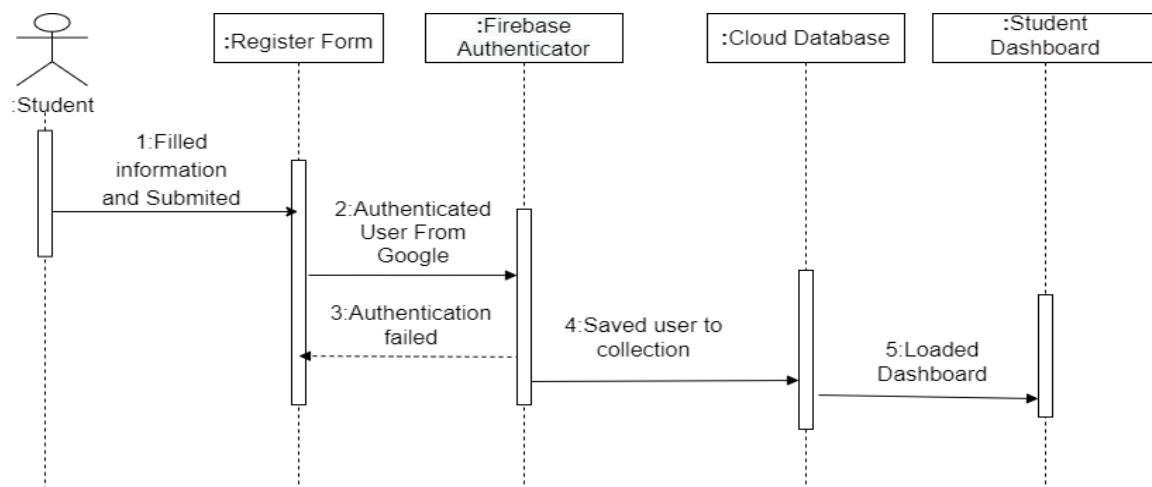


Figure 5: SSD of teacher registration

The sequence diagram for the student login module. In the database of the system, the user must already be registered. As this is considered to be the primary source of managing students and teachers this is considered a very important module. The user must provide their login information in order to log in and use the system's functionalities. After successfully login the user is directed to student dashboard.

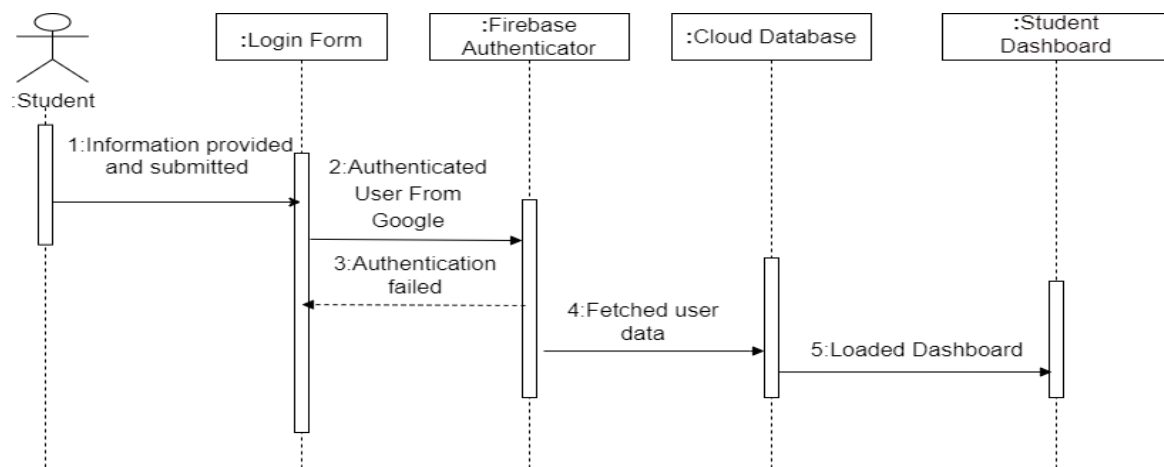


Figure 6: SSD of student login

The teacher login module's sequence diagram. In the system database, the user must already be registered. This module enables the teachers to create an account in system's database. The user must provide their login information in order to log in and use the system's features. After successfully login the user is directed to the teacher dashboard.

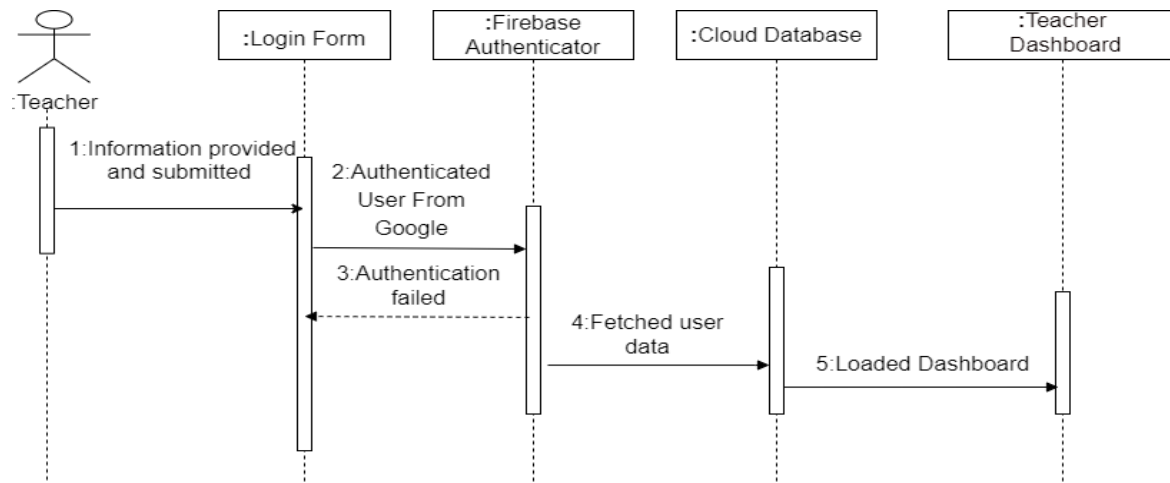


Figure 7: SSD of teacher login

The sequence diagram for the module on creating a classroom. The teacher must provide the name, section, and other information after clicking the “Create Classroom” button. As this is considered to be the primary source of managing students and teachers in a single classroom space this is considered a very important module. A classroom is established in the system database after pressing “create.”

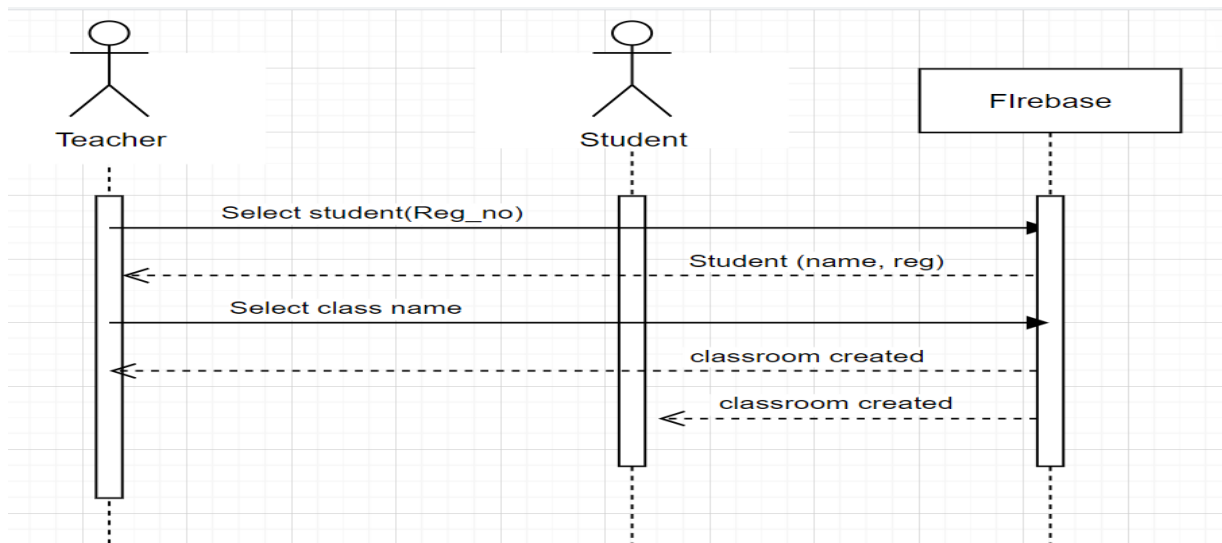


Figure 8: SSD of create classroom

The sequence diagram for the student addition module. After entering the students’ email addresses into the system database in the “students” collection, teachers can add students to their classrooms.

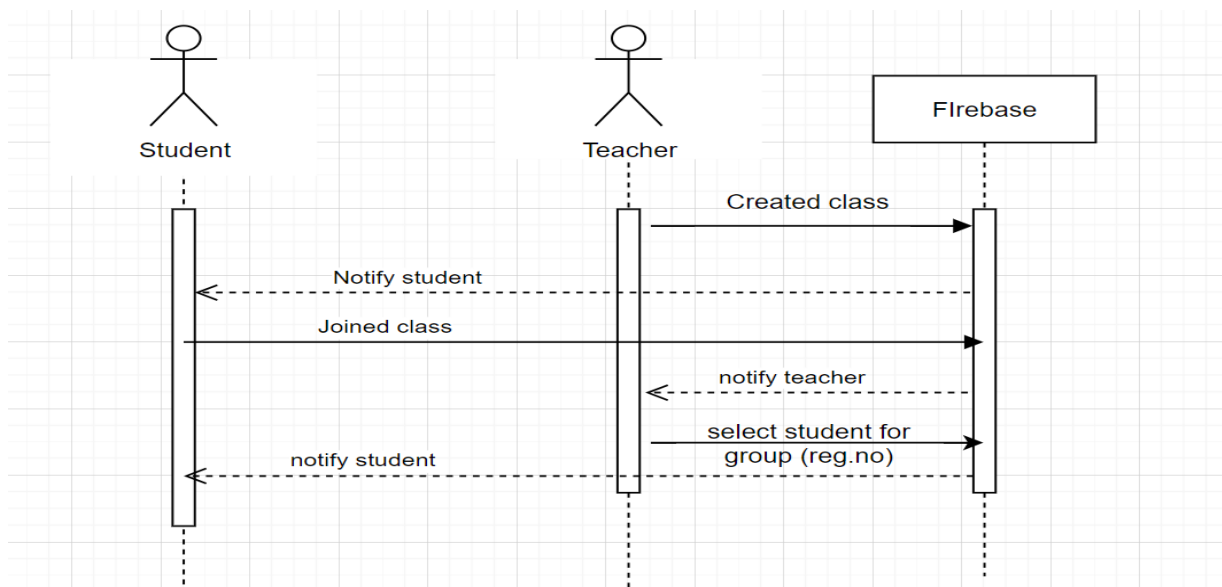


Figure 9: SSD of add students in classroom

Teacher making groups of students while ongoing class:

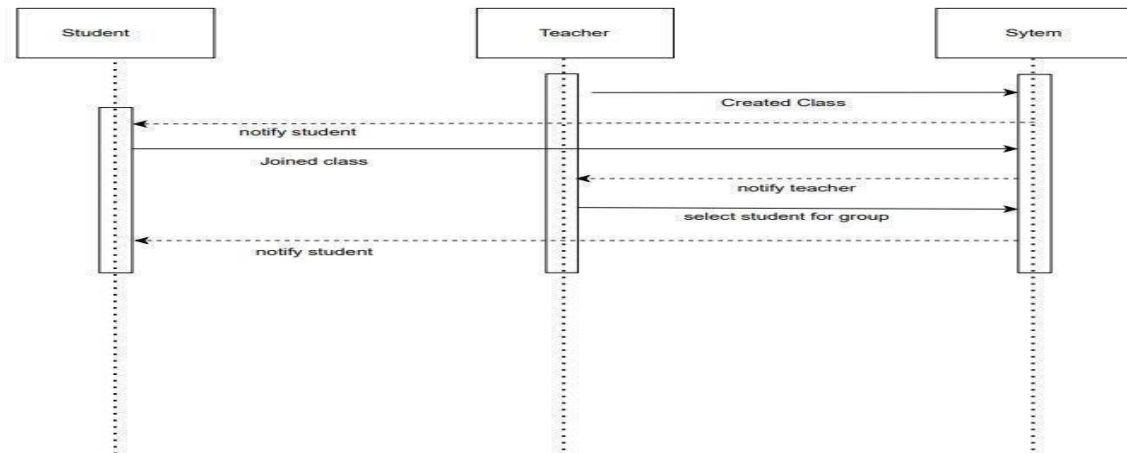


Figure 10: SSD of teacher making groups of students during class

the sequence diagram for the join classroom module. Students can join a particular class by selecting the join button

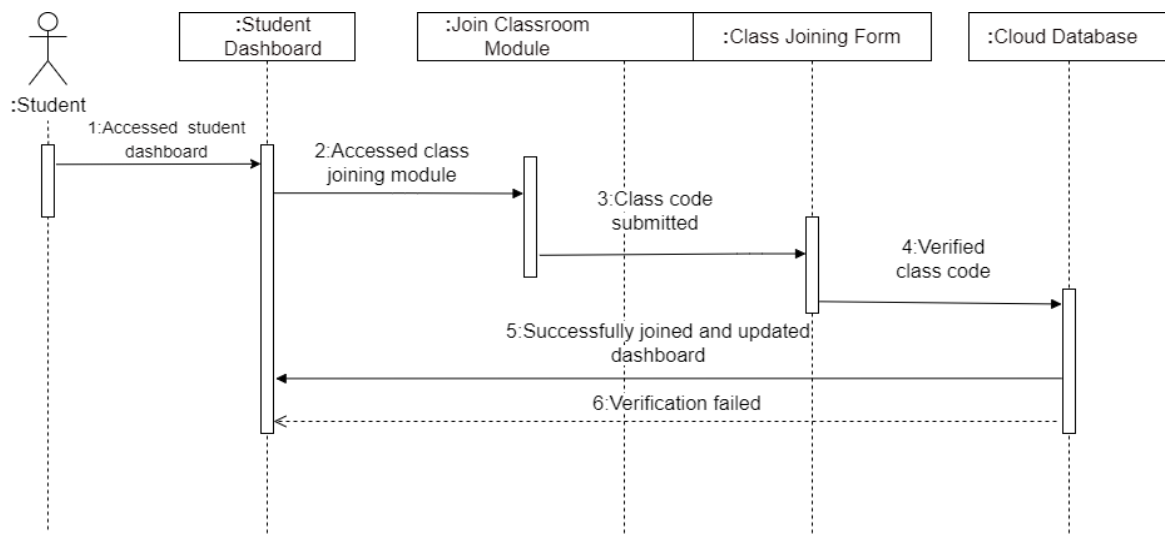


Figure 11: SSD of student join class

The assignment creation module's sequence diagram. The teacher can click on “Create Assignment,” enter a name, a description, and select a file before giving the class a task.

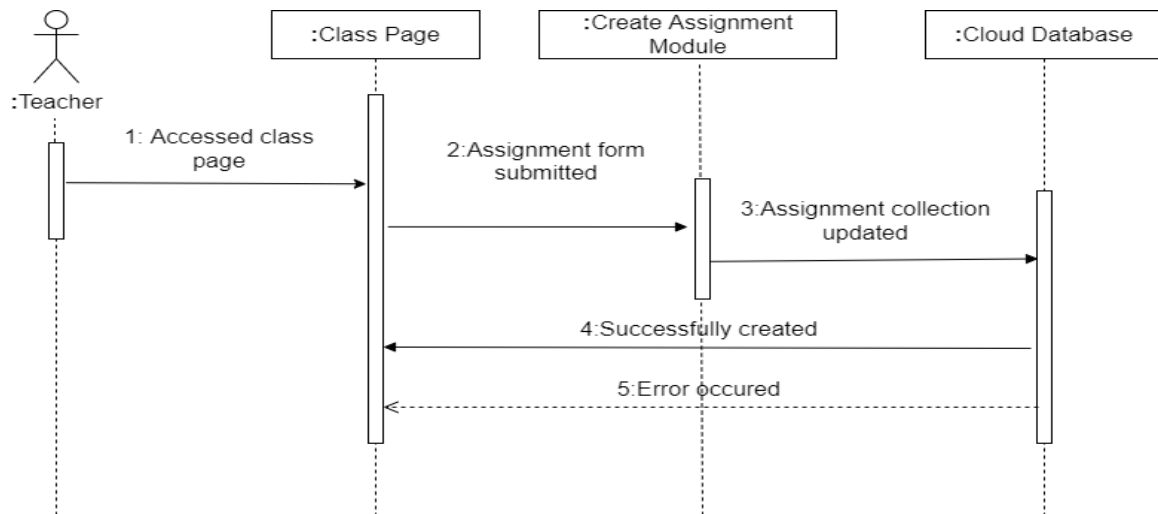


Figure 12: SSD of teacher create assignment

the sequence diagram for the assignment upload module. Selecting a file from their devices, students can upload and submit the assignment that has been given to them. The file is saved in the system database's “Assignments” collection.

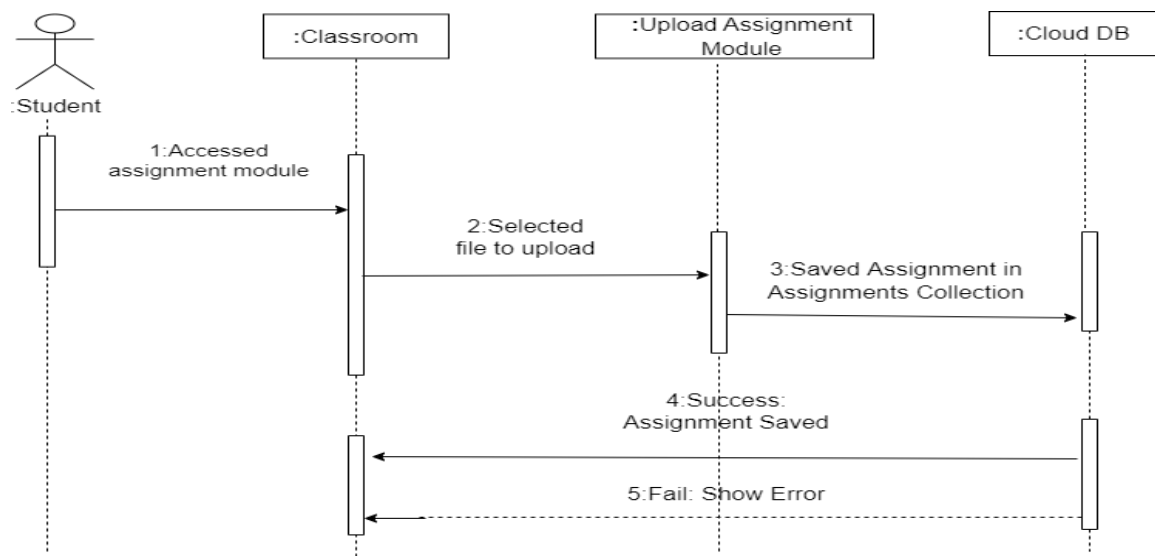


Figure 13: SSD of student submit assignment solution

the sequence of viewing notifications shows that the student can select tab of “Notifications” click on View notifications and can access and view the notifications by the teacher.

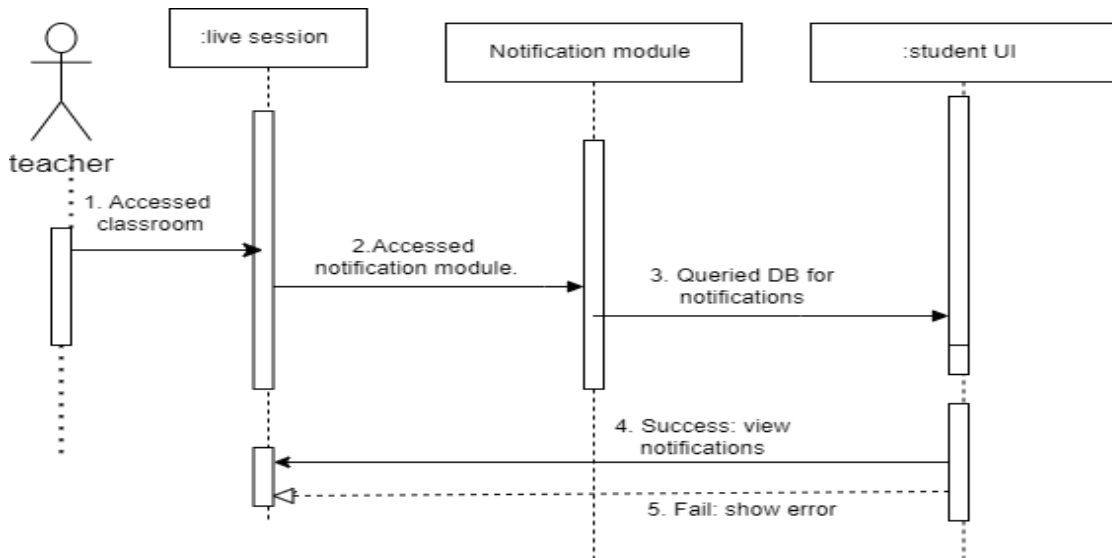


Figure 14: SSD of student receiving notification before class

The sequence diagram for the random attendance module Before beginning the live session, the teacher can begin the attendance module. At random intervals throughout the Live session, attendance pops are emitted.

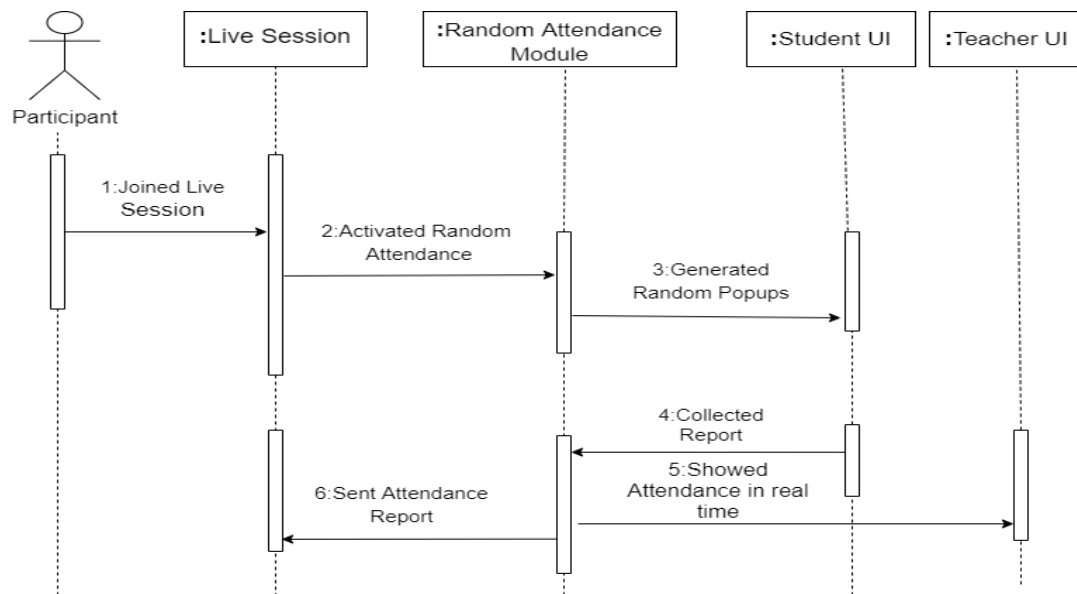


Figure 15: SSD of random attendance

the sequence diagram for the note's creation module. The students can take their own notes for any lesson during the live session, which they can store

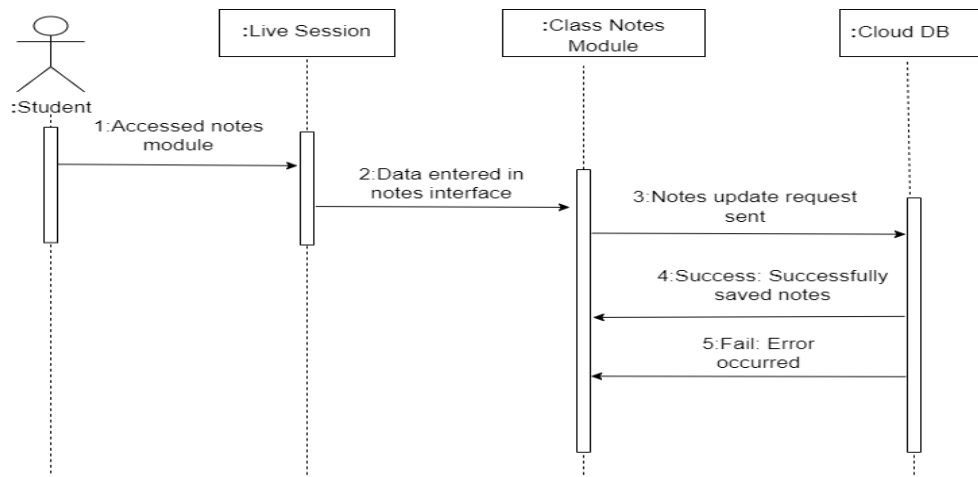


Figure 16: SSD of student create notes

sequence diagram for the view notes module. Students can easily access and view their notes at any time after selecting the “View Notes” button

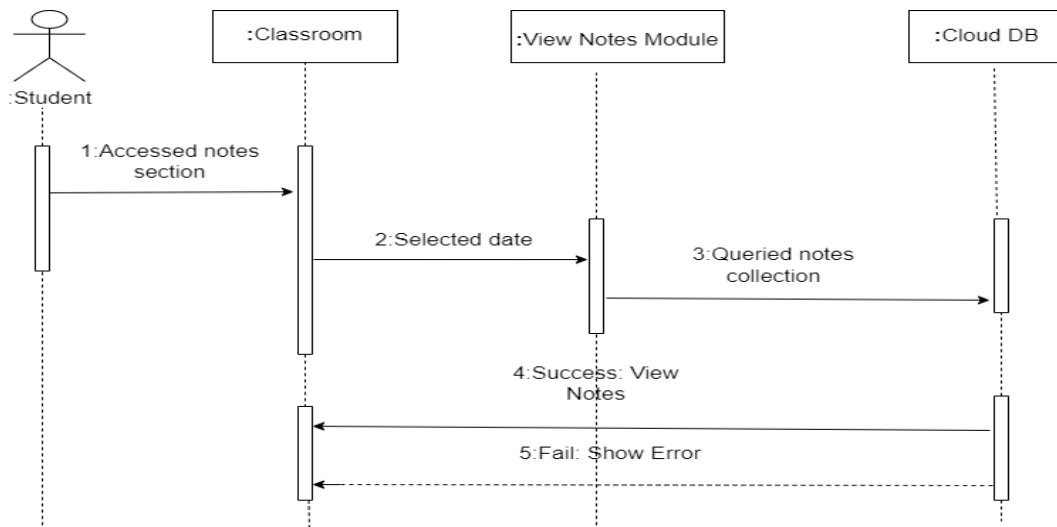


Figure 17: SSD of student view notes

the module to remove students. After choosing the remove student module and clicking on the student's name, the teacher can remove the student from the live session.

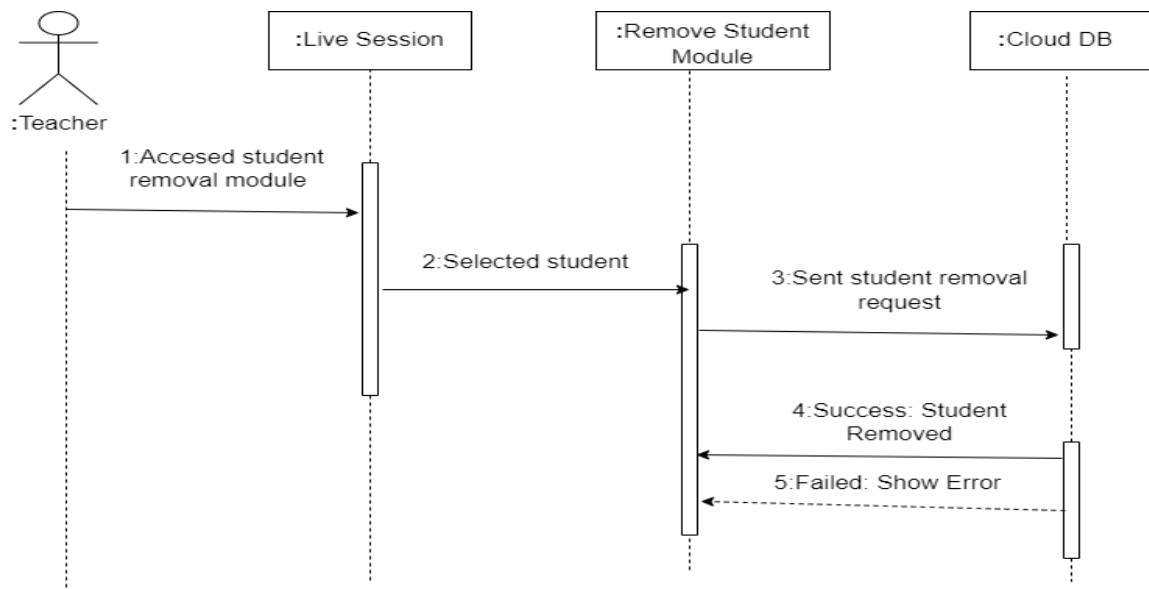


Figure 18: SSD of remove student from class

The sequence diagram for the module asking random questions from student:

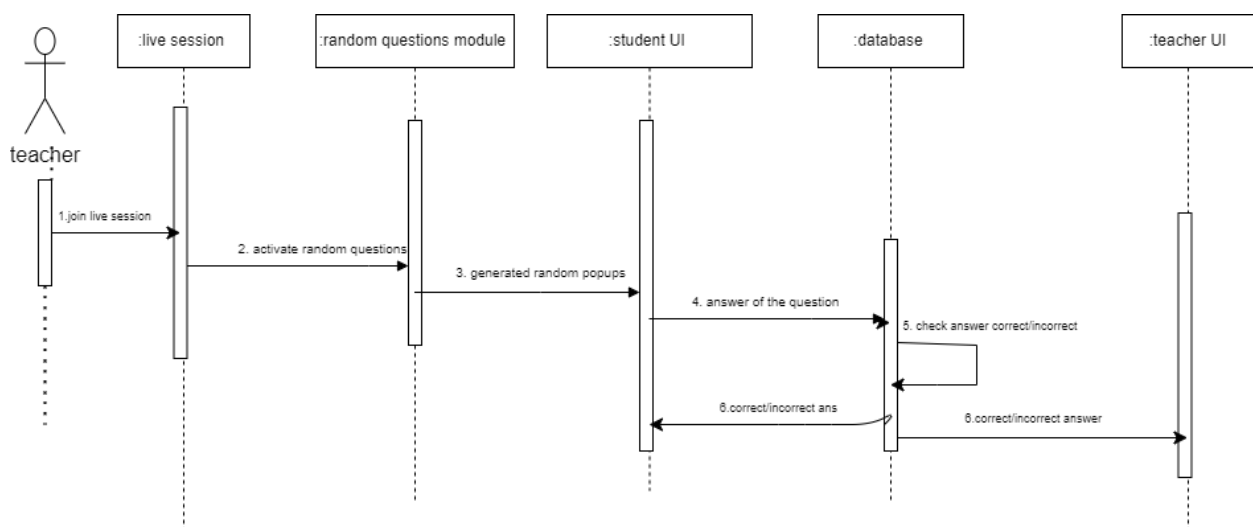


Figure 19: SSD of random questions

3.9. Domain Model

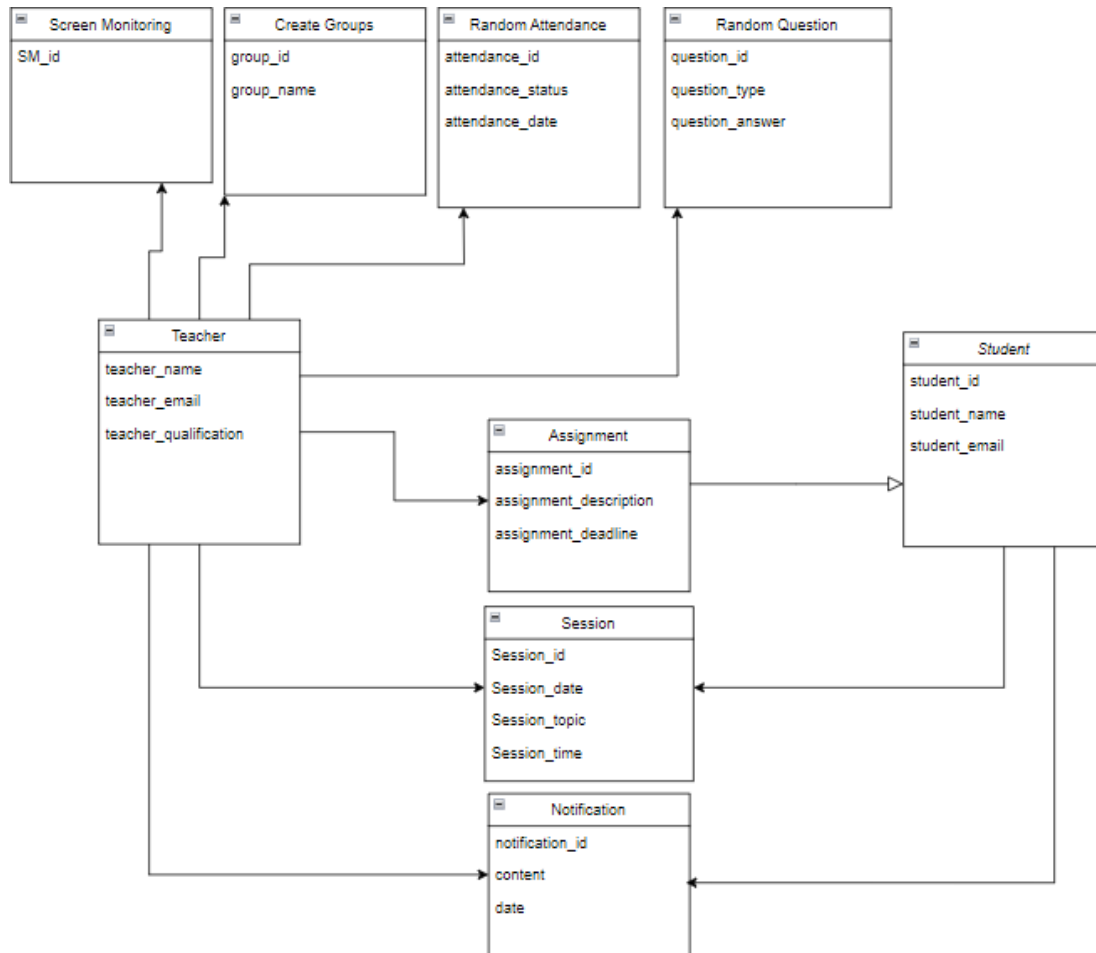


Figure 20: Domain Model of the system

A domain model in an e-classroom system represents the core concepts, entities, and relationships specific to the domain of online education. It captures the essential structure of the system, focusing on the key elements that define the functionality

Chapter 3

System Design

3.1. Software Architecture

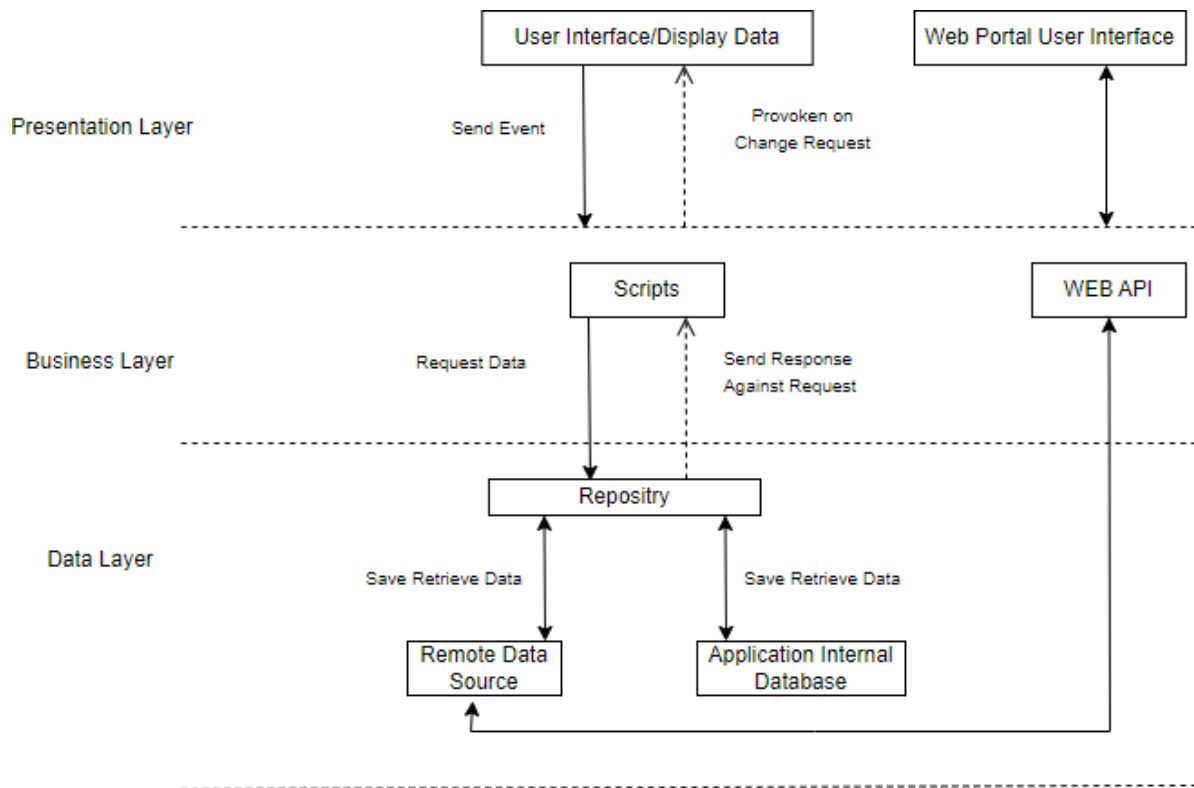


Figure 21: Software Architecture

An e-classroom software architecture typically consists of a client-side frontend and a backend system. The frontend includes a web application for desktop users and a mobile application for smartphones and tablets, often developed using frameworks like React, Angular, or Flutter. The user interface is designed with responsive design principles to ensure usability across devices and includes accessibility features such as screen reader compatibility and keyboard navigation.

3.2. Class Diagram

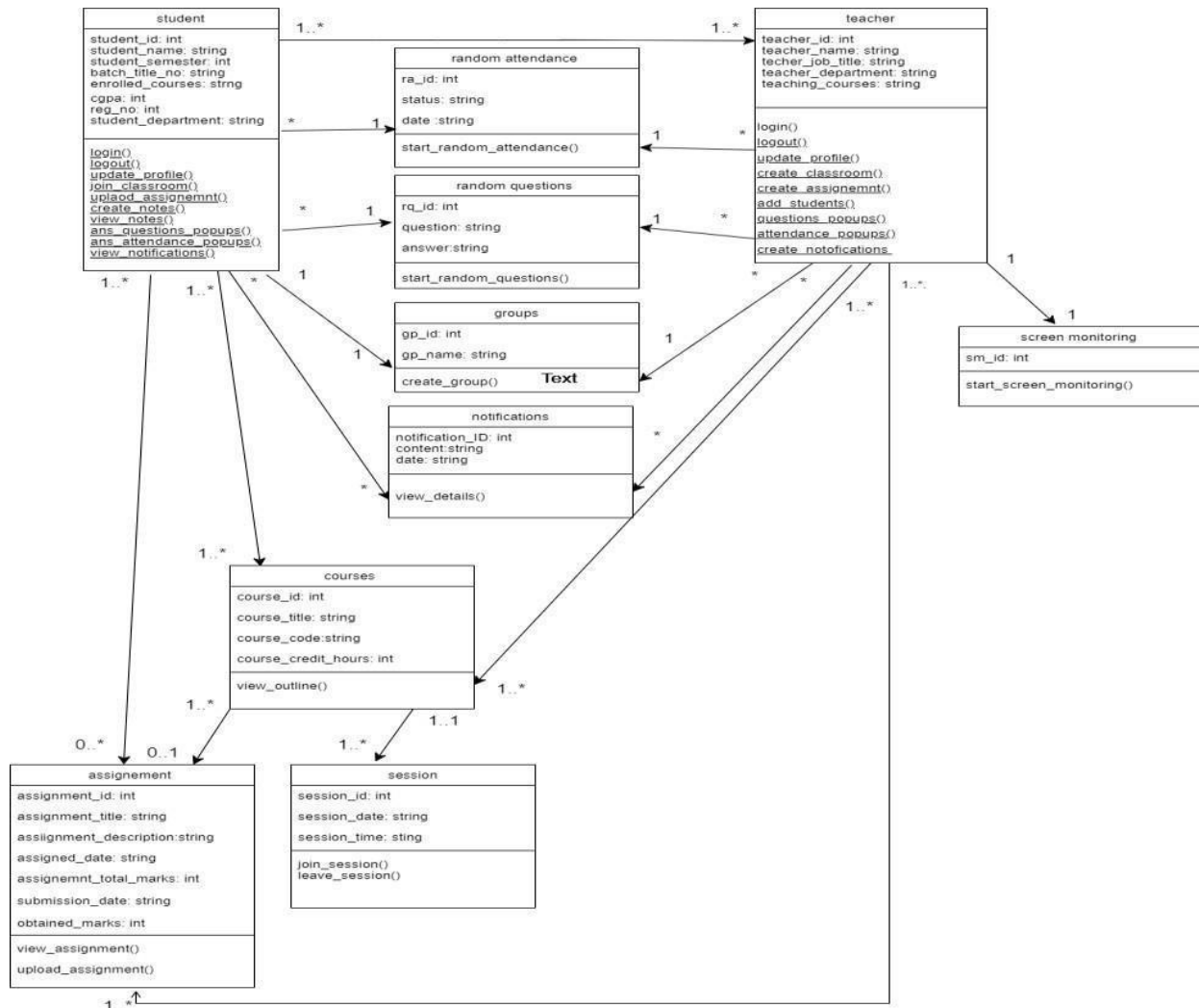


Figure 22: Class Diagram

In the class diagram for an e-classroom system, the primary classes include User, Student, Teacher, Course, Module, Lesson, Assignment, Submission, Message, and Notification. The User class serves as the base class with attributes like user-ID, name, email, password, role (such as Student, Teacher, or Admin), profile-Picture, and bio. It includes methods for basic functionalities such as login (), logout (), update-Profile (), and change-Password ().

3.3. Sequence Diagram

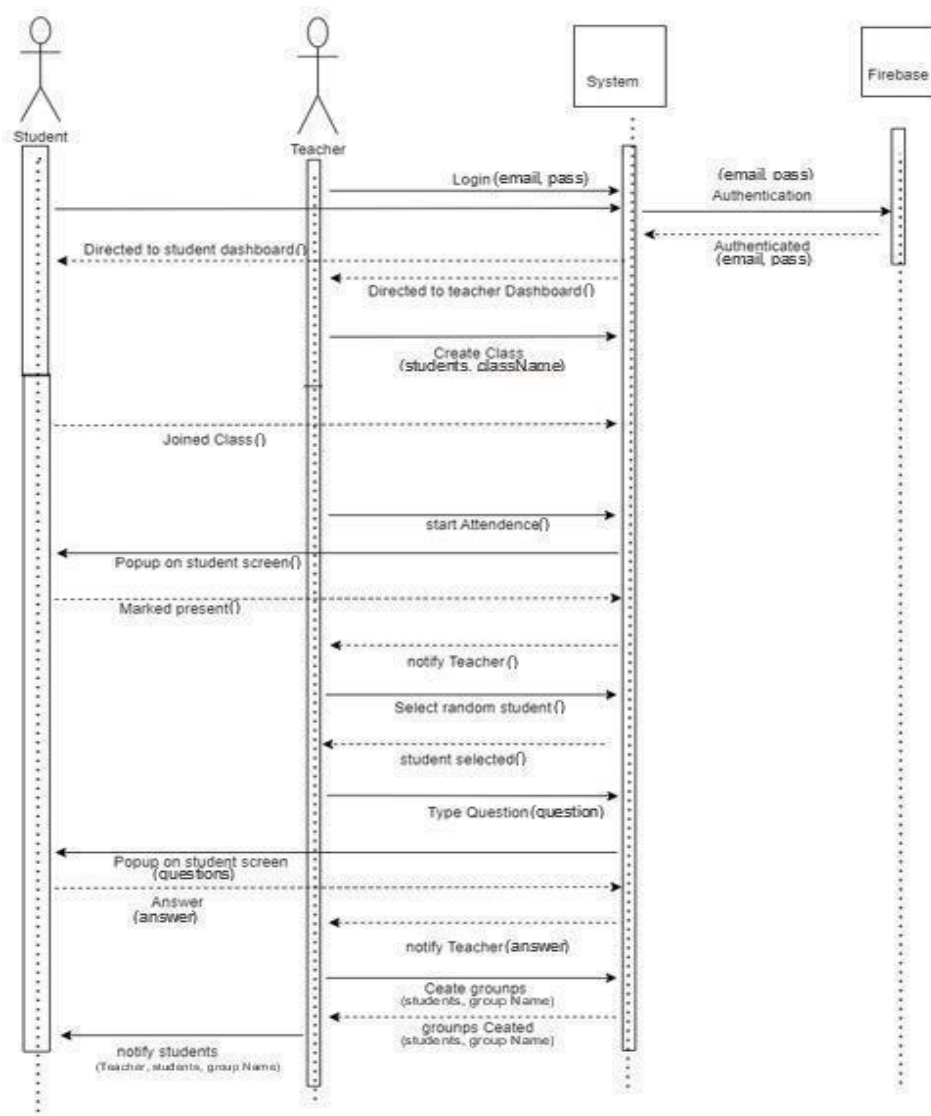


Figure 23: Sequence Diagram

In the above sequence diagram, it shows the total flow of the system which includes from students/teacher registration to the end process.

3.2. Entity Relationship Diagram

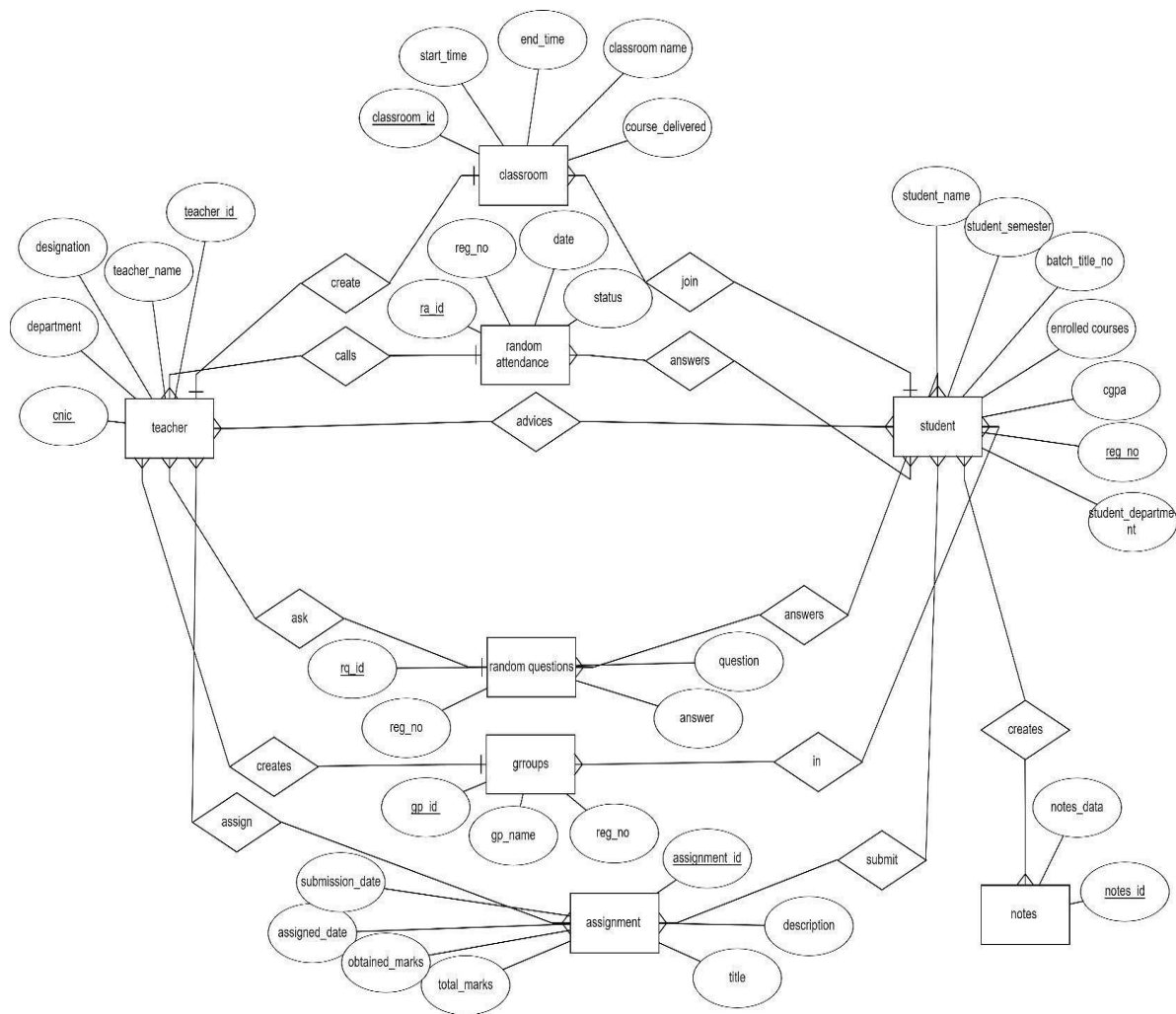


Figure 24: Entity Relationship Diagram

An E-classroom entity-relationship (ER) diagram is a visual representation of the data model for an online classroom system. It illustrates the relationships between various entities such as Students, Instructors and Assignments. Each entity is depicted as a rectangle, and the relationships between them are shown using connecting lines. Attributes of the entities, like student ID, course name, or assignment deadline, are often included in ovals or within the entity-rectangles.

3.5. Database Schema

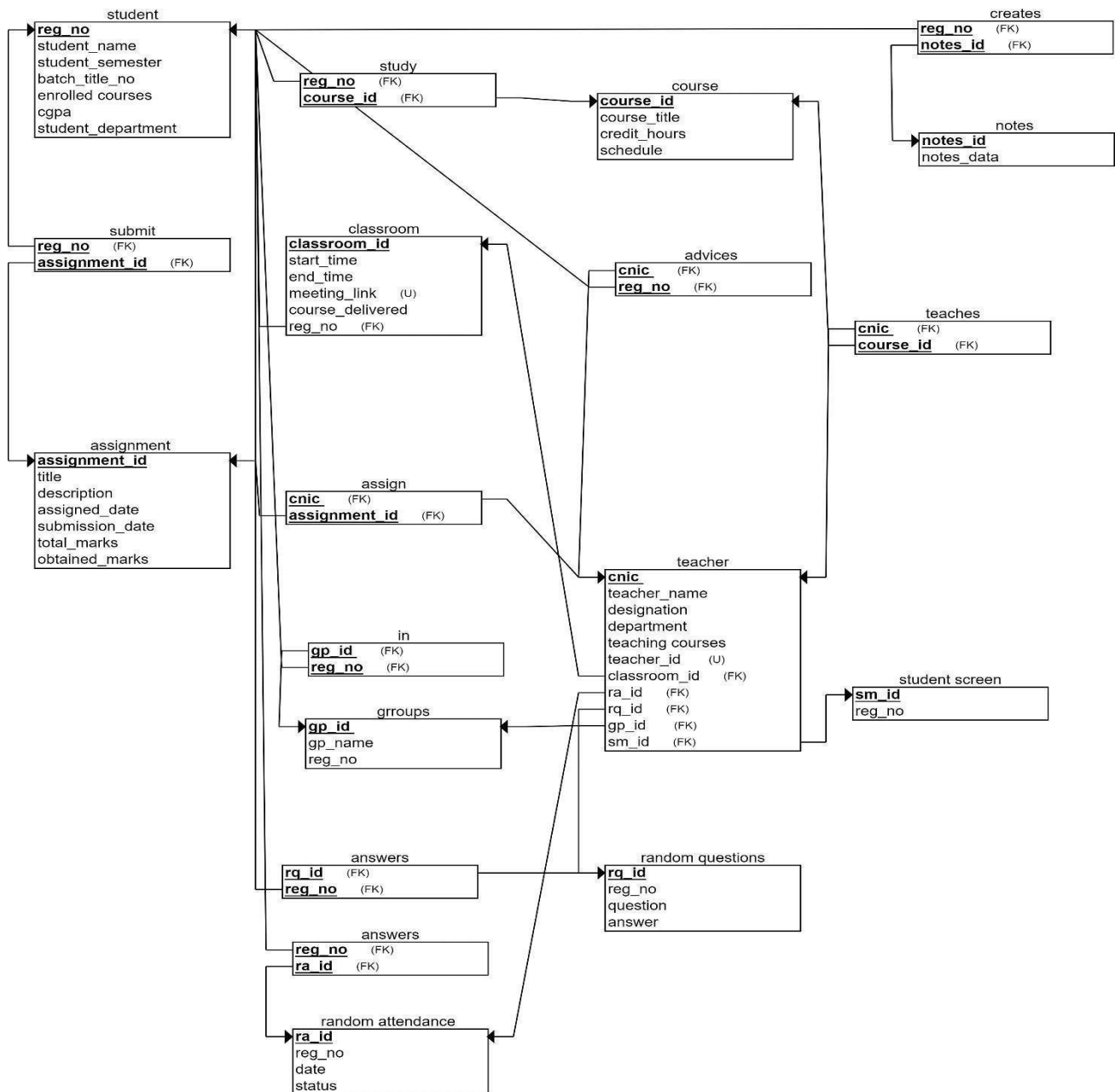


Figure 25: Database Schema

An E-classroom database schema defines the structure of a database for an online classroom system. It includes tables for entities like Students, Instructors. The schema specifies how these tables are related, ensuring data integrity and efficient query execution. It serves as a blueprint for the underlying database design.

3.6. User Interface Design

1. When user is student:

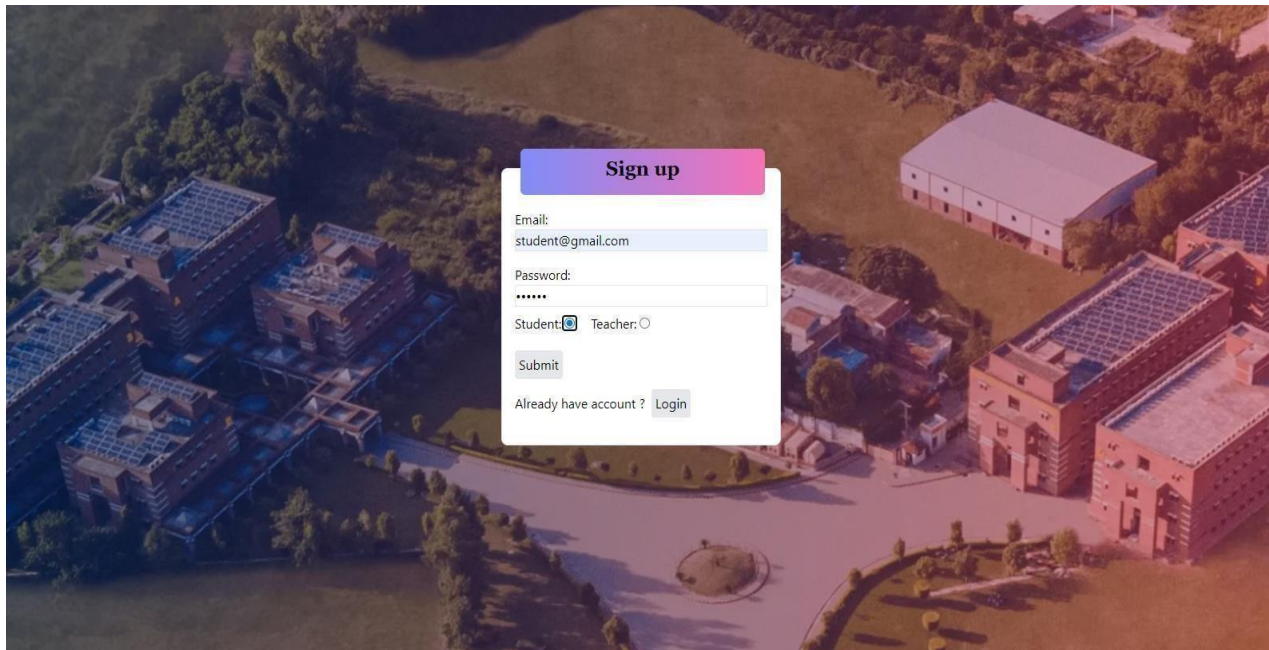


Figure 27: UI of student signup

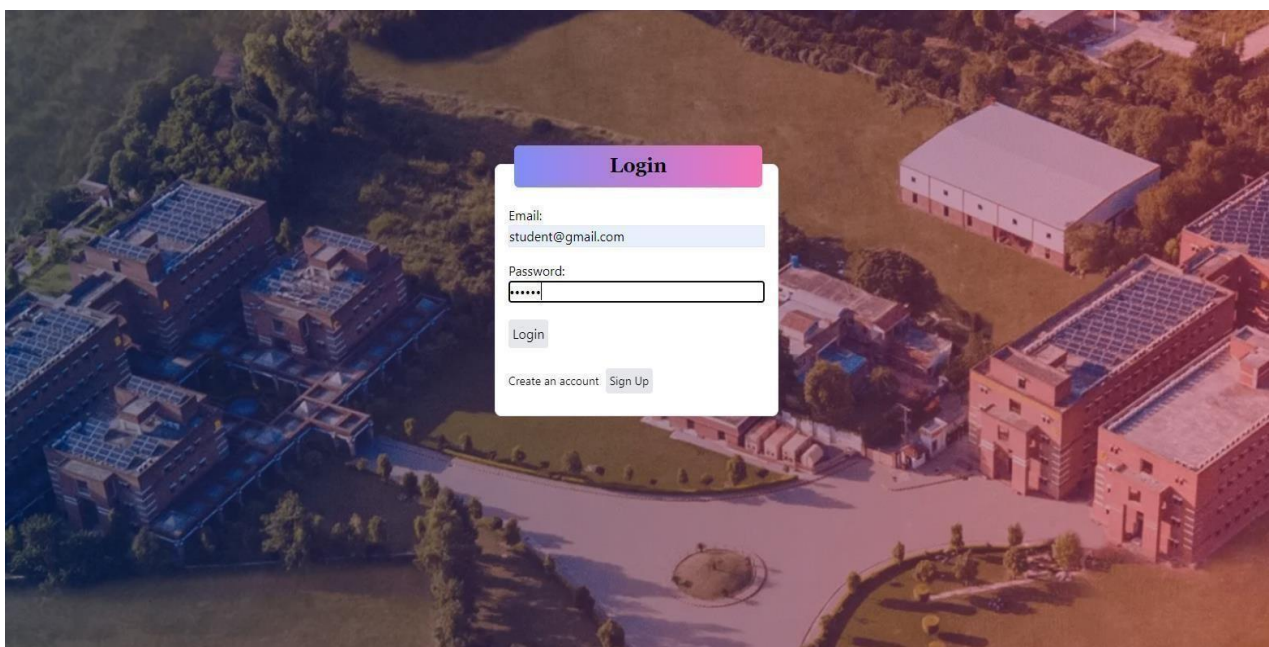


Figure 26: UI of student login

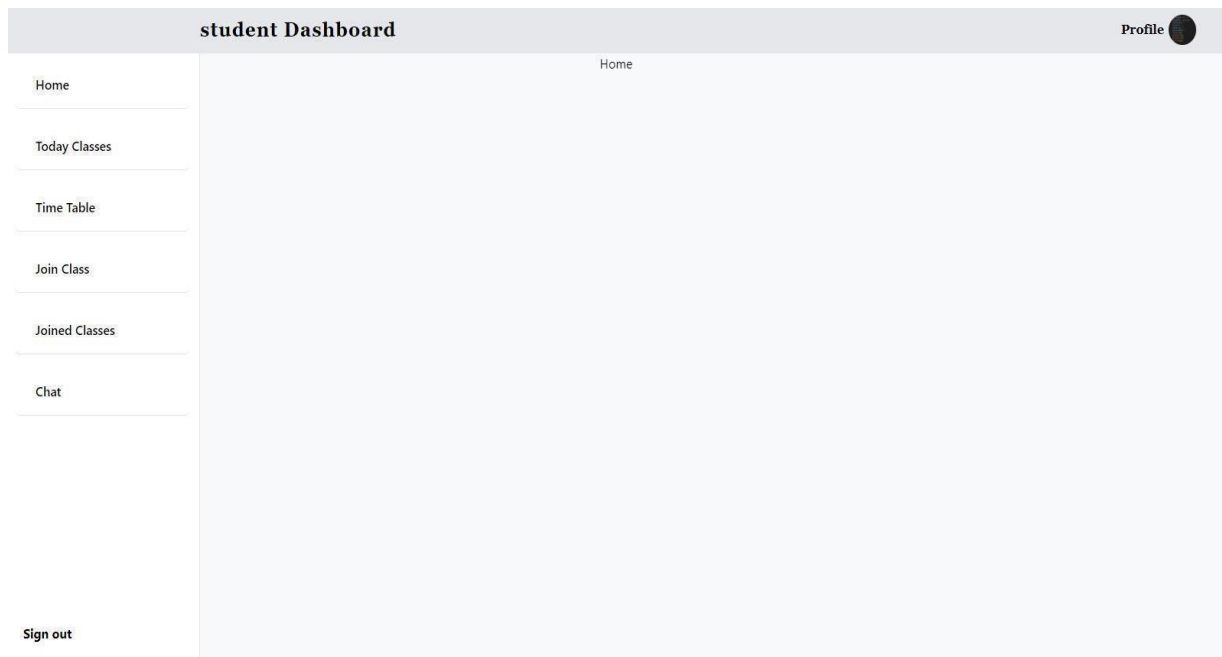


Figure 28: UI of student dashboard

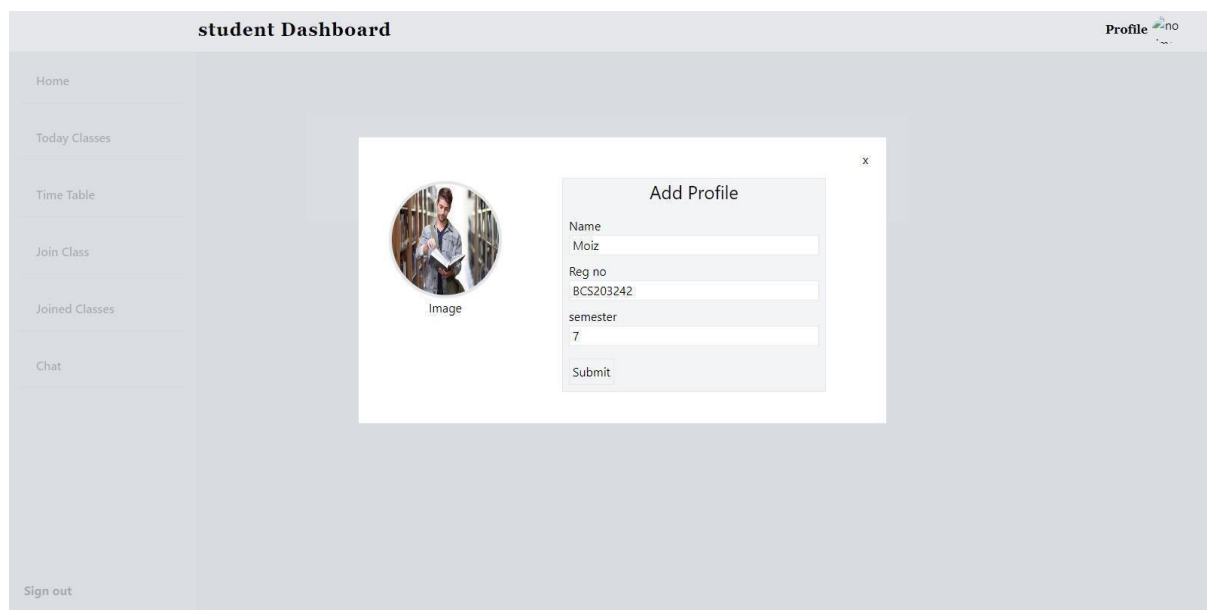


Figure 29: UI of student add profile

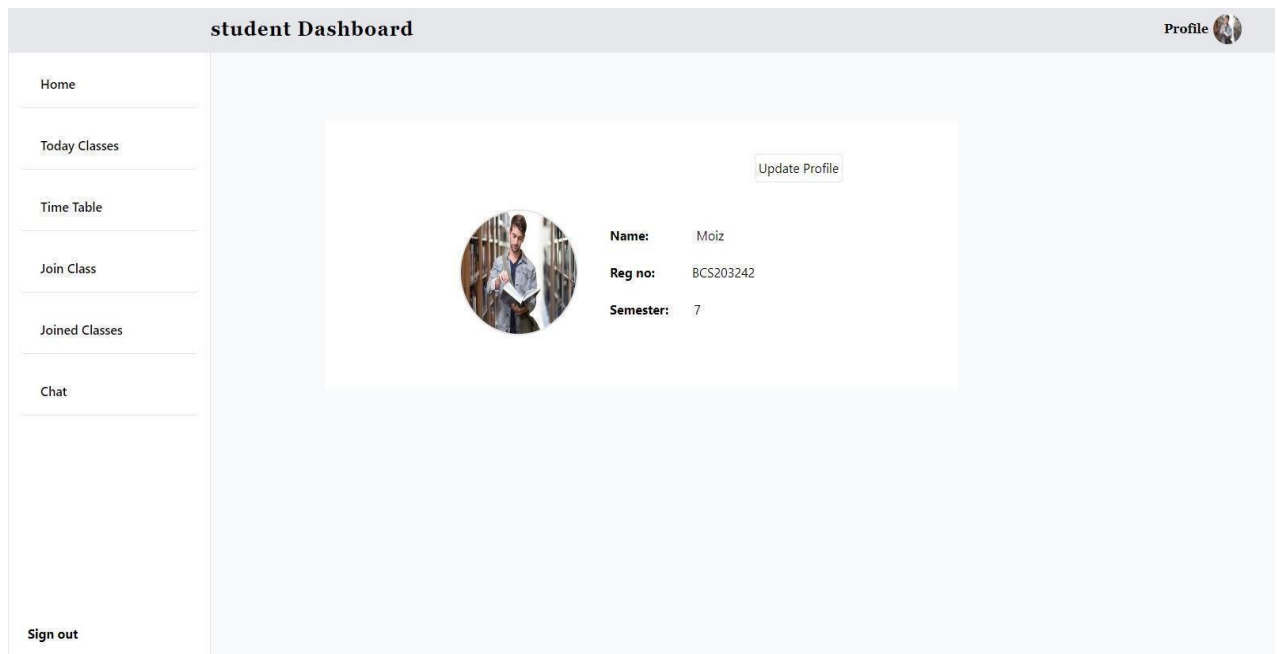


Figure 30: UI of student view profile

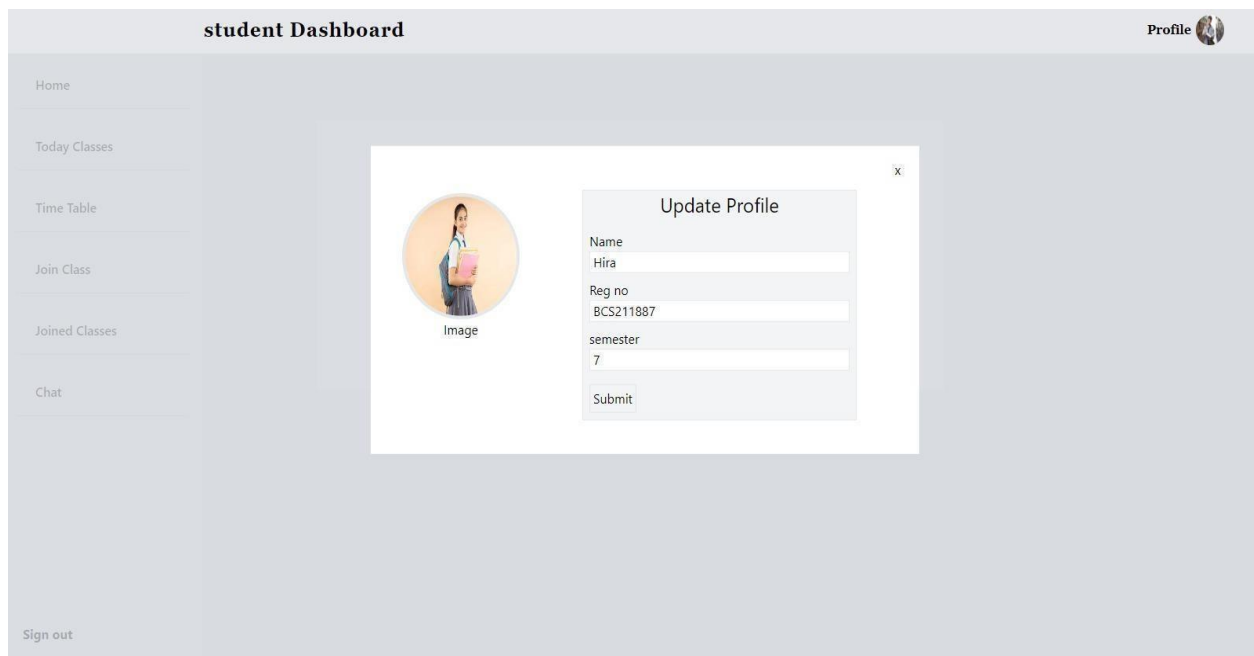


Figure 31: UI of student update profile

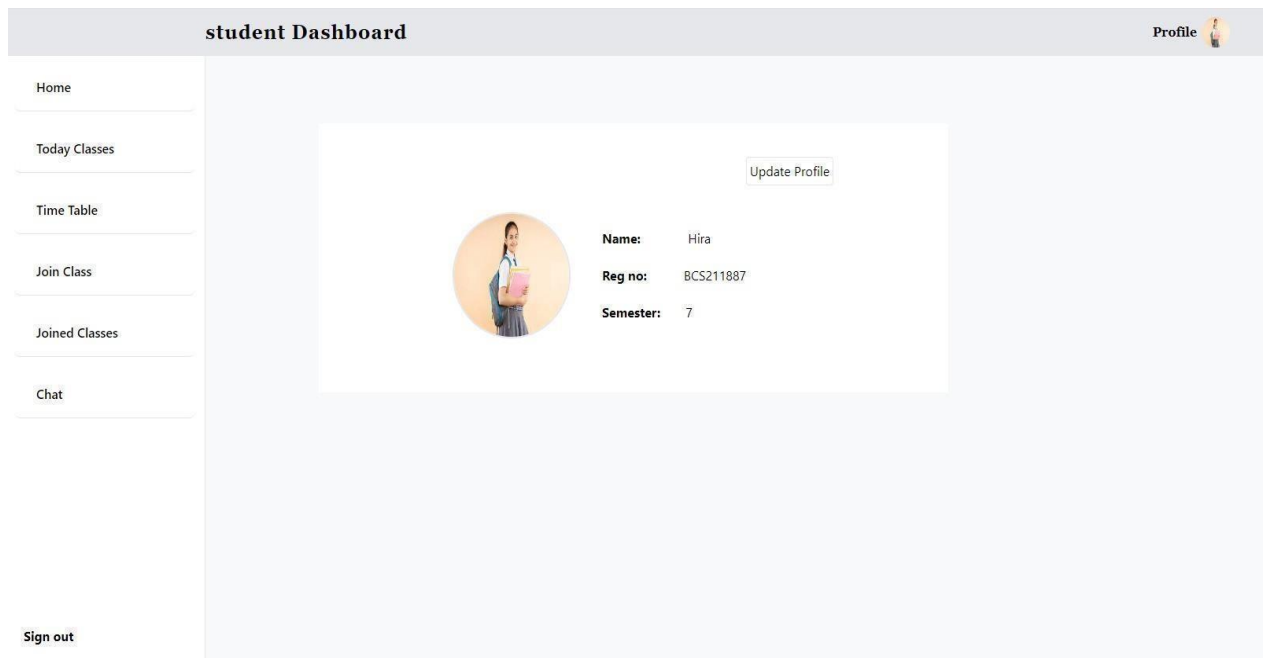


Figure 32: UI of student view updated profile

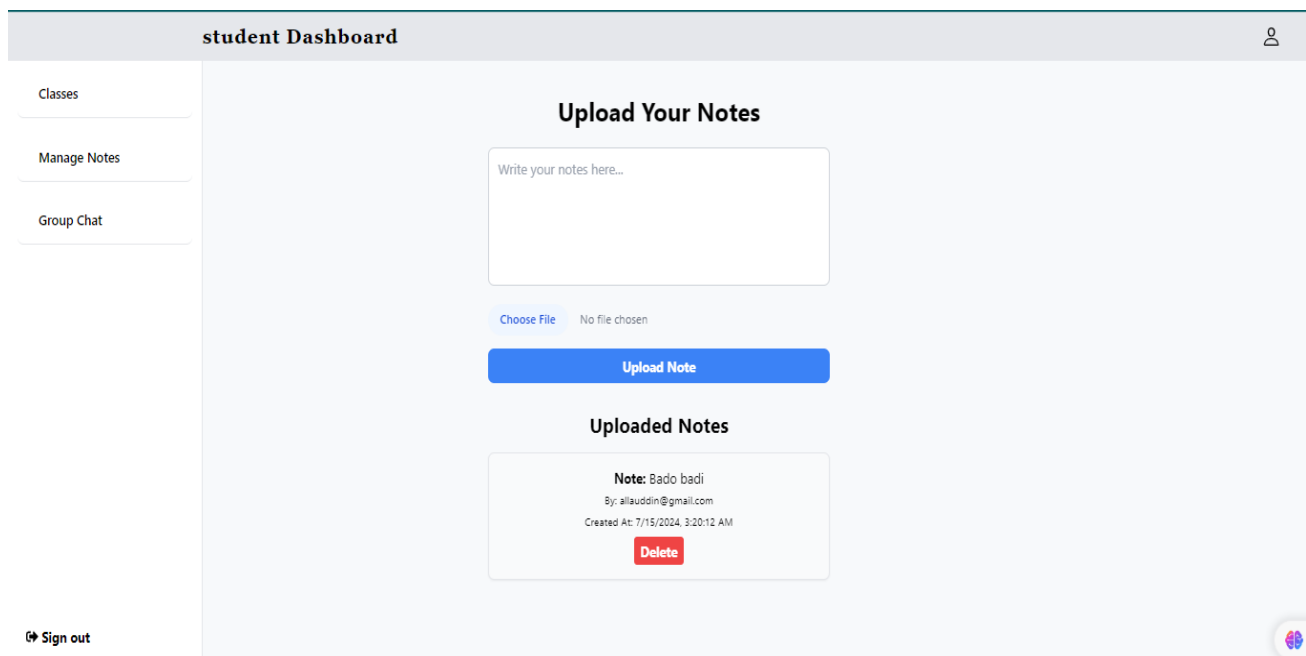


Figure 33: UI of student create and view notes

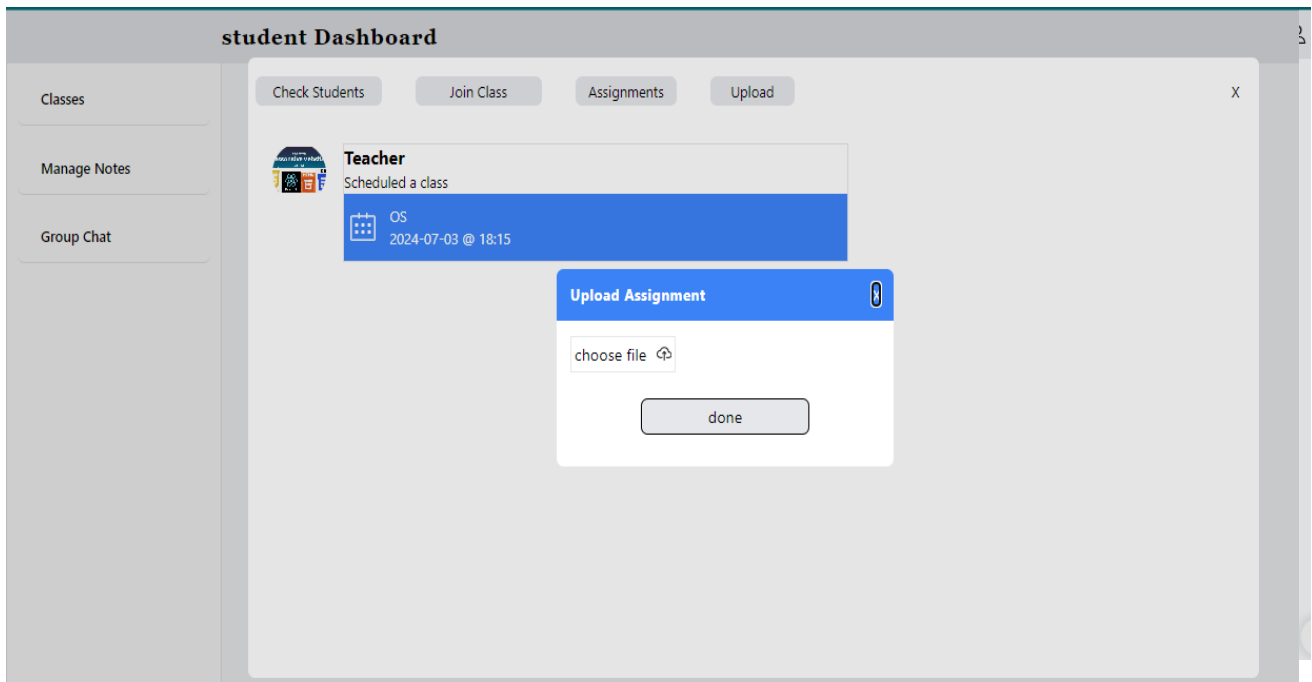


Figure 34: UI of student upload assignment solution

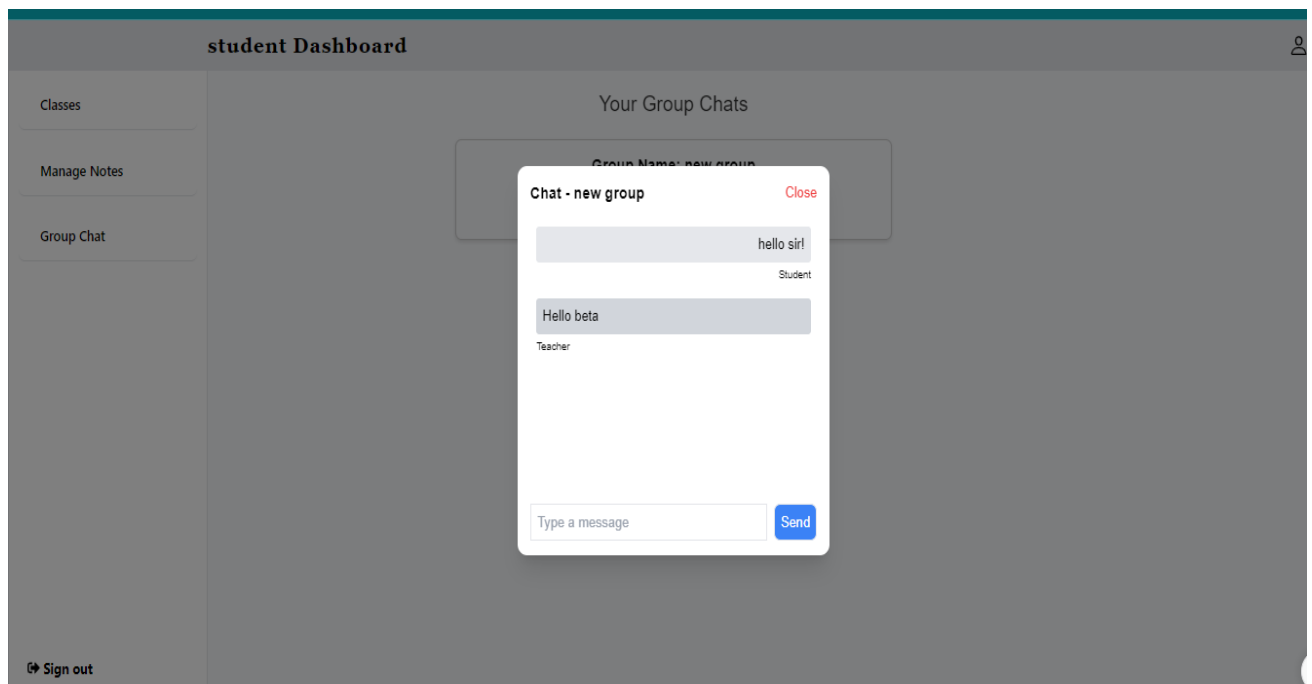


Figure 35: UI of student type and view group chat

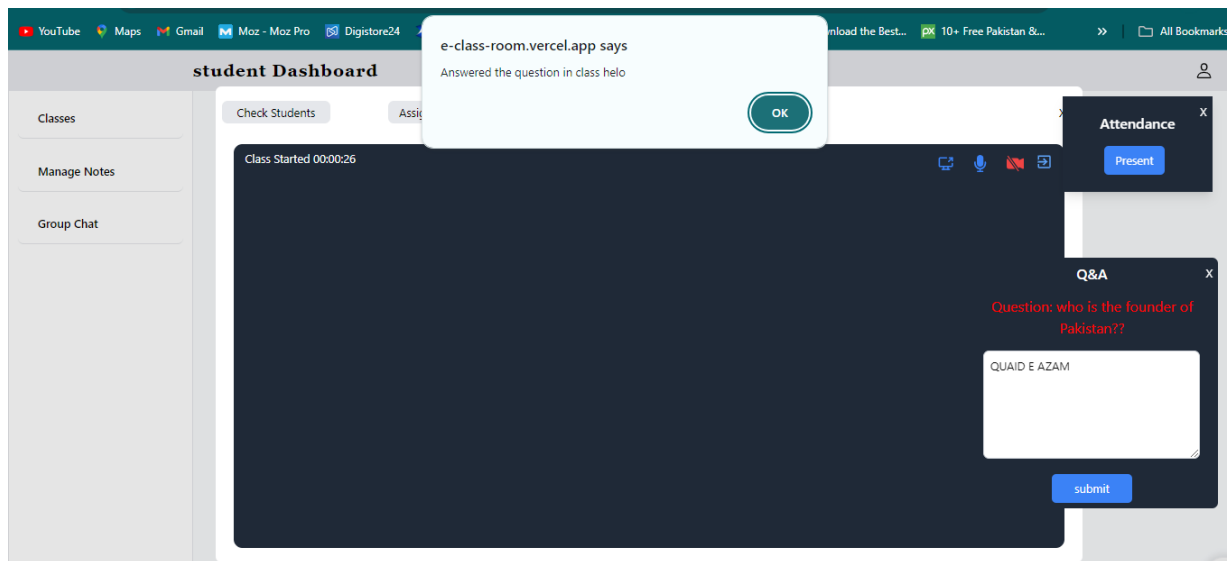


Figure 36: UI of student receiving random attendance and random question popups

2. When user is teacher:

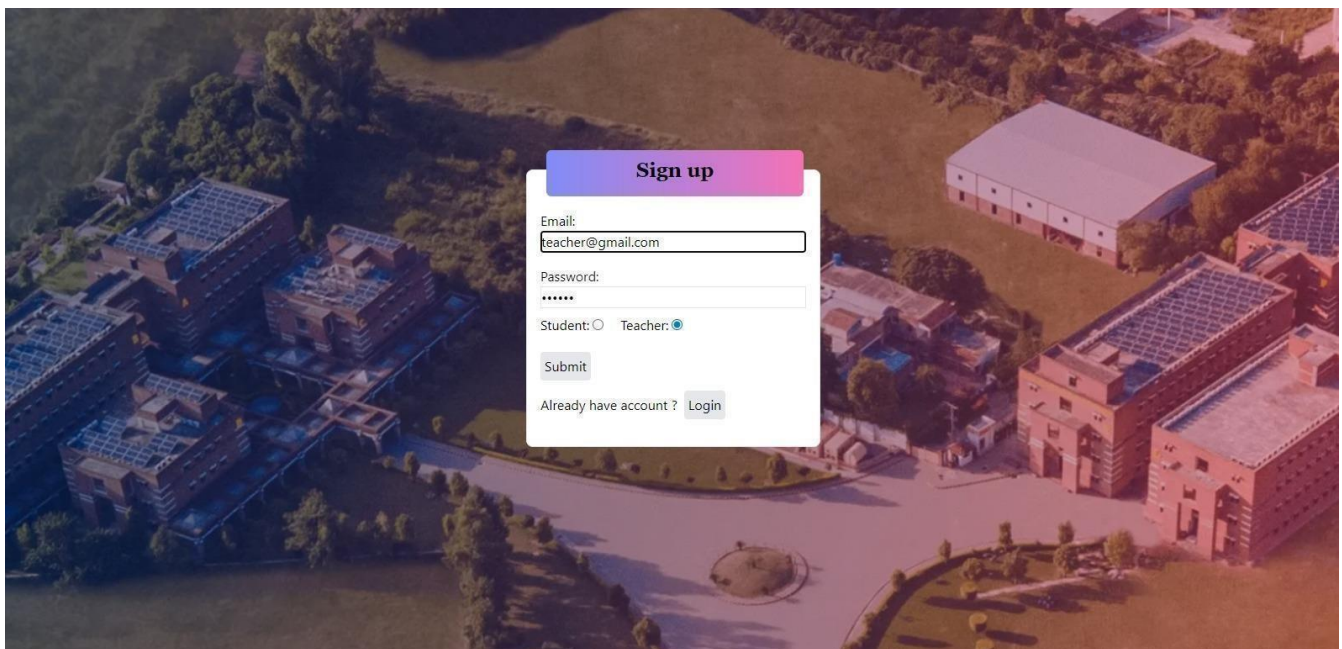


Figure 37: UI of teacher signup

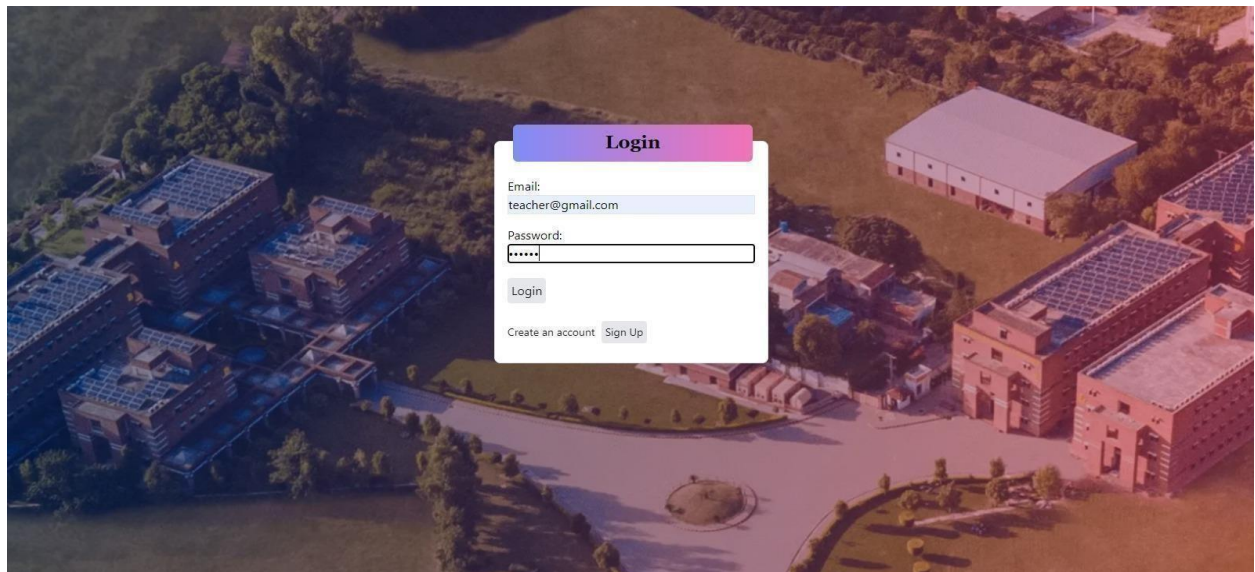


Figure 38: UI of teacher login

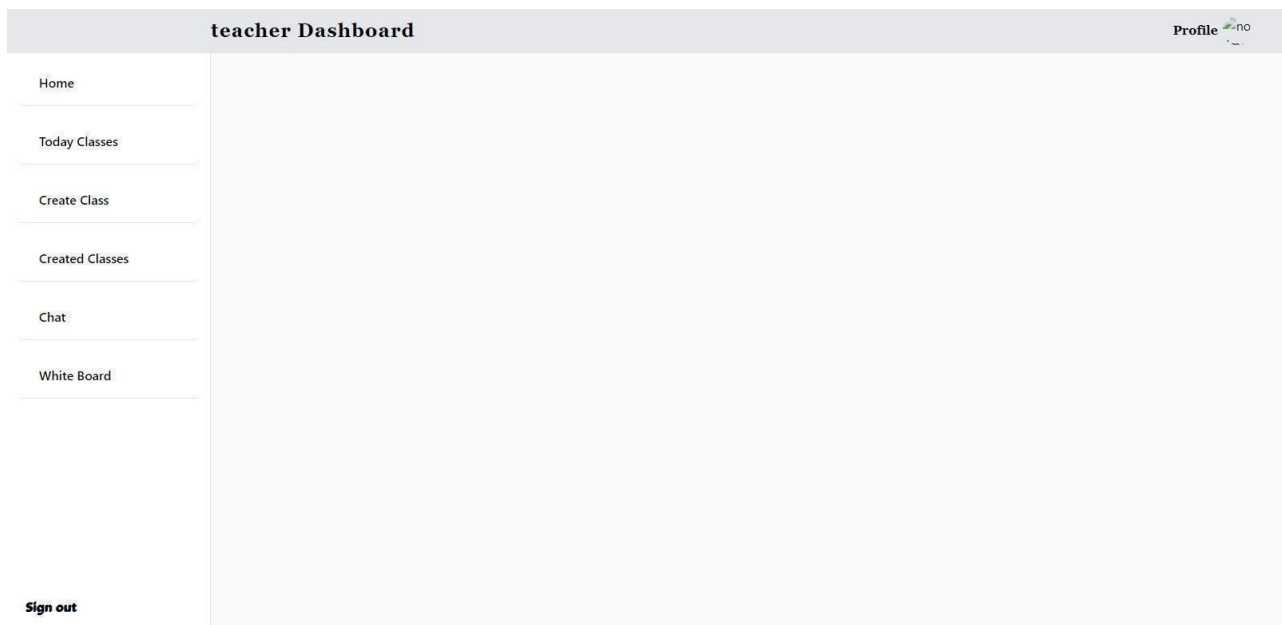


Figure 39: UI of teacher dashboard

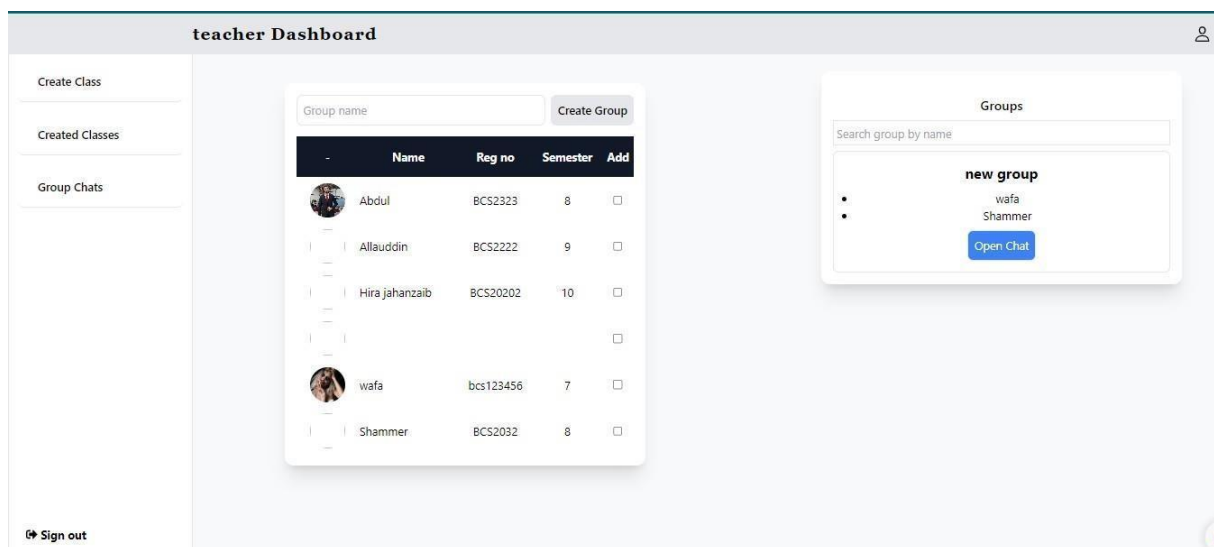


Figure 40: UI of teacher creating groups of students

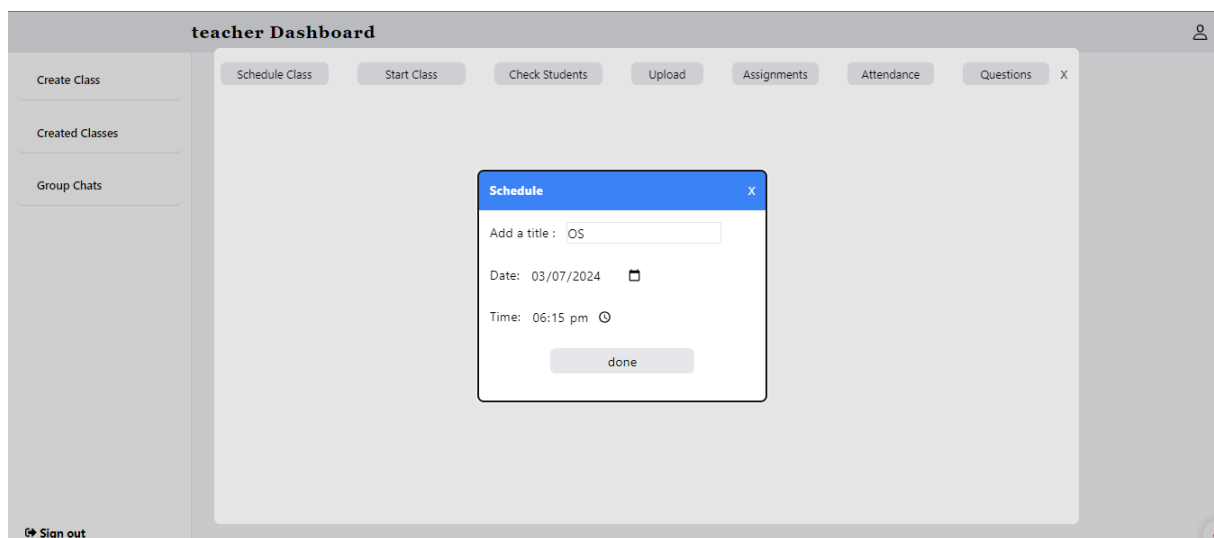


Figure 41: UI of teacher scheduling class

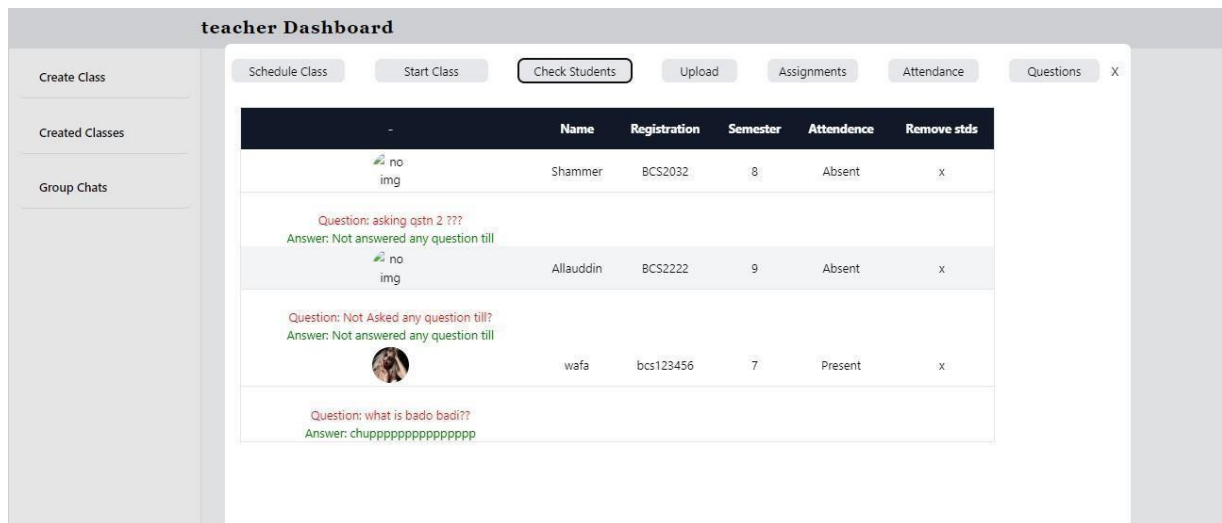


Figure 42: UI of teacher checking students' engagement status and can remove students

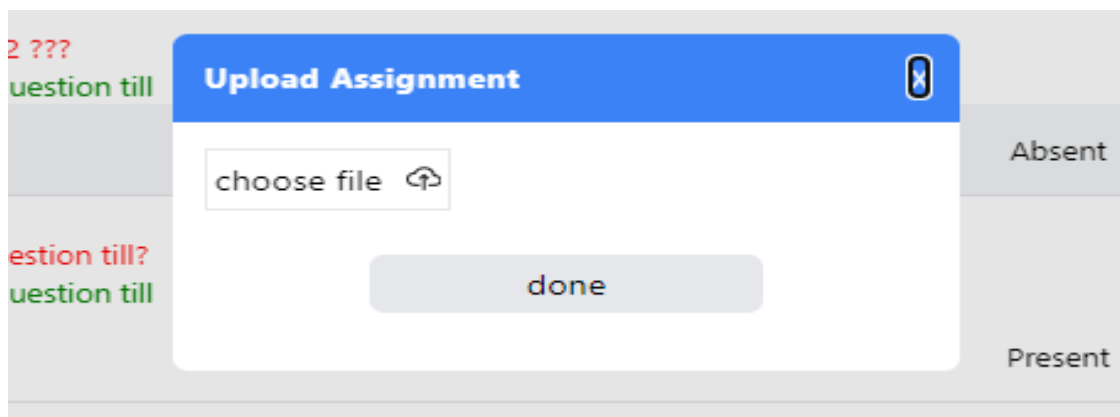


Figure 43: UI of teacher uploading assignment

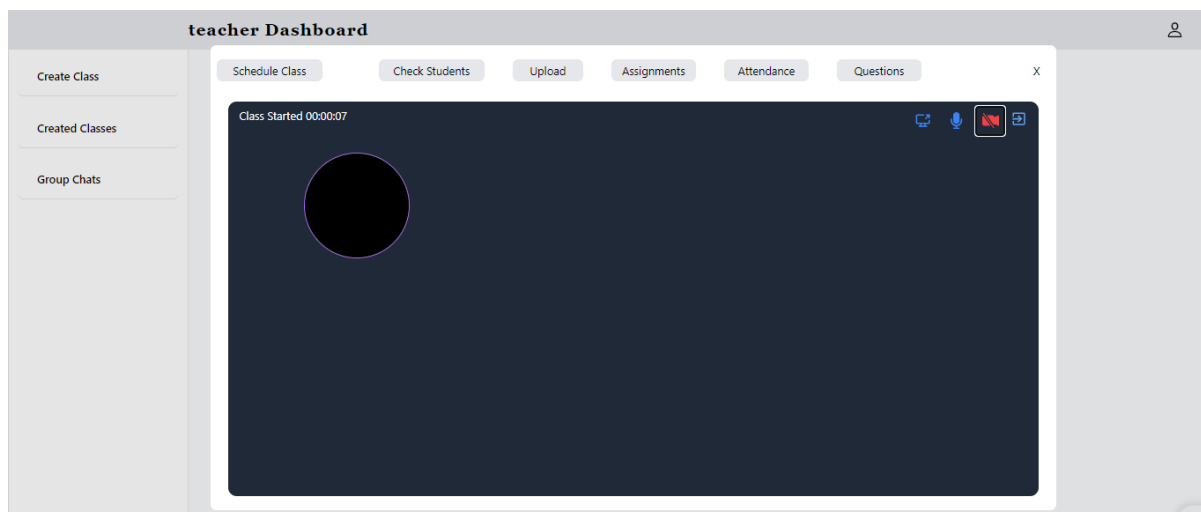


Figure 44: UI of teacher start class

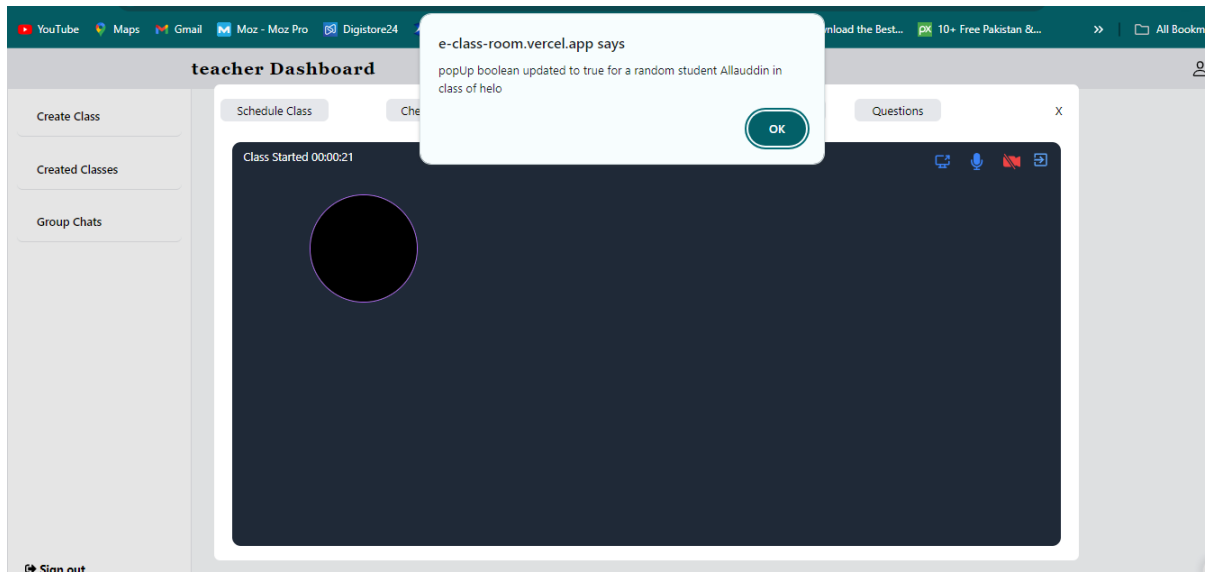


Figure 45: UI of teacher sending random attendance popup to student

3.7. Software COTS

React.js:

Identification and Version: React.js, version 17.0.2

Purpose: JavaScript library for building user interfaces or UI components.

Executable Information: Not an executable itself; it's used to build user interfaces in web applications.

Hardware/Software Resources: Minimal resource requirements. Executes in the user's web browser.

Firebase:

Identification and Version: Firebase SDK for JavaScript, version 9.6.5

Purpose: A comprehensive platform for building web and mobile applications, providing features like Realtime Database, Authentication, Cloud Functions, etc.

Executable Information: Not a standalone executable; it provides SDKs for various platforms.

Hardware/Software Resources: Utilizes cloud-based services. No specific hardware requirements.

Chapter 4

Software Development

4.1. Coding Standards

We have adopted the Camel Case naming convention and make sure that proper indentation is used in all of our code. This helps to maintain a clear and organized format, making it easier to read and understand. The use of indentation also enhances the overall readability and maintainability of the code. The names of methods are verbs, and the first letter of variables and methods is in lowercase. This helps to indicate that the name refers to a specific action or operation.

Maintaining good coding standards is essential for the success of any software development project. It helps to ensure that the code is of high quality, and it makes it easier for others to contribute to and maintain the code. By following these conventions, we are able to consistently produce code that is well-structured, easy to understand, and maintainable in the long term.

4.2. Development Environment

Tools that we are using:

- Visual Studio Code
- Visual studio

4.2.1 visual studio code

We are using VS Code because of its user-friendly interface, fast performance, and customizable settings. As a popular Integrated Development Environment (IDE) used in website development, VS Code provides a wide range of tools and features that make coding, debugging, and testing easier for developers. It offers syntax highlighting and auto-completion, helping to catch errors and improve code quality. VS Code supports multiple programming languages, making it versatile for web development. Furthermore, it offers various plugins and extensions, such as live server and prettier, to enhance its functionality. With its robust tools and features, VS Code streamlines the development process

4.2.2. visual studio code

Visual Studio is a software development environment used to create computer programs, including websites. It provides tools and features that help developers write, debug, and manage their code more efficiently. In web development, Visual Studio offers templates, code editing features, debugging tools, and integration with web technologies like HTML, CSS, and JavaScript. It streamlines the development process and allows developers to build websites faster and with fewer errors.

4.3. Software Description

4.3.1 Login process

```
const handleSubmit = (e) => {e.preventDefault();
if (!values.email || !values.password) {setError("Please fill all
fields");return;
}
setError(""); setSubmitButtonDisabled(true);
signInWithEmailAndPassword(auth, values.email,
values.password)
.then(async (res) => { setSubmitButtonDisabled(false);
const { uid: userId, displayName: role } = res.user;if (role === "student") {
navigate(`/studentDashboard/${userId}`);
} else if (role === "teacher") { navigate(`/teacherDashboard/${userId}`);
} else {
setError(res.err.
message);
}
})
.catch((err) => { setSubmitButtonDisabled(false);setError(err.message);
});
});
```

Description:

function for form submissions, handling user authentication in a web app. It prevents default form behavior, checks for filled email and password fields, and sets an error if empty. After disabling the submit button, it initiates authentication, extracting user information upon success and navigating to role-specific dashboards. Error messages are displayed

4.3.2 make profile:

```
const handleAdd = async (e) => {
  e.preventDefault();
  try {
    const userDocRef = doc(db, `teacherdata/${id}`);
    await setDoc(
      userDocRef,
      {
        ...data,
        timeStamp: serverTimestamp(),
      },
      { merge: true }
    );

    setData({
      namee: "",
      qua: "",
      des: "",
    });
    onClose();
  } catch (error) {
    console.log(error);
  }
};
```

Description:

The function begins by preventing the default behavior of a form submission. It then attempts to update a document in the "studentdata" collection with a specified id. The document is updated using the merging the existing data with a new object that includes the provided and a timestamp generated by the function. After the

update, it resets the local state (setData) by clearing, registration, semester), and then closes a modal or form (onClose). Any errors that occur during this process are caught and logged to the console. Overall, the function seems to handle the addition or modification of student data in a Fire store database

4.3.3. Update Profile:

```
const handleUpdate = async (e) => {
  e.preventDefault();
  try {
    const userDocRef = doc(db, `teacherdata/${id}/`);
    const docSnapshot = await getDoc(userDocRef);

    if (docSnapshot.exists()) {
      const updatedData = {
        ...data,
      };

      // Update the document with the new data
      await updateDoc(doc(db, `teacherdata/${id}/`), updatedData);

      setData({
        namee: "",
        qua: "",
        des: "",
      });

      onClose();
    } else {
      console.log("No matching document found.");
    }
  } catch (error) {
    console.log(error);
  }
}
```

Description:

The function starts by preventing the default behavior of a form submission. It then retrieves a document reference using the specified checks if the document exists using within the "studentdata" collection. Subsequently, it and proceeds to update the document if it does The update involves creating a new object (updatedData) by spreading the existing and then using the function to modify the document in the Firestore database. After the update, it resets local state (setData) by clearing specific fields (name, registration, semester) and sets the image field to an empty string. Additionally, it clears the file input the function handles the updatingof student data in the Firestore database, including the option to update an associated image file.

4.3.4. User (Student, Teacher) Profile:

```
useEffect(() => {
const fetchStudentData = async () => {
  try {
    // Construct the document reference
    const userDocRef = doc(db, `studentdata/${id}`);

    // Fetch the document
    const docSnapshot = await getDoc(userDocRef);

    // Check if the document exists
    if (docSnapshot.exists()) {
      // Access the data using the data() method
      const stdData = docSnapshot.data();
      setUserData(stdData);
    } else {
      console.log("Document does not exist");
    }
  } catch (error) {
    console.error("Error fetching user data:", error);
  }
};

fetchStudentData();
}, [id]);
```

Description:

Inside the effect, it constructs a document reference for a specific student (id) within the "studentdata" collection in Firestore. It then fetches the document using `getDoc`. If the document exists, it retrieves the student data using `data()` method, sets it using `setUserData`, and checks if there's an associated image. If an image is present, it constructs a storage reference, retrieves the image URL using `getDownloadURL`, and sets it as the image URL. If the document does not exist, a message is logged to the console. Any errors during this process are caught and logged as well. Overall, this is responsible for fetching and populating the component's state with student data and associated image URL.

4.3.5 Class:

For that he/she will add students:

```
const addStdInClass = (student) => {  
  //2  
  // Check if the student is already in stdList  
  const isStudentInList = stdList.some((std) => std.uid === student.uid); //3  
  
  if (isStudentInList) {  
    setStdList((prevList) =>  
      prevList.filter((std) => std.uid !== student.uid)  
    );  
  } else {  
    // If not, add the student to stdList  
    setStdList((prevList) => [...prevList, student]); //4  
  }  
};
```

4.3.6 Create button

```
const createClass = async (e) => {  
  e.preventDefault();  
  try {  
    stdList.forEach(student => {  
      student.popUp = false;  
      student.attendance = false;  
    });  
  
    const userDocRef = doc(db, `createClass/`, className);  
    await setDoc(userDocRef, {  
      ...stdList,  
      ClassTeacherID: id //that current Teacher who created class  
    });  
  
    console.log("Class created successfully");  
  } catch (error) {  
    console.error("Error creating class: ", error);  
  }  
};
```

Description:

function named `createClass`, which is likely associated with creating a class or course in a database. The function begins by preventing the default behavior of a form submission. It then constructs a document reference in the "createClass" collection with a specific class name (`className`). The function utilizes the `set` method to create or update the document, including class-related information such as student list (`stdList`), class timing (`classTime`), and the ID of the current teacher creating the class (`id`). After the class is created or updated, the function sets a timer for the current class by updating the state of `classTimers`. This state indicates the activation status of timers for different class timings. Any errors that occur during this process are caught and logged to the console. Overall, the function manages the creation or update of class information in the database and includes functionality for setting a timer for the newly created or updated class.

4.3.7 Created Class:

```
const fetchCreatedClass = async () => {
  try {
    const querySnapshot = await getDocs(collection(db, "createClass"));
    let data = {};
    querySnapshot.forEach((doc) => {
      //return only those documents who matches with current teacher id
      const docData = doc.data();
      if (docData.ClassTeacherID === id) {
        data[doc.id] = docData;
      }
    });
    setCreatedClass(data);
  } catch (error) {
    console.error("Error fetching data:", error);
  }
};
```

Description:

The hook to fetch and filter classes based on a student's UID. Upon component mount or when the dependency changes, the function is executed. This function initiates an asynchronous query to retrieve all documents from the "createClass" collection in Firestore. The retrieved data is then transformed into an array of objects, each containing the document ID and data for debugging purposes. The final step involves updating the component state (setFetchClasses) with the matched classes, or an empty array if no matches are found. Any errors encountered during this process are caught and logged to the console. In summary, this useEffect is responsible for fetching and updating the state with classes where a student with a specific UID is enrolled from the Firestore database.

4.3.8 Send Notification:

```
const sendNotificationToStudent = (student, notify) => {  
  console.log("Time matched");  
  let body = {  
    to: student.messagingToken,  
    notification: {  
      title: "Class Notification ",  
      body: `${student.name} class is starting in 5 minutes, Be ready`,  
    },  
  },
```

Description:

The provided code includes functions related to notifying students about a class starting 5 minutes before the scheduled time. These functions collectively form a system for notifying students about classes starting soon. The notification is triggered 5 minutes before the scheduled class time, and it includes information about the class and encourages students to be ready. The code also logs relevant information to the console for debugging purposes. These functions collectively form a system for notifying students about classes starting soon. The notification is triggered 5 minutes before the scheduled class time, and it includes information about the class and encourages students to be ready. The code also logs relevant information to the console for debugging purposes.

4.3.9. Search Student by Reg no:

```
const searchStudentFunc = () => {  
  const foundStudent = fetchStudentData.find(  
    (student) => student.reg.toLowerCase() === takeName.toLowerCase()  
  );  
  setFoundStudent(foundStudent);  
  setSearched(true);  
};
```

Description:

1 searchStudentFunc Function:

- Logs the takeName variable to the console.
- Utilizes the find method to search for a student in the fetchStudentData array.
- Compares the lowercase registration numbers to find a match with the provided takeName.
- Sets the found student in the foundStudent state variable

Sets the searched state variable to true, indicating that a search has been performed

2. showAllStudents Function:

- Resets the searched state variable to false, indicating that the component should display all students

4.3.10 Teacher can upload assignment:

```
const uploadAssignment = () => {  
  if (assignmentFile == null) return;  
  const fileRef = ref(  
    storage,  
    `thrAssignments/${className}/${assignmentFile.name}`  
  );  
  uploadBytes(fileRef, assignmentFile).then(() => {  
    alert("uploaded");  
    let closing = document.getElementById("modal3");  
    closing.close();  
  });  
};
```

Description:

The uploadAssignment function manages the process of uploading an assignment file to Firebase Storage. It begins by checking if an assignment file is present. If so, it creates a reference to the storage location based on the class name and file name. Then, it uploads the file bytes to this location and displays an "uploaded" alert upon completion. Additionally, it attempts to close a modal dialog identified by the ID "modal3".

4.3.11 Student can download assignment:

```
useEffect(() => {  
  // Create an array to store all download promises  
  const downloadPromises = [];  
  
  assigRefs.forEach((refInstance) => {  
    const listPromise = listAll(refInstance).then((response) => {  
      response.items.forEach((item) => {  
        const downloadPromise = getDownloadURL(item).then((url) => {  
          const fileName = item.name;  
          let removedItem = stdName.shift();  
          setAssignment((prev) => [...prev, { url, fileName, removedItem }]);  
        });  
        downloadPromises.push(downloadPromise);  
      });  
    });  
    downloadPromises.push(listPromise);  
  }, []);  
});
```

Description:

The `useEffect` hook is employed to download assignments for each reference upon component mounting. It initializes an array, `downloadPromises`, to store all download promises. For each reference instance in the `assigRefs` array, it creates a promise `listPromise` to list all items under that reference. Upon resolving the list promise, it iterates over each item in the response, creating a download promise `downloadPromise` to obtain the download URL of the file. Upon successful download, the URL and filename of the file are added to the assignment state array. The component also provides a cleanup function to cancel ongoing download operations upon unmounting. Finally, it waits for all download promises to resolve using `Promise.all`

4.3.12 User can start class

```
let joinRoomInit = async (name) => {
  try {
    client.on("user-unpublished", (user, mediaType) => {
      if (mediaType === "audio" && user.audioTrack) user.audioTrack.stop();
      if (mediaType === "video") {
        setUsers((prevUsers) =>
          prevUsers.filter((User) => User.uid !== user.uid)
        );
      }
    });

    await client.join(config.appId, name, config.token, config.uid);
    client.on("user-published", handleUserPublished);
    client.on("user-left", handleUserLeft);
    joinStream();
    setStart(true);
  } catch (error) {
    console.error("Error initializing video call:", error);
  }
};
```

Description:

The `useEffect` hook is utilized for executing code upon component mounting, operating similarly to `componentDidMount` in class components. It's set to run only once with an empty dependency array. Within this hook, the `joinRoomInit` function is defined asynchronously to initialize and join the video call room. Event listeners are established to handle user actions such as media unpublishing, joining, and leaving. The code attempts to join the video call using the Agora client, providing essential parameters. Error handling is implemented to log any initialization errors. HTML elements are inserted to display video streams, with an event listener added to manage container expansion. There's an attempt to play video tracks associated with user IDs, but a syntax error needs correction.

4.3.13. User can join classroom:

```
let joinStream = async () => {
  const userName = await fetchUserName(id);
  let player = `
<div>
<div class="video__container" id="user-container-${uid}">
<div class="video-player" id="user-${uid}"></div>
</div>
<div class="user-name">${userName}</div>
</div>
`
}
```

Description:

The `useEffect` hook in this code snippet orchestrates the setup and initialization process for a video call room within a React component. When the component mounts or certain dependencies change, it triggers the execution of two asynchronous functions. The first function, `joinRoomInit`, manages the initialization of the video call room. It establishes event listeners for various user-related events such as `user-unpublished`, `user-published`, and `user-left`. Subsequently, it proceeds to join the video call room and incorporates the local user's video stream into the session. The second function, `joinStream`, focuses on integrating the local user's video stream into the room. It accomplishes this by generating HTML markup to display the video stream, dynamically adding it to the DOM, and configuring an event listener for expanding the video display. Additionally, `joinStream` takes care of publishing both the audio.

4.3.14 User Can Mute Microphone/ toggle camera

```
const mute = async (type) => {
  if (type === "audio") {
    await tracks[0].setEnabled(!trackState.audio);
    setTrackState((ps) => {
      return { ...ps, audio: !ps.audio };
    });
  } else if (type === "video") {
    await tracks[1].setEnabled(!trackState.video);
    setTrackState((ps) => {
      return { ...ps, video: !ps.video };
    });
  }
}
```

Description:

The mute function serves as an asynchronous handler responsible for toggling the audio or video state of a media track, contingent upon the type parameter passed to it. When invoked with the type set to "audio," the function toggles the audio state of the first track within the tracks array, subsequently updating the trackState to accurately reflect the new audio state. Similarly, if the type parameter is "video," the function toggles the video state of the second track in the tracks array, ensuring synchronization with the updated trackState. The utilization of the await keyword within the function ensures that the track state is promptly updated before proceeding with subsequent execution, guaranteeing synchronous behavior and accurate reflection of the track states.

4.3.15 teacher can remove student

```
//Remove students from class
const DeleteStudents = async (id) => {
  try {
    // Loop through each class document in the createdClass object
    for (const className in createdClass) {
      if (Object.hasOwnProperty.call(createdClass, className)) {
        const classData = createdClass[className];
        // Check if the student ID exists in the current class data
        for (const studentId in classData) {
          if (Object.hasOwnProperty.call(classData, studentId)) {
            const student = classData[studentId];
            // Check if the student's id matches the specified id
            if (student.id === id) {
              // Delete the student object from the class data
              delete classData[studentId];
              // Update the Firestore document with the modified class data
              fetchCreatedClass();
              await updateClassDocument(className, classData);
              console.log("Student document deleted successfully");
              return; // Exit the function after deleting the student document
            }
          }
        }
      }
    }
    console.log(`Student with ID ${id} not found in any class`);
  } catch (error) {
    console.error("Error deleting student document: ", error);
  }
};
```

Description:

The DeleteStudents function asynchronously removes a student from all classes in the createdClass object. It loops through each class and student to find and delete the student with the specified ID. Once found, it updates the Firestore document and calls fetchCreatedClass to refresh the class data. If the student ID is not found, it logs a message.

4.3.16 user can share screen

```
let toggleScreen = async (e) => {
  setScreenShare(false);

  if (!sharingScreen) {
    sharingScreen = true;

    localScreenTracks = await AgoraRTC.createScreenVideoTrack();

    userIdInDisplayFrame = `user-container-${uid}`;
    localScreenTracks.play(`user-${uid}`);

    await client.unpublish([tracks[1]]);
    await client.publish([localScreenTracks]);

    let videoFrames = document.getElementsByClassName("video_container");
    for (let i = 0; videoFrames.length > i; i++) {
      if (videoFrames[i].id !== userIdInDisplayFrame) {
        videoFrames[i].style.height = "100px";
        videoFrames[i].style.width = "100px";
      }
    }
  } else {
    sharingScreen = false;

    document.getElementById(`user-container-${uid}`).remove();
    await client.unpublish([localScreenTracks]);

  }
};
```

Description:

The toggleScreen function handles the toggling of screen sharing for a video call. When the user starts sharing their screen, it creates a screen video track, updates the display frame, and publishes the screen track to the Agora client while unpublishing the camera track. It also adjusts the video frames for all other users. When stopping screen sharing, it removes the screen track, republishes the camera track, and reverts the display to the camera feed.

4.3.17 student can create and save notes

```
const handleSaveNotes = async () => {
  if (!notes.trim() && !file) {
    setError("Please provide notes or select a file.");
    return;
  }
  try {
    if (!user) {
      setError("User is not authenticated.");
      return;
    }

    let fileUrl = "";
    if (file) {
      const fileRef = ref(storage, `notes/${user.uid}/${file.name}`);
      await uploadBytes(fileRef, file);
      fileUrl = await getDownloadURL(fileRef);
    }

    await addDoc(collection(db, "notes"), {
      uid: user.uid,
      email: user.email,
      note: notes.trim(),
      fileUrl: fileUrl,
      createdAt: new Date(),
    });
    setNotes("");
    setFile(null);
    setError("");
    alert("Notes saved successfully!");
  } catch (err) {
    console.error("Error saving notes:", err);
    setError("Error saving notes. Please try again.");
  }
};
```

Description:

he handleSaveNotes function saves user notes and optionally a file to a Firestore collection. It first checks if notes or a file are provided, and if the user is authenticated. If a file is selected, it uploads the file to Firebase Storage and retrieves its URL. Then, it saves the notes, file URL, user ID, email, and timestamp to the Firestore "notes" collection. After saving, it resets the form and handles any errors by displaying appropriate messages.

4.3.18 student can view notes

```
useEffect(() => {
  const fetchNotes = async () => {
    if (!user) return;

    try {
      const q = query(
        collection(db, "notes"),
        where("uid", "==", user.uid), // Filter notes by current user's uid
        orderBy("createdAt", "desc") // Order notes by createdAt descending
      );

      const unsubscribe = onSnapshot(q, (snapshot) => {
        const fetchedNotes = snapshot.docs.map((doc) => ({
          id: doc.id,
          ...doc.data(),
        }));
        setNotesList(fetchedNotes);
      });

      return () => unsubscribe();
    } catch (error) {
      console.error("Error fetching notes:", error);
      setError("Failed to fetch notes. Please try again.");
    }
  };

  fetchNotes();
}, [user]);
```

Description:

The `useEffect` hook fetches and listens for real-time updates to the user's notes from a Firestore collection. It defines an asynchronous function, `fetchNotes`, that queries the "notes" collection, ordering the results by the creation date. It uses `onSnapshot` to listen for changes and updates the local state with the fetched notes. If the user is not authenticated, the function exits early. The hook also cleans up the subscription when the component unmounts or the user changes. Errors during fetching are caught and handled appropriately.

4.3.19 student can delete notes

```
const handleDeleteNote = async (noteId, fileUrl) => {
  try {
    if (fileUrl) {
      const fileRef = ref(storage, fileUrl);
      await deleteObject(fileRef);
    }
    await deleteDoc(doc(db, "notes", noteId));
    setNotesList((prevNotes) =>
      prevNotes.filter((note) => note.id !== noteId)
    );
    alert("Note deleted successfully!");
  } catch (err) {
    console.error("Error deleting note:", err);
    setError("Error deleting note. Please try again.");
  }
};
```

Description:

The handleDeleteNote function allows for the deletion of a note and associated file (if applicable) from Firestore and Firebase Storage. It first checks if a fileUrl exists; if so, it deletes the corresponding file from Firebase Storage using deleteObject. Next, it deletes the document with noteId from the Firestore collection using deleteDoc. After successful deletion, it updates the local notesList state by filtering out the deleted note. Any errors encountered during deletion are caught and handled by setting an error message for user feedback. Finally, it alerts the user upon successful deletion of the note.

4.3.20 teacher create groups

```
const createGroup = async (e) => {
  e.preventDefault();
  if (!groupName || selectedStudents.length === 0) {
    alert("Group name and at least one student are required.");
    return;
  }

  try {
    const groupData = {
      groupName,
      members: selectedStudents,
      createdBy: id,
    };

    const groupDocRef = doc(db, `groups/${groupName}`);
    await setDoc(groupDocRef, groupData);

    setGroupList((prevList) => [...prevList, groupData]);
    setFilteredGroups((prevList) => [...prevList, groupData]);
    setGroupName("");
    setSelectedStudents([]);
    console.log("Group created successfully");
  } catch (error) {
    console.error("Error creating group: ", error);
  }
};
```

Description:

The createGroup function handles the creation of a group in a Firestore database. It first prevents the default form submission behavior and checks if both a groupName and at least one student (selectedStudents) are provided; otherwise, it alerts the user to provide these details. Upon successful creation, it updates both groupList and filteredGroups state arrays with the new groupData, clears the input fields (groupName and selectedStudents), and logs a success message to the console. If all required fields are present, it constructs groupData object containing the groupName, selectedStudents array, and createdBy identifier. It then uses setDoc to set this data in the Firestore document located at groups/\${groupName}.

4.3.21 user can view groups

```
const fetchGroups = async () => {
  try {
    const querySnapshot = await getDocs(collection(db, "groups"));
    const groups = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));
    setGroupList(groups);
  } catch (error) {
    console.error("Error fetching groups:", error);
  }
};

fetchGroups();
```

Description:

The `useEffect` hook in this snippet fetches and initializes groups from a Firestore collection upon component mount. It defines `fetchGroups`, which initializes an empty array `groups` and attempts to fetch documents from `groups/`. Documents are processed using `getDocs`, iterating over `querySnapshot` to extract each document's data and ID, which are then stored in `groups`. The hook updates both `groupList` and `filteredGroups` state arrays with the fetched data. Error handling logs any fetch errors to the console. The hook runs once on component mount due to an empty dependency array, ensuring efficient data fetching without unnecessary re-runs.

4.3.22 user can start chatting

```
const handleSendMessage = async () => {
  if (newMessage.trim() === "") return;

  try {
    const messageData = {
      text: newMessage,
      senderId: currentUser.uid,
      senderRole: currentUser.role, // Add senderRole to message data
      timestamp: serverTimestamp(),
    };
    await addDoc(collection(db, `groups/${groupId}/messages`), messageData);
    console.log("Message sent:", messageData);
    setNewMessage("");
  } catch (error) {
    console.error("Error sending message: ", error);
  }
}
```

Description:

The `useEffect` hook in this code manages real-time message updates for a specific group identified by `groupId` from a Firestore collection. When `groupId` changes or initializes, it sets up a query (`q`) to fetch messages sorted by timestamp in ascending order (`orderBy("timestamp", "asc")`). It subscribes to updates using `onSnapshot`, which listens for changes in the Firestore collection `groups/${groupId}/messages`. Upon receiving a snapshot update, it constructs message objects from each document and updates the component's state with `setMessages`.

The `handleSendMessage` function handles sending a new message to the current group. It first checks if `newMessage` is not empty. If valid, it constructs `messageData` including the message text, sender ID, sender role, and timestamp. The message data is added to the Firestore collection `groups/${groupId}/messages` using `addDoc`, and upon successful sending,

4.3.23 teacher sends random question to student

```
const AddQuestionForRandomStudent = async () => {
  try {
    // Get the document reference for the class
    const classDocRef = doc(db, "createClass", className);
    const classDocSnapshot = await getDoc(classDocRef);

    if (classDocSnapshot.exists()) {
      const classData = classDocSnapshot.data();

      const _classes = { ...classData };
      _classes.ClassTeacherID && delete _classes.ClassTeacherID;
      console.log("helo palo ", _classes)
      // Get all student IDs within the current class
      const studentIds = Object.keys(_classes);

      if (studentIds.length === 0) {
        console.log(`No students found in class ${className}`);
        return;
      }
      // Choose a random student ID from the current class
      const randomStudentId =
        studentIds[Math.floor(Math.random() * studentIds.length)];

      // Update the 'popUp' field to true for the random student
      _classes[randomStudentId]["questionPopUp"] = true;
      _classes[randomStudentId]["question"] = question;

      // Update the document with the modified data
      await updateDoc(classDocRef, _classes);
      alert(
        `Question sent to a random student ${classData[randomStudentId].name} in class of
        ${className}`
      );
    }
  }
};
```

Description:

AddQuestionForRandomStudent selects a random student from a specified class in Firestore and updates their data to include a question and a boolean flag (questionPopUp). It begins by fetching the class document and verifying its existence. Student IDs are extracted from the class data, and a random ID is chosen. The function updates the selected student's data with the question and sets questionPopUp to true. After updating the Firestore document, it alerts the user and logs confirmation messages. Error handling is implemented to manage any issues encountered during data retrieval or update processes

4.3.24 Teacher sends random attendance to student

```
const updatePopUpForRandomStudent = async () => {
  debugger;
  try {
    // Get the document reference for the class
    const classDocRef = doc(db, "createClass", className);
    const classDocSnapshot = await getDoc(classDocRef);

    if (classDocSnapshot.exists()) {
      const classData = classDocSnapshot.data();
      const _classes = { ...classData };
      _classes.ClassTeacherID && delete _classes.ClassTeacherID;
      // Get all student IDs within the current class
      const studentIds = Object.keys(_classes);

      if (studentIds.length === 0) {
        console.log(`No students found in class ${className}`);
        return;
      }
      // Choose a random student ID from the current class
      const randomStudentId =
        studentIds[Math.floor(Math.random() * studentIds.length)];

      // Update the 'popUp' field to true for the random student
      _classes[randomStudentId]["popUp"] = true;
      // Update the document with the modified data
      await updateDoc(classDocRef, _classes);

      alert(
        `popUp boolean updated to true for a random student ${classData[randomStudentId].namee}
        in class of ${className}`
      );
    }
  }
};
```

Description:

updatePopUpForRandomStudent function fetches a class document from Firestore based on className. It checks if the document exists and retrieves class data, excluding ClassTeacherID. Student IDs are extracted, and a random student ID is selected. The function updates the popUp field to true for the random student within _classes. It then updates the Firestore document classDocRef with the modified _classes data using updateDoc. After successful update, it alerts the user and logs confirmation messages. Error handling is included to manage potential errors during data retrieval or update processes.

Chapter 5

Software Testing

5.1 Testing Methodology

After implementation, the process flow manager is tested for functional errors. We are going to do Black Box Testing (by passing random selected values and mapping it against the expected output in a normal flow), Unit and Integration Testing which is the testing of the functional requirements implemented in our system without regard to code. The test cases are done manually without the use of any tool.

5.2 Test Cases

5.2.1 Teacher Test Cases:

5.2.1.1 login

Valid Input:

Email: kainat2001@gmail.com Password: kainat786

Test Case Description:

The “login” test case evaluates the functionality of login into a system E classroom. It assesses whether the teacher can successfully login into a system or not.

Table 22: test-case 01 teacher login

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher login successfully	Test ID:1
Version:1	Test Type: Black box
Input:	
Email= kainat2001@gmail.com	
Password= kainat786	
Expected Result: Login successfully	
Actual Result: passed	

Table 23: test-case 02 teacher login

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher login successfully	Test ID:2
Version:1	Test Type: Black box
Input: Email= kainat2001gmail.com Password= kainat786	
Expected Result: Login successfully	
Actual Result: Pass	

5.2.1.2 Register

Test Case Description:

The “register” test case evaluates the functionality of user into a system E classroom. It assesses whether the teacher can successfully register into a system or not.

Table 24: test-case 03 teacher register

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher register successfully	Test ID:3
Version:1	Test Type: Black box
Input: Email= hiraatif45@gmail.com Password= hira123	
Expected Result: register successfully	
Actual Result: passed (genre not selected)	

5.2.1.3 logout:

Test Case Description:

The “logout” test case evaluates the functionality of logout from a system E classroom. It assesses whether the teacher can successfully logout from a system or not.

Table 25: test-case 04 teacher logout

Date: 10 Jan 2024	
System: E-Classroom	
Objective: Logout successfully	Test ID:4
Version:1	Test Type: Black box
Input: Logout Button = Clicked	
Expected Result: Logout successfully	
Actual Result: passed	

5.2.1.4 Update Profile

Test Case Description

The “Update profile” test case evaluates the functionality of teacher updating their profile. It assesses whether the teacher can successfully update their profile or not.

Table 26:test-case 05 teacher update profile

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher profile update	Test ID:5
Version:1	Test Type: Black box
<i>Input:</i> 1.Update profile Button = Clicked2.name button = clicked 3. designation = clicked 4. qualification = clicked	
<i>Expected Result:</i> profile updated successfully	
<i>Actual Result:</i> passed	

Table 27: test-case 06 teacher update profile

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher profile update	Test ID:6
Version:1	Test Type: Black box
Input: 1.none of the field changed	
Expected Result: profile updated successfully	
Actual Result: failed	

5.2.1.6 Create Classroom:

Test Case Description:

The “create classroom” test case evaluates the functionality of creating classroom for students in E-classroom platform. It assesses whether the teacher can successfully create class or not.

Table 28: test-case 07 teacher create classroom

Date: 10 Jan 2024	
System: E-Classroom	
Objective: classroom created successfully	Test ID:7
Version:1	Test Type: Black box
Input: 1.Create classroom Button = Clicked2. select student button = clicked 3. give name to class = clicked 4. select time button = clicked 5. create button = clicked	
Expected Result: classroom created successfully	
Actual Result: passed	

Table 29: test-case 08 teacher create classroom

Date: 10 Jan 2024	
System: E-Classroom	
Objective: classroom created successfully	Test ID:8
Version:1	Test Type: Black box
<i>Input:</i> 1.Create classroom Button = Clicked 3. give name to class = clicked 4. select time button = clicked 5. create button = clicked	
<i>Expected Result:</i> classroom created successfully	
<i>Actual Result:</i> failed (no student selected)	

5.2.1.7 Created Classroom

Test Case Description:

The “created classroom” test case evaluates the functionality of showing the created classroom which was made by teachers earlier. It assesses whether the teacher can successfully see their created classes or not.

Table 30: test-case 09 teacher can see created classroom

Date: 10 Jan 2024	
System: E-Classroom	
Objective: all the created classroom showed successfully	Test ID: 09
Version:1	Test Type: Black box
<i>Input:</i> 1.Created classroom Button = Clicked	
<i>Expected Result:</i> all the created classroom showed successfully	
<i>Actual Result:</i> passed	

5.2.1.8 Teacher Start Class

Test Case Description:

The “start class” test case evaluates the functionality of teacher starting the class. It assesses whether the teacher can successfully start the class or not.

Table 31: test-case 10 teacher start class

Date: 10 April 2024	
System: E-Classroom	
Objective: teacher can start class	Test ID:10
Version:1	Test Type: Black box
<i>Input:</i> Select class = clicked Start class button = clicked	
<i>Expected Result: Class started</i>	
<i>Actual Result: Passed</i>	

5.2.1.9 Upload Assignment Question

Test Case Description:

The “upload assignment” test case evaluates the functionality of teacher uploading the assignments for students. It assesses whether the teacher can successfully upload the assignment or not.

Table 32: test-case 11 teacher upload assignment question

Date: 10 April 2024	
System: E-Classroom	
Objective: Teacher can create assignment (uploadassignment question)	Test ID:11
Version:1	Test Type: Black box
<i>Input:</i> Upload button = clicked Select file button = clicked Select file from My PC = clicked Done button = clicked	
<i>Expected Result: Assignment uploaded</i>	
<i>Actual Result: Passed</i>	

5.2.1.10 Teacher Can Mute Microphone

Test Case Description:

The “Mute microphone” test case evaluates the functionality of teacher toggles their microphone. It assesses whether the teacher can successfully toggle their microphone or not.

Table 33: test-case 12 teacher can mute microphone

Date: 10 April 2024	
System: E-Classroom	
Objective: teacher can mute microphone	Test ID:12
Version:1	Test Type: Black box
Input: Join call button = clicked Mic button = clicked	
Expected Result: Mic Muted	
Actual Result: Passed	

5.2.1.11 Teacher Can Toggle his/her Camera

Test Case Description:

The “Toggle camera” test case evaluates the functionality of teacher toggles their camera. It assesses whether the teacher can successfully toggle their camera or not.

Table 34: test-case 13 teacher can toggle camera

Date: 10 April 2024	
System: E-Classroom	
Objective: teacher can turn on/off camera	Test ID:13
Version:1	Test Type: Black box
Input: Select class Join call button = clicked camera icon = clicked	
Expected Result: teacher's camera is turned on/off successfully	
Actual Result: Passed	

5.2.1.12 Teacher Can Toggle Screen

Test Case Description:

The “Toggle screen” test case evaluates the functionality of teacher toggles their screen. It assesses whether the teacher can successfully toggle their screen or not.

Table 35: test-case 14 teacher can toggle screen

Date: 10 April 2024	
System: E-Classroom	
Objective: teacher can share his/her screen	Test ID:14
Version:1	Test Type: Black box
Input: Select class Join call button = clicked Share screen icon = clicked	
Expected Result: teacher's screen is shared successfully	
Actual Result: Passed	

5.2.1.13 Teacher Can Exit Class

Test Case Description:

The “Toggle camera” test case evaluates the functionality of teacher toggles their camera. It assesses whether the teacher can successfully toggle their camera or not.

Table 36: test-case 15 teacher can exit class

Date: 10 April 2024	
System: E-Classroom	
Objective: teacher can exit class	Test ID:15
Version:1	Test Type: Black box
Input: Select class Join call button = clicked exit icon = clicked	
Expected Result: teacher leaves the class successfully	
Actual Result: Passed	

5.2.1.14 Teacher Can Download Assignment Solution

Test Case Description:

The “Download assignment” test case evaluates the functionality of teacher downloading the assignments of students. It assesses whether the teacher can successfully download the assignments or not.

Table 37: test-case 16 teacher can download assignment solution

Date: 10 Jan 2024	
System: E-Classroom	
Objective: teacher can download student assignment	Test ID:16
Version:1	Test Type: Black box
Input: Assignment button = clicked Download Icon = clicked	
Expected Result: Assignment downloaded	
Actual Result: Passed	

5.2.1.15 Teacher Create Groups

Test Case Description:

The “Create Groups” test case evaluates the functionality of teacher creating the groups of students during the class. It assesses whether the teacher can successfully create students’ groups or not.

Table 38: test-case 17 teacher can create groups

Date: 12 July 2024	
System: E-Classroom	
Objective: teacher create groups of students	Test ID:17
Version:1	Test Type: Black box
Input: Group chat button = clicked Select students Select group name Create group = clicked All groups created will be shown	
Expected Result: groups created successfully	
Actual Result: Passed	

5.2.1.16 Teacher Type and View Group Chats

Test Case Description:

The “View and type in group chats” test case evaluates the functionality of teacher communicating with students during class in a group chat. It assesses whether the teacher can successfully view and type in group or not.

Table 39: test-case 18 teacher can type can view group chats

Date: 12 July 2024	
System: E-Classroom	
Objective: teacher view group chats	Test ID:18
Version:1	Test Type: Black box
Input: Select group by name Open chat button = clicked (selected group)View and type message	
Expected Result: teacher view group chats successfully	
Actual Result: Passed	

5.2.1.17 Teacher Can Remove Students

Test Case Description:

The “Remove students” test case evaluates the functionality of teacher removing the students from class. It assesses whether the teacher can successfully remove the students from class or not.

Table 40: test-case 19 teacher can remove students

Date: 12 July 2024	
System: E-Classroom	
Objective: teacher remove students	Test ID:19
Version:1	Test Type: Black box
Input: Select class Check students' button = clickedRemove icon (delete) = clicked Student removed from class	
Expected Result: Student removed from class successfully	
Actual Result: Passed	

5.2.1.18 Teacher Send Random Attendance

Test Case Description:

The “Random Attendance” test evaluates the functionality of sending random attendance popups to students. It assesses whether teacher can successfully send the attendance popups or not.

Table 41: test-case 20 teacher send random attendance to students

Date: 12 July 2024	
System: E-Classroom	
Objective: attendance send randomly to students	Test ID:20
Version:1	Test Type: Black box
Input: Select class Attendance button = clicked System will pick random student from class and send popup to them	
Expected Result: attendance send randomly to students successfully	
Actual Result: Passed	

5.2.1.19 Teacher View Marked Attendance

Test Case Description:

The “View mark attendance” test case evaluates the functionality of teacher viewing students marked attendance. It assesses whether the teacher can successfully view attendance or not.

Table 42: test-case 21 teacher can view marked attendance of the students

Date: 12 July 2024	
System: E-Classroom	
Objective: teacher can view the student’s submitted attendance	Test ID:21
Version:1	Test Type: Black box
Input: Select class Check students’ button = clicked Attendance (present/absent) will be appeared against student name.	
Expected Result: Answers of students viewed successfully	
Actual Result: Passed	

5.2.1.20 teacher start random question

Test Case Description:

The “Random Question” test evaluates the functionality of sending random question popups to students. It assesses whether teacher can successfully send the question popups or not.

Table 43: test-case 22 teacher start random questions

Date: 12 July 2024	
System: E-Classroom	
Objective questions send randomly to students	Test ID:22
Version:1	Test Type: Black box
<i>Input:</i> Select class Questions button = clicked Pop up appear, teacher will type question in a dialogue box Submit question button = clicked System will pick random student from class and send popup question to them	
<i>Expected Result: Questions send randomly to students successfully</i>	
<i>Actual Result: Passed</i>	

5.2.1.21 teacher can view submitted answers

Test Case Description:

The “View Submitted Ans” test case evaluates the functionality of teacher viewing students submitted answers. It assesses whether the teacher can successfully view students submitted answers or not.

Table 44: test-case 23 teacher can view submitted answers of the students

Date: 12 July 2024	
System: E-Classroom	
Objective: teacher can view the student's submitted answer	Test ID:23
Version:1	Test Type: Black box
<i>Input:</i> Select class Check students' button = clicked Questions along with the answer will be appeared against student name.	
<i>Expected Result: Answers of students viewed successfully</i>	
<i>Actual Result: Passed</i>	

5.2.2 Student Test Cases

5.2.2.1 login

Valid Input:

Email= moiz@gmail.com

Password= 12345

Test Case Description:

The “login” test case evaluates the functionality of login into a system E classroom. It assesses whether the student can successfully login into a system or not.

Table 45: test-case 24 student login into system

Date: 13 Jan 2024	
System: E-Classroom	
Objective: student login successfully	Test ID: 24
Version:1	Test Type: Black box
Input:	
Email= moizgmail.com	
Password= 123456	
Expected Result: Login successfully	
Actual Result: passed	

Table 46: test-case 25 student login into system

Date: 13 Jan 2024
System: E-Classroom
Objective: Student login successfully
Version:1
Input:
Email= moiz@gmail.com
Password= 123456
Expected Result: Login successfully
Actual Result: passed

5.2.2.2 Register

Test Case Description:

The “register” test case evaluates the functionality of user into a system E classroom. It assesses whether the student can successfully register into a system or not.

Table 47: test-case 26 student register

Date: 13 Jan 2024	
System: E-Classroom	
Objective: student register successfully	Test ID:26
Version:1	Test Type: Black box
<i>Input:</i> Select Genre = StudentEmail= moiz@gmail.com Password= 123456	
<i>Expected Result: Register successfully</i>	
<i>Actual Result: passed</i>	

Table 48: test-case 27 student register

Date: 13 Jan 2024	
System: E-Classroom	
Objective: student register successfully	Test ID:27
Version:1	Test Type: Black box
<i>Input:</i> Select Genre = not selected Email= moiz@gmail.com Password= 123456	
<i>Expected Result: Register successfully</i>	
<i>Actual Result: failed</i>	

5.2.2.3 Logout

Test Case Description:

The “logout” test case evaluates the functionality of logout from a system E classroom. It assesses whether the teacher can successfully logout from a system or not.

Table 49: test-case 28 student logout

Date: 13 Jan 2024	
System: E-Classroom	
Objective: student Logout successfully	Test ID: 28
Version:1	Test Type: Black box
Input: Logout Button = Clicked	
Expected Result: Logout successfully	
Actual Result: passed	

5.2.2.4 Update Profile

Test Case Description:

The “Update profile” test case evaluates the functionality of teacher updating their profile. It assesses whether the teacher can successfully update their profile or not.

Table 50: test-case 29 student update profile

Date: 10 Jan 2024	
System: E-Classroom	
Objective: student profile update	Test ID:29
Version:1	Test Type: Black box
1. Update profile Button = click 2.name button = clicked 3.regno = clicked 4. semester = clicked	
Expected Result: profile updated successfully	
Actual Result: passed	

5.2.2.5 Notification

Test Case Description:

The “Notification” test case evaluates the functionality of student receiving notification 5 minutes prior to the class. It assesses whether the student can successfully receive the notification or not.

Table 51: test-case 30 student receive notification

Date: 13 Jan 2024	
System: E-Classroom	
Objective: student receive notification successfully	Test ID:30
Version:1	Test Type: Black box
Input: Class Name = “Compiler”Class Time = “2:00 PM” Add Students = “Hira, Moiz, Allauddin” CheckCurrentTime() == CalculateNotificationTime() :: SendNotification()	
Expected Result: notified successfully	
Actual Result: passed	

Table 52: test-case 31 student receive notification

Objective: student did not receive notification	Test ID: 31
Version:1	Test Type: Black box
Input: ClassName = “Compiler” ClassTime = “” AddStudents = “Hira, Moiz, Allauddin” CheckCurrentTime() == CalculateNotificationTime() :: SendNotification()	
Expected Result: notified successfully	
Actual Result: Failed	

5.2.2.6 Student Start Class

Test Case Description

The “start class” test case evaluates the functionality of student starting the class. It assesses whether the student can successfully start the class or not.

Table 53: test-case 32 student start class

Date: 10 April 2024	
System: E-Classroom	
Objective: student can start class	Test ID:32
Version:1	Test Type: Black box
Input: Select class = clicked Start class button = clicked	
Expected Result: Class started	
Actual Result: Passed	

5.2.2.7 Student Join Class

Test Case Description:

The “Join class” test case evaluates the functionality of student joining the class. It assesses whether the student can successfully join class or not.

Table 54: test-case 33 student join class

Date: 10 April 2024	
System: E-Classroom	
Objective: Student can join classroom	Test ID: 33
Version:1	Test Type: Black box
<i>Input:</i> Select subject = clicked Join button = clicked	
<i>Expected Result: student joined class</i>	
<i>Actual Result: passed</i>	

5.2.2.8 Student Mute Microphone

Test Case Description:

The “Mute microphone” test case evaluates the functionality of student toggles their microphone. It assesses whether the student can successfully toggle their microphone or not.

Table 55: test-case 34 student can mute microphone

Date: 10 April 2024	
System: E-Classroom	
Objective: student can mute microphone	Test ID:34
Version:1	Test Type: Black box
Input: Join call button = clicked Mic button = clicked	
Expected Result: Mic Muted	
Actual Result: Passed	

5.2.2.9 Student Toggle Camera

Test Case Description:

The “Toggle camera” test case evaluates the functionality of student toggles their camera. It assesses whether the student can successfully toggle their camera or not.

Table 56: test-case 35 student can toggle their camera during class

Date: 10 April 2024	
System: E-Classroom	
Objective: student can turn on/off camera	Test ID: 35
Version:1	Test Type: Black box
Input: Select class Join call button = clickedcamera icon = clicked	
Expected Result: student’s camera is turned on/off successfully	
Actual Result: Passed	

5.2.2.10 Student Can Toggle Screen

Test Case Description:

The “Toggle screen” test case evaluates the functionality of student toggles their screen. It assesses whether the student can successfully toggle their screen or not.

Table 57: test-case 36 student can toggle screen

Date: 10 April 2024	
System: E-Classroom	
Objective: student can share his/her screen	Test ID: 36
Version:1	Test Type: Black box
Input: Select class Join call button = clicked Share screen icon = clicked	
Expected Result: student's screen shared successfully	
Actual Result: Passed	

5.2.2.11 Student Can Exit Classroom

Test Case Description:

The “Toggle camera” test case evaluates the functionality of student toggles their camera. It assesses whether the student can successfully toggle their camera or not.

Table 58: test-case 37 students can exit classroom

Date: 10 April 2024	
System: E-Classroom	
Objective: student can exit class	Test ID: 37
Version:1	Test Type: Black box
Input: Select class Join call button = clicked exit icon = clicked	
Expected Result: student leaves the class successfully	
Actual Result: Passed	

5.2.2.12 Student Download Assignment Question

Test Case Description:

The “Student download assignment” test case evaluates the functionality of student downloading the uploaded assignment. It assesses whether the student can successfully download assignment or not.

Table 59: test-case 38 student can download assignment question

Date: 10 Jan 2024	
System: E-Classroom	
Objective: student download assignment question	Test ID: 38
Version:1	Test Type: Black box
Input: Assignment button = clicked Download Icon = clicked	
Expected Result: Assignment downloaded	
Actual Result: Passed	

5.2.2.13 Student Upload Assignment Solution

Test Case Description:

The “upload assignment” test case evaluates the functionality of student upload the assignments. It assesses whether the student can successfully upload the assignments or not

Table 60: test-case 39 student can upload assignment solution

Date: 10 April 2024	
System: E-Classroom	
Objective: Student can upload assignment	Test ID: 39
Version:1	Test Type: Black box
Input: Upload button = clicked Select file button = clicked Select file from My PC = clicked Done button = clicked	
Expected Result	
Actual Result: passed	

5.2.2.14 Student Type and View Group Chat

Test Case Description:

The “View and type in group chats” test case evaluates the functionality of student communicating with teacher in a group chat. It assesses whether the student can successfully view and type in group or not.

Table 61: test-case 40 student can type and view group chat

Date: 12 July 2024	
System: E-Classroom	
Objective: Student view group chat	Test ID: 40
Version:1	Test Type: Black box
<i>Input:</i> Group chat button = clicked Select group in which you’re added by teacher View and type in group chat	
<i>Expected Result: student can view and start conversation in the chat successfully</i>	
<i>Actual Result: passed</i>	

5.2.2.15 Student Can Create Notes

Test Case Description:

The “Create note” test case evaluates the functionality of student creating notes. It assesses whether the student can successfully create notes or not.

Table 62: test-case 41 student can create notes

Date: 12 July 2024	
System: E-Classroom	
Objective: Student can manage their notes	Test ID: 41
Version:1	Test Type: Black box
<i>Input:</i> Manage notes button = clicked Create notes by typing or by adding file from browser. Upload note button= created Created notes by student will be shown beneath	
<i>Expected Result: student created notes successfully</i>	
<i>Actual Result: passed</i>	

5.2.2.16 Student View Notes

Test Case Description:

The “View notes” test case evaluates the functionality of student viewing their notes. It assesses whether the student can successfully view their notes or not

Table 63: test-case 42 student can view their created notes

Date: 12 July 2024	
System: E-Classroom	
Objective: Student can view their notes	Test ID: 42
Version:1	Test Type: Black box
Input: Manage notes button = clicked All the notes created by the student will be shown	
Expected Result: student viewed notes successfully	
Actual Result: passed	

5.2.2.17 Student Can Delete Notes

Test Case Description:

The “Delete Notes” test case evaluates the functionality of student deleting their created notes. It assesses whether the student can successfully delete the notes or not

Table 64: test-case 43 student can delete notes

Date: 12 July 2024	
System: E-Classroom	
Objective: Student can delete their notes	Test ID: 43
Version:1	Test Type: Black box
Input: Manage notes button = clicked Delete button = clicked (specific notes) Popup appear with acknowledgement of notes deleted successfully	
Expected Result: notes deleted successfully	
Actual Result: passed	

5.2.2.18 Student Receive Random Attendance Popup

Test Case Description:

The “Receive Random Attendance” test case evaluates the functionality of student receiving the random attendance popups during the class. It assesses whether the student can successfully receive attendance popups or not.

Table 65: test-case 44 student receive random attendance popup

Date: 12 July 2024	
System: E-Classroom	
Objective: Student receive attendance popup	Test ID: 44
Version:1	Test Type: Black box
<i>Input:</i> Select class Start class button= clicked Student entered in classroom Popup appears with attendance Present button = clicked (if present) Popup will appear on the screen for less than a minute	
<i>Expected Result: random attendance pop appeared in student's UI successfully</i>	
<i>Actual Result: passed</i>	

5.2.219 Student Receive Random Question Popup

Test Case Description:

The “Receive Random Attendance” test case evaluates the functionality of student receiving the random question popups from teachers during the class. It assesses whether the student can successfully receive question popups or not.

Table 66: test-case 45 student receive random question popup

Date: 12 July 2024	
System: E-Classroom	
Objective: Student receive question popup	Test ID: 45
Version:1	Test Type: Black box
<i>Input:</i> Select class Start class button= clicked Student entered in classroomPopup appears with questionType answer of that questionsubmit button = clicked Popup will appear on the screen for less than a minute	
<i>Expected Result: student received and answered question popup successfully</i>	
<i>Actual Result: passed</i>	

Chapter 6

Software Deployment

Chapter 6 of the E-Classroom project delves into the phase of software deployment. Deployment is a critical phase in software development where the developed application is made available for users to access and interact with. This process involves configuring the environment, setting up servers, and ensuring the application is accessible through a specific domain. Effective deployment ensures the application runs smoothly and is scalable to handle varying user loads.

Software deployment involves several key stages:

1. **Preparation:** This stage involves getting the application ready for deployment. It includes building the application, running tests, and preparing configuration files.
2. **Environment Configuration:** The deployment environment needs to be configured. This includes setting up servers, databases, and any other necessary infrastructure. Environment variables and configuration settings specific to the deployment environment are also set up during this stage.
3. **Deployment:** This is the actual process of transferring the application from the development environment to the production environment. It involves uploading files, setting up services, and ensuring all components are properly integrated.
4. **Testing:** Post-deployment testing ensures that the application is functioning correctly in the production environment. This includes checking the functionality, performance, and security of the application.
5. **Monitoring:** Once the application is live, it is continuously monitored to ensure it is running smoothly. Any issues that arise are promptly addressed.

6.1. Installation / Deployment Process Description

With the domain e-classroom. Website secured, the user proceeded to set up both the frontend and backend of the project using Vercel. Vercel is a platform renowned for its ease of use and powerful features for deploying modern web applications.

Setting Up the React.js Frontend on
Vercel
Installed Vercel CLI:

Ensured Node.js was installed: The user verified that Node.js, a prerequisite for running Vercel CLI, was installed on their system.

Installed the Vercel CLI globally: Using the terminal, the user installed the Vercel CLI globally with the command: `npm install -g vercel`

Logged into Vercel:

Logged into the Vercel account: The user authenticated their Vercel account via the CLI by executing: `vercel login`

Initialized the React Project:

Navigated to the React project directory: The user moved to the directory containing their React project. Initialized the project with Vercel: The user ran the command: `vercel`. This command prompted a series of questions regarding the project setup, which the user answered to configure the deployment.

Configured Project Settings:

Ensured proper build and start scripts: The user checked the package.json file to confirm it contained the necessary scripts for building and starting the React application.

Adjusted build settings in the Vercel dashboard: If needed, the user made further adjustments to the build settings via the Vercel dashboard to optimize the deployment process.

Deployed the React Project:

Deployed the project: The user deployed the React application using: `vercel --prod`

Project live on Vercel URL: The React frontend became live on a Vercel-provided URL, which was then customized to use the e-classroom website domain.

Setting Up the Firebase Backend Initialized Firebase in the Project:

Installed Firebase CLI: The user installed the Firebase CLI globally using the command: `npm install -g firebase-tools`

Logged into Firebase: The user authenticated their Firebase account via the CLI by executing: `firebase login`

Initialized Firebase: The user initialized Firebase in their project directory using: `firebase init`. This command prompted a series of questions regarding the Firebase setup, which the user answered to configure the deployment.

Configured Firebase Settings:

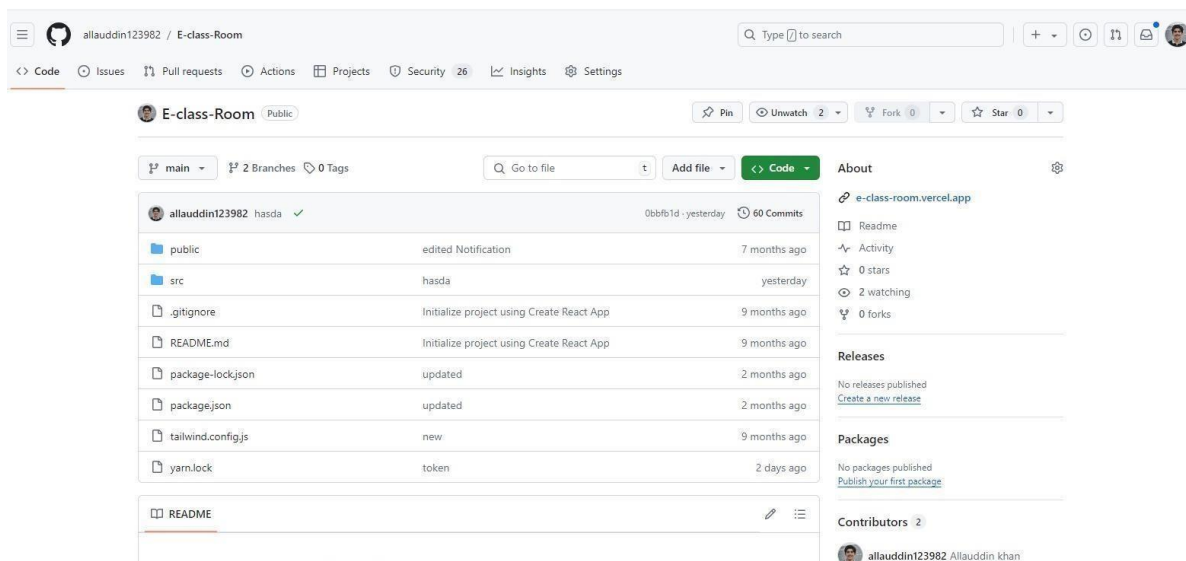
Set up Firebase project: The user selected or created a Firebase project for the application.

Configured Firebase services: The user configured the required Firebase services such as Firestore, Authentication, and Hosting as needed.

Deployed the Firebase Backend:

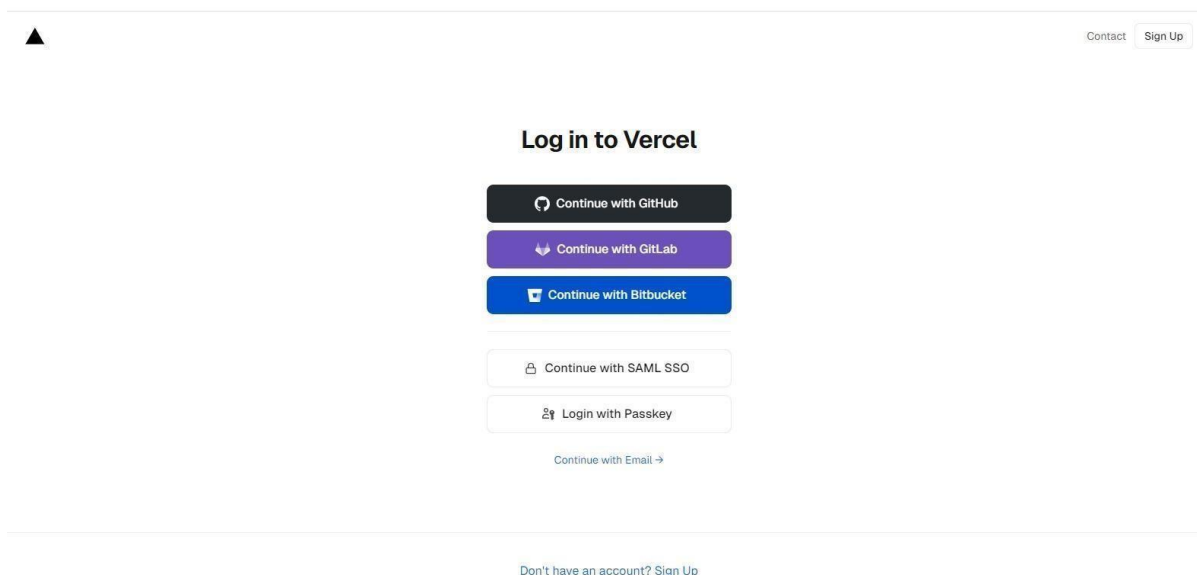
Deployed Firebase services: The user deployed the Firebase backend using: `firebase deploy`

Verified backend integration: After deployment, the user tested the backend services to ensure proper integration with the frontend.



Make a GitHub repository and Upload your project

Figure 46: GitHub repository and project upload



Login to vercel from the same GitHub account

Figure 47: vercel login

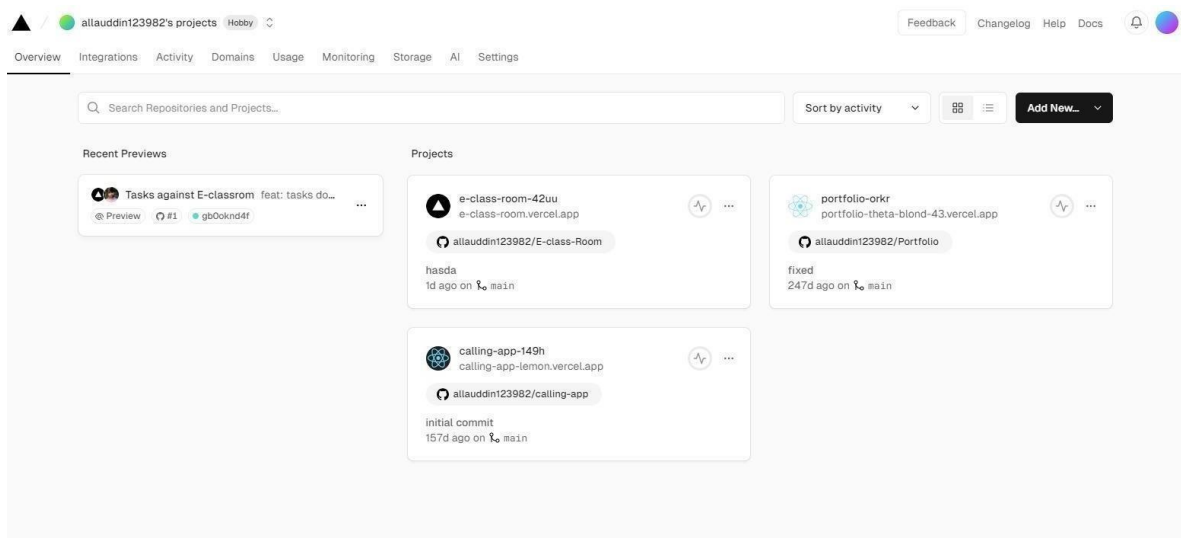


Figure 48: vercel project upload

click on Add new and then project

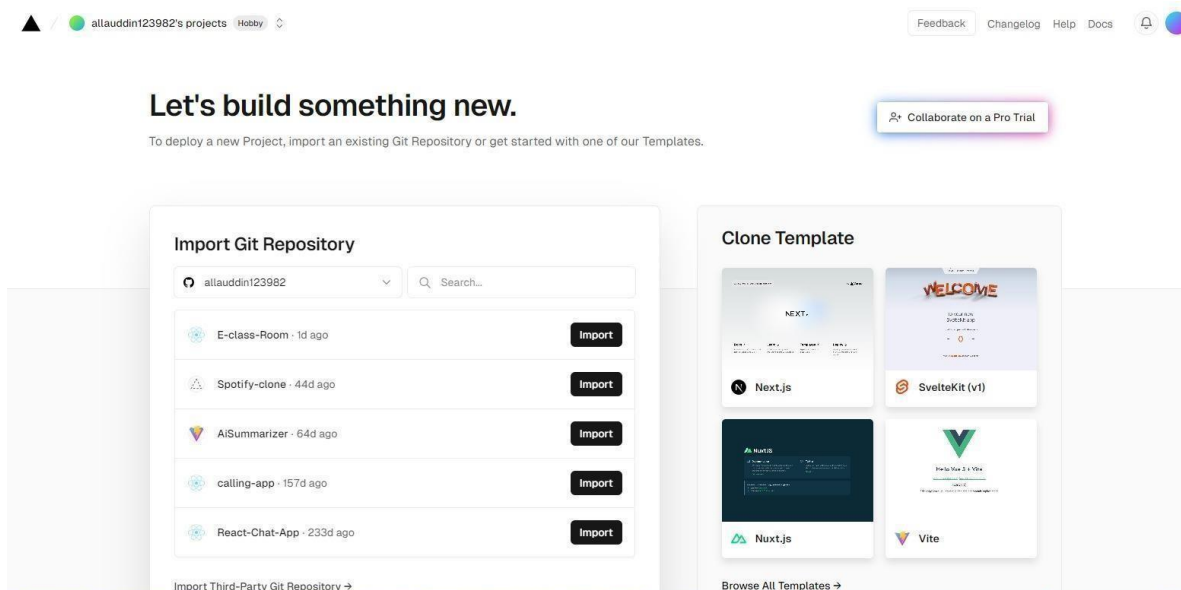


Figure 49: import repository

Import your repository from GitHub

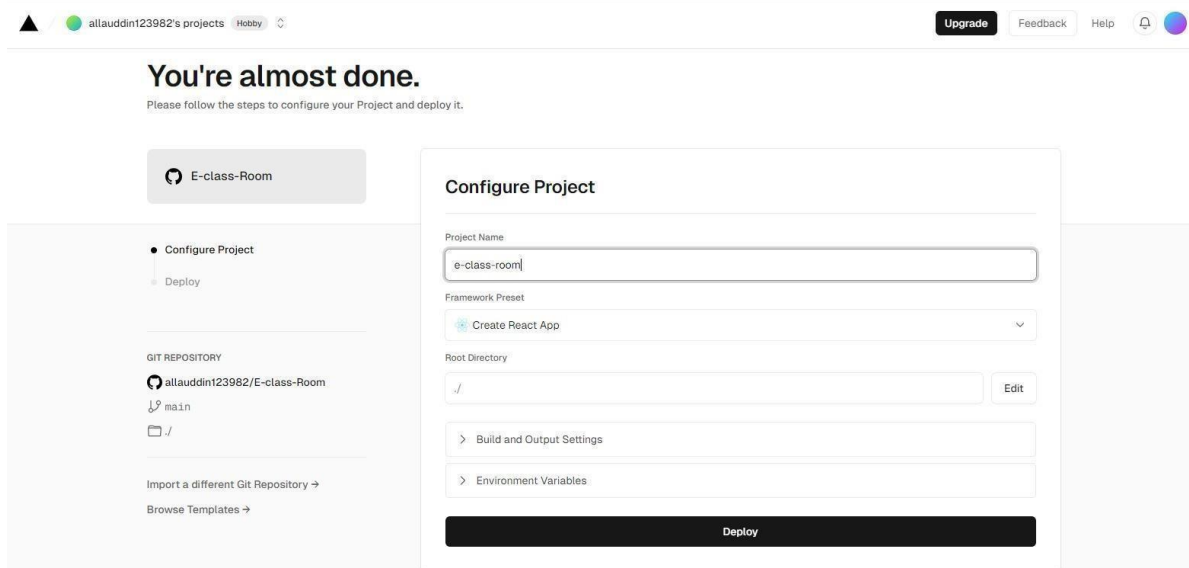


Figure 50: configuration and deployment

add configurations and click on deploy

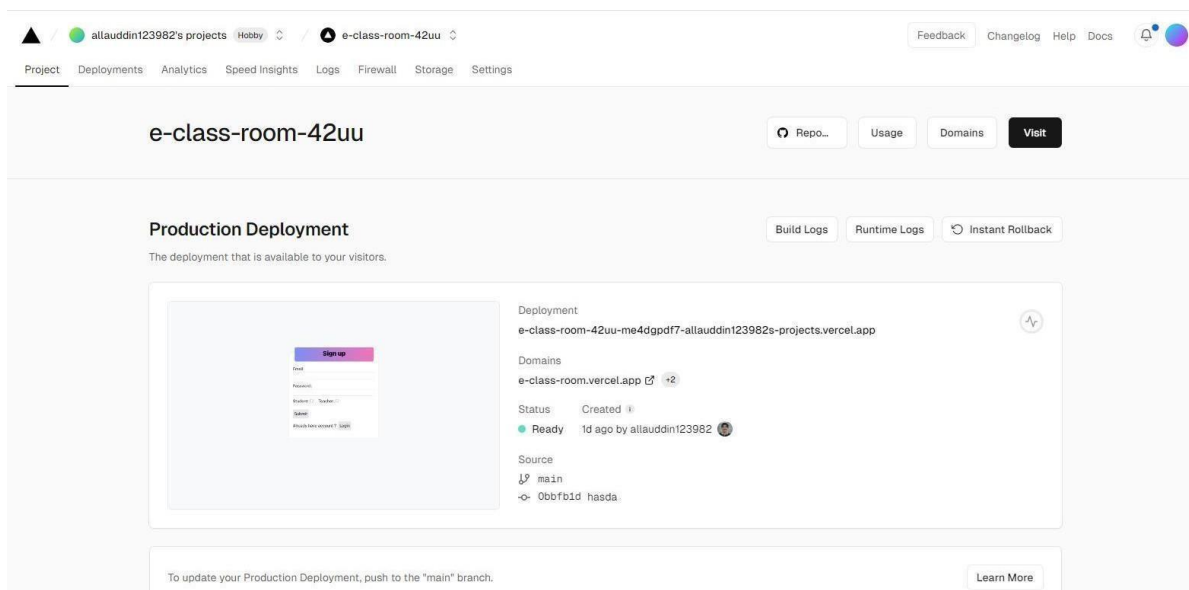


Figure 51: website live for end users

finally, the website is live for the end users

6.2. Conclusion:

The process of deploying the project involved several critical steps, each contributing to the seamless deployment of both the frontend and backend components. The user successfully purchased and registered the domain `e-classroom.website` from Hostinger.pk, providing a unique and professional address for the project.

Following this, the user leveraged Vercel's powerful platform to deploy the React.js frontend and Firebase backend. The steps included installing necessary tools, initializing projects, configuring settings, and deploying both components. Finally, linking the custom domain ensured the project was accessible through `e-classroom.website`.

This thorough deployment process not only established a strong online presence for the project but also utilized modern deployment practices to ensure scalability, reliability, and ease of access. The use of Vercel's platform streamlined the process, providing robust hosting solutions for both the frontend and backend, thereby setting a solid foundation for future growth and development.

Chapter 7

Project Evaluation

7.1. Project Evaluation Report

The report of the project “E-Classroom” has been approved based on the following guidelines:

Table 67: artifacts guidelines

Artifacts Guidelines	Remarks
Analysis and Design artifacts are syntactically correct (Use-case model, SSDs, Domain model, Class diagram, SD, ERD).	
Consistency and traceability have been maintained among different artifacts	
General Guidelines	
Formatting (font-style, indentation) is according to the FYP template and consistent throughout the document.	
Captions are added to the figures and tables. Figure captions must be placed below each figure, and table captions must be provided above the table.	
Each figure or table is followed by some text describing what it represents.	

Dr. Muhammad Siraj Rathore
(Supervisor)

Mr. Salman Ahmed
(Examiner 1)

Ms. Memmona Bilal
(Examiner 2)

Mr. Abdul Haseeb
(Examiner 3)

References

- [1]. ReactJS Team 2023, 'ReactJS Documentation (Legacy)', ReactJS, accessed 05 May 2023, <https://legacy.reactjs.org/docs/getting-started.html>.
- [2]. Firebase Team 2023, 'Firebase Documentation', Firebase, accessed 16 July 2024, <https://firebase.google.com/docs>
- [3]. Agora Team 2023, 'Agora Documentation', Agora, accessed 16 July 2024, <https://docs.agora.io/en>.

